

Architektury systemów agentowych

Od modelu generatywnego do systemu ukierunkowanego na cel

DLA KOGO JEST TA KSIĄŻKA

Początkujący, którzy chcą najpierw zrozumieć strukturę, zanim wybiorą specjalizację. Książka jest przeznaczona dla samouków, studentów, czytelników technicznych i praktyków, którzy chcą zrozumieć, jak składa się systemy agentowe, bez wcześniejszego wiązania się z konkretnym frameworkiem lub stosem produkcyjnym.

ZAKRES

Ta książka jest podręcznikiem koncepcyjnym poświęconym architektonicznemu kręgosłupowi systemów agentowych: temu, jak model generatywny staje się ograniczonym systemem ukierunkowanym na cel dzięki jawnym celom, pętlom sterowania, stanowi, pamięci, planowaniu, wyborowi działań, interfejsom narzędziowym, groundingowi, informacji zwrotnej, refleksji, zatrzymaniu i przekazaniu. Głównym przykładem prowadzonym przez całą książkę jest ograniczony asystent do przeglądu literatury, a dobór architektury jest tu traktowany jako pochodna profilu zadania.

Szczegółowy spis treści

Część I — Podstawy architektury

Ustanawia różnicę między modelem a agentem, a następnie buduje reprezentacyjny kręgosłup celów, stanu, kontekstu i pamięci.

1. Rozdział 1 — Od modelu generatywnego do systemu agentowego

Odróżnia sam model generatywny od systemów interaktywnych, skryptowych przepływów pracy i pętli agentowych, używając asystenta do przeglądu literatury jako stałego punktu odniesienia.

2. Rozdział 2 — Cele, specyfikacje zadań i kryteria sukcesu

Zamienia prośby w operacyjne specyfikacje zadań, które mogą sterować kontrolą, oceną i zatrzymaniem.

3. Rozdział 3 — Pętle, stan, kontekst i ślady wykonania

Pokazuje rozmieszczenie informacji w czasie i wyjaśnia, dlaczego kontekst nie jest trwałym stanem.

4. Rozdział 4 — Typy pamięci, trwałość i polityki

Traktuje pamięć jako reprezentację plus politykę, a nie jako ogólną bazę danych czy większe okno kontekstu.

Część II — Deliberacja, działanie i regulacja

Pokazuje, jak ograniczeni agenci wybierają działania, korzystają z narzędzi, planują, odbierają informację zwrotną i decydują, czy kontynuować.

1. Rozdział 5 — Wybór działań, korzystanie z narzędzi i sprzężenie ze środowiskiem

Wyjaśnia, jak agent wybiera między myśleniem, pytaniem, działaniem i zatrzymaniem oraz jak kontrakty narzędzi ograniczają zachowanie oparte na grounding.

2. Rozdział 6 — Planowanie, dekompozycja i ponowne planowanie

Przedstawia planowanie jako selektywną strategię sterowania nad przestrzenią działań, a nie jako synonim inteligencji.

3. Rozdział 7 — Grounding, informacja zwrotna, refleksja i analiza błędów strukturalnych

Pokazuje, jak kontrole niosące dowody, informacja zwrotna i pętle nadające się do przeprojektowania korygują porażki strukturalne.

4. Rozdział 8 — Zatrzymanie, doprecyzowanie, przekazanie i ograniczona autonomia

Domyka opowieść o sterowaniu przez zdefiniowanie uzasadnionych zakończeń, doprecyzowania, eskalacji i granic autonomii.

Część III — Ocena architektur i synteza

Przekształca wiedzę o komponentach w porównywanie, dopasowanie zadanie-architektura, granice zakresu i obroniony projekt końcowy.

1. Rozdział 9 — Porównywanie architektur według profilu zadania i zakresu

Porównuje kandydackie architektury według horyzontu, niepewności, potrzeb pamięciowych, zależności od narzędzi, bogactwa informacji zwrotnej i granicy autonomii.

2. Rozdział 10 — Projektowanie asystenta do przeglądu literatury

Składa od zera ograniczonego asystenta do przeglądu literatury, uzasadnia każdy komponent i broni projektu końcowego za pomocą śladu wykonania oraz rewizji.

Założenia i notacja

Założenia

- Czytelnik dobrze czuje się z abstrakcją, rozumowaniem w kategoriach „pudełek i strzałek” oraz śledzeniem procesów krok po kroku. Podstawowa znajomość programowania jest pomocna, ale nie jest wymagana.
- Reinforcement learning rozumiane jako uczenie polityki, wewnątrz mechanizmów retrieval, symboliczne języki planowania i solvery, wdrażanie, obserwowalność, governance, tutoriale frameworków oraz szczegółowe omówienia implementacji kodu pozostają poza zakresem tej książki.
- Asystent do przeglądu literatury jest traktowany jako ograniczony system informacyjny z koncepcyjnymi interfejsami wyszukiwania, czytania, notatek i sprawdzania cytowań, a nie jako produkt produkcyjny czy autonomiczna platforma badawcza.

Notacja stosowana w całej książce

Symbol	Znaczenie
$G = \langle \text{objective, constraints, success criteria, stop/handoff conditions} \rangle$	Pakiet specyfikacji zadania.
t	Indeks kroku w śladzie wykonania.
C_t	Bieżący kontekst modelu w kroku t .
S_t	Jawny stan zewnętrzny lub stan sterowania przenoszony między krokami.
O_t	Obserwacja lub wynik uzyskany w kroku t .
P	Bieżąca struktura planu.
M	Trwałe zasoby pamięci i związane z nimi polityki.
$A_t \in \{\text{think, ask, act, stop}\}$	Zwięzła notacja wyboru następnego kroku, gdy jest to użyteczne.

Część I — Podstawy architektury

Ustanawia różnicę między modelem a agentem, a następnie buduje reprezentacyjny kręgosłup celów, stanu, kontekstu i pamięci.

Rozdział 1 — Od modelu generatywnego do systemu agentowego

AA-S01 — Od modelu do agenta

Jakie wymagania ujawnia przykład przewodni

Czytelnik prosi o ograniczony przegląd literatury:

Zadanie przewodnie używane w całej książce

Przygotuj przegląd literatury na 1 200 słów o asystentach odpowiadania na pytania dla studentów, których odpowiedzi są oparte na cytowaniach, ograniczony do lat 2021-2024. Użyj 8-12 artykułów, jeśli to możliwe. Oddziel wzorce architektoniczne od twierdzeń ewaluacyjnych. Dodaj krótką macierz porównawczą i zatrzymaj się albo zapytaj, jeśli zakres jest niejednoznaczny lub materiał dowodowy jest zbyt słaby.

Silny model często potrafi od razu przygotować coś, co wygląda jak odpowiedź. Stały przepływ pracy też może wykonać sekwencję w rodzaju search → read → summarize. Oba podejścia bywają użyteczne. Żadne z nich samo w sobie nie gwarantuje jednak własności, która w tej książce jest najważniejsza: **zachowania ukierunkowanego na cel w czasie, w ramach jawnych ograniczeń.**

Odpowiedź oparta wyłącznie na promptcie może brzmieć gładko, a jednocześnie po cichu pominąć regułę zatrzymania, oczekiwania dotyczące cytowań albo granice zakresu. Skryptowy przepływ pracy może z kolei zachować sekwencję kroków, a mimo to zawieść wtedy, gdy pierwsze wyszukiwanie daje słaby materiał dowodowy i system powinien się dostosować. Wymaganie projektowe w tym rozdziale jest więc proste: **co trzeba dodać wokół modelu, aby sensownie nazwać cały system agentowym?**

Plan rozdziału

Ten rozdział ustanawia główną perspektywę całej książki. Model jest komponentem generatywnym. System agentowy to zorganizowana całość zbudowana wokół tego modelu: reprezentacja zadania, pętla, stan, obsługa obserwacji, wybór działań i logika zatrzymania. Celem nie jest pomniejszanie możliwości modelu. Chodzi o ich precyzyjne umiejscowienie, tak aby późniejsze dodatki architektoniczne miały jasne uzasadnienie.

Po tym rozdziale będzie można przeczytać opis systemu i odpowiedzieć na trzy pierwsze pytania:

1. Gdzie znajduje się cel?
2. Co utrzymuje się między krokami?
3. Jaki mechanizm wybiera, czy kontynuować, pytać, działać czy się zatrzymać?

Kluczowe pojęcia i definicje

Model to komponent generatywny, który odwzorowuje bieżący kontekst wejściowy na kolejny wynik. W tej książce najczęściej chodzi o model językowy, ale punkty architektoniczne nie zależą od jednego dostawcy ani produktu.

System interaktywny odpowiada tura po turze, zwykle przy założeniu, że to użytkownik wyznacza następny ruch. Może być skuteczny, ale nie musi utrzymywać jawnego sterowania zależnego od celu między kolejnymi krokami.

Skryptowy przepływ pracy wykonuje z góry ustaloną sekwencję etapów. Może działać dobrze, gdy profil zadania jest stabilny, ale zdolność adaptacji ogranicza się do tego, co skrypt przewidział wcześniej.

System agentowy to zależna od celu pętla z jawnymi aktualizacjami stanu i adaptacyjnym wyborem następnego kroku pod wpływem obserwacji. Taki system nie tylko generuje tekst. Decyduje, w granicach swoich uprawnień, jak kontynuować.

Minimalna pętla sterowania w tej książce ma pięć elementów koncepcyjnych:

- **observe:** zaobserwuj, co się wydarzyło albo co jest teraz dostępne,
- **think:** zastanów się, co to oznacza dla bieżącego zadania,
- **act:** działaj wtedy, gdy uzasadniony jest krok zewnętrzny,
- **update:** zaktualizuj stan albo pamięć, aby późniejsze kroki dziedziczyły właściwe informacje,
- **stop:** zatrzymaj się albo przekaz sprawę dalej, gdy dalsza kontynuacja nie jest już uzasadniona.

System może być **mniej lub bardziej agentowy**. Agentowość nie jest magicznym podziałem binarnym. Rośnie wtedy, gdy coraz więcej z następujących elementów jest jawnych i operacyjnych, zamiast ukrytych i kruchych: reprezentacja celu, stan między krokami, adaptacja napędzana obserwacjami, ograniczona autonomia oraz logika zatrzymania lub przekazania.

Najłatwiej pomylić: model z systemem agentowym

Model jest generatorem. System agentowy jest zorganizowaną całością zbudowaną wokół niego. Zdanie „model zdecydował, że trzeba jeszcze raz wyszukać” jest zwykle niechlujną analizą. Lepiej powiedzieć: „system użył modelu wewnątrz pętli, która po otrzymaniu obserwacji wybrała kolejny krok wyszukiwania”. Nie chodzi tu o czystość języka. Chodzi o jasność przyczynową.

Najpierw intuicja

Model generatywny bardzo dobrze radzi sobie z **lokalną spójnością**. Jeśli dostanie bieżący kontekst, często potrafi użytecznie go rozwinąć. Zadanie wieloetapowe nie jest jednak po prostu długą kontynuacją. To sekwencja wyborów dokonywanych pod ograniczeniami, przy zmieniającym się materiale dowodowym, częściowym postępie i powodach, by się zatrzymać.

To rozróżnienie dobrze widać na przykładzie przewodniego zadania z przeglądem literatury. Odpowiedź jednorazowa może stworzyć wiarygodny kształt przeglądu, bo model nauczył się wielu wzorców pisania akademickiego (Brown et al., 2020). Prośba nie brzmi jednak tylko „napisz przegląd”. Mówi też:

- trzymaj się podanego zakresu dat,
- użyj ograniczonej liczby artykułów,
- oddziel architekturę od twierdzeń ewaluacyjnych,
- dołącz macierz porównawczą,
- zatrzymaj się albo zapytaj, jeśli materiał dowodowy jest zbyt słaby albo zakres niejasny.

Te warunki mają znaczenie **w trakcie** wykonania, a nie tylko na początku. System musi utrzymywać je przy życiu wtedy, gdy wyszukuje, odsiewa, czyta, podsumowuje i decyduje, czy ma już dość materiału dowodowego. To właśnie jest praca architektoniczna.

Przydatny nieformalny test jest taki: jeśli przyszłe zachowanie systemu zależy od tego, co system obserwuje po drodze, a ta zależność jest zaimplementowana przez jawne sterowanie, a nie przez pojedynczy statyczny prompt, to przechodzi się od użycia modelu do architektury agentowej.

Analiza techniczna

Co model potrafi zrobić samodzielnie

Silny model generatywny potrafi:

- interpretować instrukcje w języku naturalnym,
- generować kandydackie dekompozycje albo podsumowania,
- proponować wiarygodne kolejne kroki w formie tekstu,
- naśladować gatunki takie jak przeglądy, notatki czy plany,
- kompresować i przekształcać zaobserwowany materiał.

To są ważne możliwości. To właśnie one sprawiają, że systemy agentowe budowane wokół modeli generatywnych w ogóle warto projektować.

Czego model nie potrafi samodzielnie utrzymać

Samo wywołanie modelu **nie** zapewnia automatycznie:

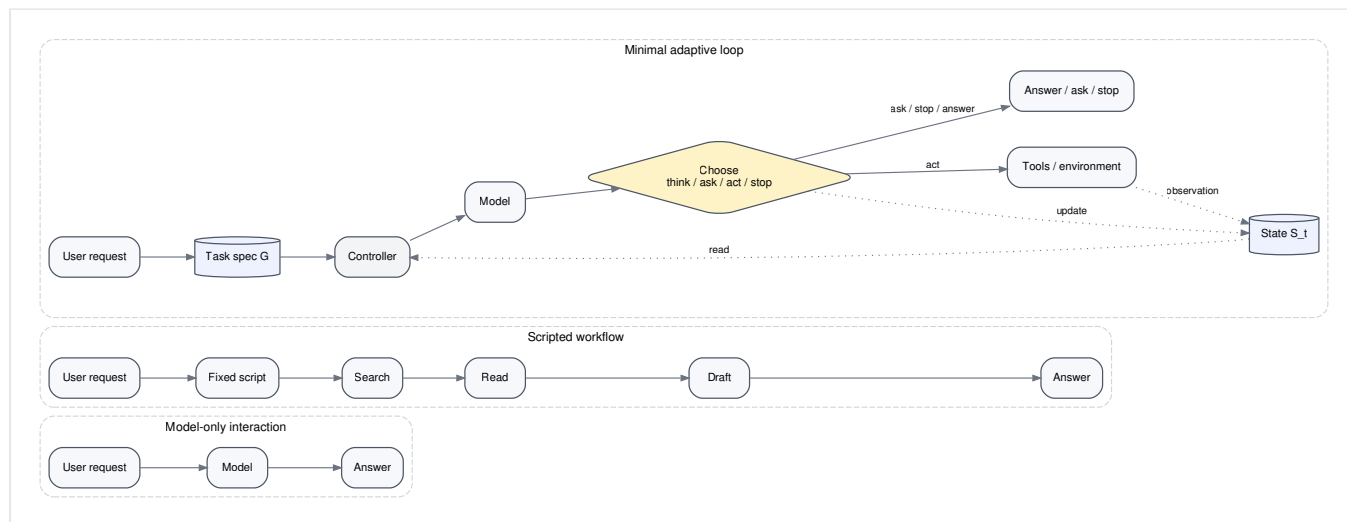
- trwałego jawnego stanu między krokami,
- gwarantowanego zachowania kryteriów sukcesu,
- trwałego zapisu tego, co już zostało wypróbowane,
- polityki określającej, co i kiedy zapisywać do pamięci,
- uzasadnionego wyboru między think, ask, act i stop,
- jawnych warunków zatrzymania lub przekazania.

Nawet gdy długi prompt wydaje się „zawierać wszystko”, pozostaje tylko bieżącym kontekstem wejściowym. Jeśli późniejsze kroki zależą od informacji, które nie są niezawodnie przechowywane lub aktualizowane, zadanie zaczyna dryfować.

Trzy kategorie systemów dla tego samego ograniczonego zadania

Tabela 1.1 porównuje trzy sposoby obsłużenia przewodniego zadania z przeglądem literatury.

Forma systemu	Gdzie znajduje się cel	Co utrzymuje się między krokami	Jak wybierane są kolejne kroki	Typowy problem w przykładzie przewodnim
Interaktywna odpowiedź oparta wyłącznie na promptcie	Głównie w jednym bloku instrukcji i w pamięci użytkownika	Niewiele poza bieżącą turą	Kontynuacją steruje użytkownik	Zbyt wcześnie wygląda na kompletną; nie potrafi samodzielnie zebrać materiału dowodowego ani zatrzymać się z uzasadnionych powodów
Stały skryptowy przepływ pracy	W sekwencji zaprojektowanej przez autora skryptu	To, co przepływ pracy jawnie przekazuje dalej	Z góry ustalone przez skrypt	Idzie dalej nawet wtedy, gdy wyniki wyszukiwania pokazują, że początkowa dekompozycja była błędna
Minimalna pętla adaptacyjna	W jawnej specyfikacji zadania oraz stanie pętli	Stan zewnętrzny aktualizowany po każdej obserwacji	Wybierane w czasie działania na podstawie bieżących obserwacji	Nadal może źle się zapętlić, jeśli reguły zatrzymania są słabe albo stan źle rozmieszczony



Rysunek 1.1. To samo ograniczone zadanie z przeglądem literatury potraktowane jako interakcja oparta wyłącznie na modelu, skryptowy przepływ pracy oraz minimalny system oparty na pętli. Tylko trzeci wariant jawnie wiąże przyszłe sterowanie z obserwacjami i aktualizacjami stanu.

Nie chodzi o to, że systemy prompt-only albo skryptowe są bezużyteczne. Dla małych, stabilnych zadań często są dokładnie tym, czego potrzeba. Chodzi o to, że **agentowość jest własnością struktury sterowania**, a nie synonimem imponującego wyniku modelu.

Anatomia minimalnej pętli

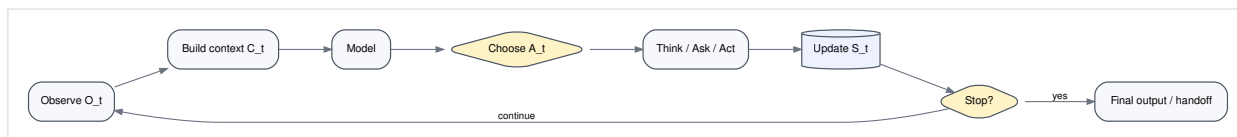
Najmniejszą użyteczną pętlę można wyrazić lekką notacją:

- Specyfikacja zadania: G
- Bieżący kontekst dla następnego wywołania modelu: C_t
- Jawnie przenoszony stan: S_t
- Nowa obserwacja: O_t
- Wybrany następny ruch: $A_t \in \{\text{think, ask, act, stop}\}$

Krok koncepcyjny wygląda tak:

1. Złóż C_t z G , istotnych części S_t i najnowszej obserwacji.
2. Użyj modelu, aby zaproponować albo ocenić następny ruch.
3. Wykonaj wybrany ruch.
4. Zapisz wynik w $S_{(t+1)}$.
5. Oceń, czy należy kontynuować.

Ta pętla jest architektoniczna, bo ważna praca nie polega na „lepszym promptcie”. Polega na rozmieszczeniu informacji o zadaniu, ścieżce aktualizacji stanu i decyzji sterującej o tym, co stanie się dalej.



Rysunek 1.2. Minimalna gramatyka pętli używana w całej książce. Model uczestniczy wewnątrz pętli; sam nie jest tą pętlą.

Dlaczego samo korzystanie z narzędzi nie wystarcza

Powracające nieporozumienie brzmi tak: jeśli model potrafi wywoływać narzędzia, to jest agentem. Niekoniecznie. Korzystanie z narzędzi poszerza to, co system może wiedzieć i zrobić (Schick et al., 2023), ale narzędzia może też wywoływać skrypt. Pytanie architektoniczne nadal brzmi: **kto decyduje, kiedy wywołać narzędzie, na podstawie jakich obserwacji, przy jakim stanie i jakich regułach zatrzymania?**

Model, który zawsze wykonuje to samo wyszukiwanie wewnątrz stałego pipeline’u, jest nadal bliższy skryptowemu przepływowi pracy niż adaptacyjnemu systemowi agentowemu. Dostęp do narzędzi zmienia możliwości. Sam z siebie nie tworzy sterowania ukierunkowanego na cel.

Stopnie agentowości

Lepiej myśleć tu w kategoriach stopniowalnych. System staje się bardziej agentowy, gdy ma więcej z poniższych cech:

1. **Jawna reprezentacja zadania.** Cel, ograniczenia i warunki zatrzymania są operacyjne.
2. **Stan między krokami.** System zachowuje informacje potrzebne do późniejszych wyborów.

3. **Sterowanie napędzane obserwacjami.** Nowy materiał dowodowy zmienia to, co dzieje się dalej.
4. **Adaptacyjny wybór działania.** Następny ruch nie jest z góry ustalony.
5. **Ograniczona autonomia.** System może kontynuować działanie w pewnych granicach bez czekania na użytkownika po każdej turze.
6. **Logika zatrzymania lub przekazania.** System potrafi decydować nie tylko o tym, jak kontynuować, ale też kiedy nie kontynuować.

Kolejne rozdziały rozwiną każdy z tych wymiarów. Na razie najważniejsza lekcja jest taka, że sprawczość jest przede wszystkim **architektonicznym rusztowaniem wokół lokalnej generacji** (Xi et al., 2023).

Przykład krok po kroku: porównanie na przykładzie przewodnim

Założmy, że system otrzymuje ograniczone zadanie z przeglądem literatury.

Odpowiedź oparta wyłącznie na promptcie. Model od razu pisze schludny szkic przeglądu literatury. Może wspomnieć znane artykuły, a nawet ułożyć odpowiedź tematycznie. Ale jeśli zapytać: „Które twierdzenia miały grounding w przeczytanych źródłach, a które pochodzą z wcześniejszej wiedzy modelu?”, architektura nie ma jawnej odpowiedzi. Stan materiału dowodowego nigdy nie został wyprowadzony na zewnątrz.

Stały skrypt. Przepływ pracy wykonuje sekwencję: uruchom dwa zapytania, przeczytaj osiem najlepszych wyników, podsumuj każdy z nich, a potem napisz przegląd. To często działa lepiej, bo przynajmniej dotyka materiału dowodowego. Jeśli jednak dwa pierwsze zapytania zwracają głównie chatboty tutoringowe bez wsparcia w cytowaniach, skrypt nadal idzie dalej. Nie potrafi uznać, że rodzina zapytań jest błędna albo że materiał dowodowy jest zbyt słaby.

Minimalna pętla adaptacyjna. Pętla dostaje tę samą prośbę, wyszukuje raz, zauważa, że wyniki mieszają ogólne agenty tutoringowe z mniejszym podzbiorem systemów opartych na cytowaniach, i wybiera doprecyzowanie zapytania zamiast natychmiastowego podsumowania. To najmniejszy ruch architektoniczny, który zaczyna zasługiwać na miano *agentowego*: system zmienił kurs, bo obserwacja zmieniła sterowanie.

Przykład w pseudokodzie

```

procedure MINIMAL_AGENT_RUN(user_request):
  G <- compile_bounded_task_spec(user_request)
  S0 <- initialize_state(G)

  for t = 0, 1, 2, ...:
    O_t <- latest_observation(S_t)
    C_t <- build_context(G, S_t, O_t)
    A_t <- choose_next_move(C_t)      # think | ask | act | stop

    if A_t == think:
      S_(t+1) <- update_state_with_reasoning(S_t)
    else if A_t == ask:
      return request_clarification(G, S_t)
    else if A_t == act:
      R_t <- execute_external_step(S_t)
      S_(t+1) <- update_state_with_result(S_t, R_t)
    else if A_t == stop:
      return produce_or_handoff_result(S_t)

  if stop_condition_met(G, S_(t+1)):
    return produce_or_handoff_result(S_(t+1))

```

Ten pseudokod jest celowo oszczędny. Ukrywa szczegóły implementacyjne i uwypatnia strukturę: jawną specyfikację zadania, stan, obserwację, adaptacyjny wybór następnego kroku oraz ścieżkę zatrzymania.

Aktualizacja wspólnego artefaktu

Po tym rozdziale asystent do przeglądu literatury zyskuje pierwszy widoczny artefakt architektoniczny:

- **minimalną pętlę adaptacyjną,**
- nazwane słownictwo sterowania: observe / think / act / update / stop,
- analityczne rozróżnienie między **modelem**, **skryptowym przepływem pracy** a **systemem agentowym**.

Ta pętla jest jeszcze zbyt słaba, by działać dobrze. Nie ma jawnej specyfikacji zadania, stabilnego schematu stanu, polityki pamięci ani dyscypliny zatrzymania. Tym zajmą się kolejne rozdziały.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Jeśli pętli brakuje, asystent do przeglądu literatury staje się systemem jednorazowym albo sterowanym przez użytkownika. Może tworzyć elegancki tekst, ale nie potrafi samodzielnie zbierać, porównywać ani korygować materiału dowodowego.

Jeśli pętla istnieje, ale jest używana źle, pojawiają się dwa częste problemy:

1. **Pseudo-sprawczość.** System nazywa siebie agentem, ale następny krok jest w praktyce z góry przesądzony. Pętla pełni funkcję dekoracyjną.

2. **Mielenie w pętli.** System potrafi kontynuować, ale nie ma powodów specyficznych dla zadania, by się zatrzymać, więc dalej wyszukuje lub przepisuje tekst bez poprawy wyniku.

Lekcja jest już widoczna: dodanie pętli rozwiązuje jedną klasę problemów, ale wprowadza inną. Projektowanie architektury zawsze dotyczy kompromisów, a nie magicznych ulepszeń.

Tryby awarii, ograniczenia i błędne przekonania

- **„Agent to po prostu chatbot z lepszym promptem.”** Nie. Lepszy prompt może poprawić pojedyncze wywołanie modelu, ale agentowość zależy od struktury sterowania, stanu i adaptacyjnej kontynuacji.
- **„Każde użycie narzędzia czyni system agentowym.”** Nie. Narzędzia poszerzają przestrzeń działań. Skrypt może korzystać z narzędzi bez adaptacyjnego sterowania.
- **Antropomorfizowanie modelu.** Mówienie „model chciał jeszcze raz zajrzeć” ukrywa rzeczywisty mechanizm. To architektura wybrała kolejny krok na podstawie obsługi stanu i obserwacji.
- **„Żeby ten temat miał sens, trzeba najpierw znać reinforcement learning albo planowanie formalne.”** Nie. To obszary sąsiednie, a nie warunki wstępne. Ta książka zaczyna od ograniczonych pętli sterowania, a nie od uczonych polityk czy formalnych planerów (Sutton and Barto, 2018; Russell and Norvig, 2021).

Podsumowanie rozdziału

Model generatywny jest silnikiem lokalnej generacji. System agentowy jest zorganizowaną strukturą, która zamienia ten silnik w ograniczone zachowanie ukierunkowane na cel w czasie. Minimalne składniki to jawna reprezentacja zadania, pętla, aktualizacje stanu, obsługa obserwacji, adaptacyjny wybór następnego kroku i sposób zatrzymania.

Ten rozdział wprowadza pierwsze stabilne rozróżnienie w tej książce: **model $\neq$ system**. Następny rozdział pyta, co dokładnie system próbuje zrobić i jak przedstawić to zadanie na tyle jasno, by sterowało pętlą.

Źródła i dalsza lektura

- (Brown et al., 2020) jako kontekst dla generacji ograniczonej kontekstem, która stanowi podłoże późniejszej architektury.
- (Yao et al., 2022; Schick et al., 2023) jako przykłady przeplatania generacji z działaniem.
- (Xi et al., 2023) jako szeroki przegląd badań nad agentami opartymi na dużych modelach językowych.
- (Russell and Norvig, 2021; Sutton and Barto, 2018) jako tradycje sąsiednie, do których książka będzie się odwoływać, ale których nie wymaga jako warunku wstępnego.

Co ten rozdział dodaje do architektury

- **Dodano:** minimalną pętlę zależną od celu wokół modelu.

- **Zaadresowany problem:** zachowanie jednorazowe albo czysto skryptowe, które nie potrafi dostosować się do obserwacji.
- **Nowe ryzyko:** zapętlenie bez jasnej reprezentacji zadania i dyscypliny zatrzymania.

Rozdział 2 — Cele, specyfikacje zadań i kryteria sukcesu

AA-S02 — Cele, specyfikacja zadania i kryteria sukcesu

Jakie wymagania ujawnia przykład przewodni

Pętla z rozdziału 1 może działać dalej. Ale dalej **w stronę czego**, dokładnie?

Prośba kierowana do asystenta do przeglądu literatury brzmi jasno tylko do chwili, gdy pojawi się pierwsze pytanie rozgałęziające:

- Czy „dla studentów” oznacza tutoring K-12, wsparcie w studiowaniu na uczelni czy asystentów wyszukiwania bibliotecznego?
- Czy preprinty są dopuszczalne, czy tylko artykuły recenzowane?
- Czy „oparte na cytowaniach” oznacza cytowania w tekście, weryfikację cytatów czy dowolny odzyskany materiał dowodowy?
- Kiedy asystent wykonał już dość pracy, aby się zatrzymać?

Pętla bez jasnej specyfikacji zadania jest jak kierownica niepołączona z kołami. System może się poruszać, ale ten ruch nie odpowiada przed zdefiniowanym celem. Dlatego ten rozdział zamienia mgliste intencje w **operacyjny obiekt projektowy**.

Plan rozdziału

Ten rozdział odróżnia prośbę użytkownika od specyfikacji zadania systemu. Wprowadza lekką notację używaną w książce

`G = <objective, constraints, success criteria, stop/handoff conditions>`

i używa jej po to, by umożliwić sterowanie. Nie chodzi o biurokratyczną dokumentację. Chodzi o to, że cele kierują zachowaniem tylko wtedy, gdy są przedstawione wystarczająco jasno, by wpływać na to, co system robi dalej.

Po tym rozdziale będzie można wyjaśnić, dlaczego dwa systemy, które otrzymały „tę samą prośbę”, mogą wymagać innych pętli, polityk pamięci i logiki zatrzymania, jeśli różnią się ich specyfikacje zadań.

Kluczowe pojęcia i definicje

Prośba użytkownika to to, o co dana osoba prosi w języku naturalnym.

Specyfikacja zadania to pełny operacyjny pakiet, którego system używa do sterowania: cel, ograniczenia, kryteria sukcesu oraz warunki zatrzymania lub przekazania.

Cel to składnik celu właściwego wewnątrz tego pakietu.

Podcel to cel tymczasowy wprowadzany na potrzeby większego zadania.

Kryteria sukcesu określają, co liczy się jako adekwatne wykonanie. Nie są tym samym co cel. „Przygotuj przegląd literatury” to cel. „Każde istotne twierdzenie jest wsparte co najmniej jednym przeczytanym źródłem, a przegląd zawiera macierz porównawczą” to kryteria sukcesu.

Kryteria zatrzymania określają, kiedy system powinien przestać próbować kontynuować. **Warunki przekazania** określają, kiedy system powinien eskalować do użytkownika lub innego agenta, zamiast działać dalej autonomicznie.

Zadanie może być **otwarte** albo **ograniczone**. Zadania otwarte zapraszają do niekończącego się rozwijania. Zadania ograniczone wyznaczają granice zakresu, materiału dowodowego, głębokości lub autonomii.

Najłatwiej pomylić: prośbę użytkownika ze specyfikacją zadania

Prośba użytkownika to to, co użytkownik mówi. Specyfikacja zadania to to, na czym system naprawdę pracuje. Dobre systemy nie zakładają, że te dwie rzeczy są identyczne. Zamieniają prośbę w coś operacyjnego.

Podobnie **cel** jest tylko jedną częścią specyfikacji zadania. Ograniczenia, kryteria sukcesu oraz logika zatrzymania lub przekazania są odrębnymi zobowiązaniami architektonicznymi.

Najpierw intuicja

Wiele porażek przypisywanych „słabemu rozumowaniu” to w rzeczywistości porażki specyfikacji zadania.

Założmy, że asystent słyszy: „Przejrzyj najnowsze prace o asystentach odpowiadania na pytania dla studentów, których odpowiedzi są oparte na cytowaniach”. Jeśli po prostu zacznie wyszukiwać, kilka ukrytych wyborów zostało już podjętych bez żadnej dyskusji: co znaczy „najnowsze”, czy chatboty edukacyjne bez cytowań się liczą, jak ma wyglądać wynik, ile materiału dowodowego wystarczy i kiedy niejednoznaczność jest na tyle poważna, że trzeba zadać pytanie.

Ta ukrytość ma znaczenie, bo architektury nie da się ocenić, jeśli zadanie nie jest jawne. System, który zwraca dopracowany trzyakapitowy przegląd, może wydawać się skuteczny, dopóki użytkownik nie oczekiwał macierzy porównawczej z 10 źródłami. Inny system może wyglądać na powolny albo przesadnie drobiazgowy, bo prosi o doprecyzowanie, a właśnie to doprecyzowanie może sprawiać, że architektura pozostaje ograniczona i poprawna.

W interakcji człowiek-komputer doprecyzowanie nie jest stanem porażki. To ruch projektowy służący zestrojeniu zachowania systemu z intencją użytkownika (Amershi et al., 2019). W tej książce doprecyzowanie jest także działaniem sterującym: czasem właściwym następnym krokiem jest **ask**, a nie **act**.

Analiza techniczna

Specyfikacja zadania jako krotka

W całej książce będziemy używać lekkiej notacji:

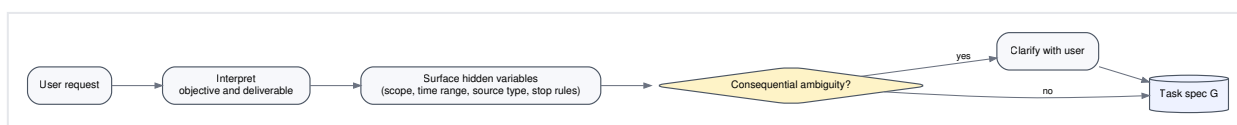
G = <objective, constraints, success criteria, stop/handoff conditions>

Dla przykładu przewodniego silniejsza specyfikacja zadania niż sama surowa prośba wygląda następująco:

- **cel:** przygotować przegląd literatury na 1 200 słów o asystentach odpowiadania na pytania dla studentów, których odpowiedzi są oparte na cytowaniach;
- **ograniczenia:** użyć materiału z lat 2021-2024, uwzględnić 8-12 artykułów, jeśli to możliwe, oddzielić wzorce architektoniczne od twierdzeń ewaluacyjnych, dołączyć krótką macierz porównawczą, unikać szczegółów implementacyjnych i produktów konkretnych dostawców;
- **kryteria sukcesu:** każde istotne twierdzenie ma grounding w co najmniej jednym przeczytanym źródle, przegląd obejmuje co najmniej trzy wzorce architektoniczne, macierz porównawcza zawiera pola źródła, metody groundingu, użycia narzędzi i ograniczenia, a niepewność jest ujawniana tam, gdzie materiał dowodowy jest skąpy;
- **warunki zatrzymania lub przekazania:** zapytaj, jeśli „dla studentów” albo „oparte na cytowaniach” są istotnie niejednoznaczne, zatrzymaj się, gdy kryteria sukcesu zostaną spełnione przy wystarczającym materiale dowodowym, i przekaz sprawę dalej, jeśli dostępny materiał jest zbyt skąpy albo sprzeczny w sposób, którego asystent nie potrafi odpowiedzialnie rozstrzygnąć.

To jest dłuższe niż prośba użytkownika. I bardzo dobrze. Sterowanie potrzebuje więcej struktury, niż zwykle dostarcza sama rozmowa.

Przekształcenie prośby w specyfikację zadania



Rysunek 2.1. Prośba użytkownika staje się operacyjną specyfikacją zadania dzięki interpretacji, doprecyzowaniu tam, gdzie jest potrzebne, oraz jawnym decyzjom ograniczającym.

To przekształcenie zwykle obejmuje cztery ruchy:

1. **Zinterpretuj prośbę.** Zidentyfikuj pozorny cel i kształt wyniku.
2. **Ujawnij ukryte zmienne.** Zakres czasu, typ źródeł, głębokość, domenę, ryzyko i granice autonomii.
3. **Doprecyzuj wtedy, gdy niejednoznaczność ma znaczenie.** Nie każda niejasność jest warta pytania; ważne jest to, czy zmienia architekturę albo ocenę.
4. **Skompiluj ograniczoną specyfikację zadania.** Uczyń jawnymi cel, ograniczenia, kryteria sukcesu oraz logikę zatrzymania i przekazania.

Forma wyniku jest częścią sterowania

Architektura zachowuje się inaczej w zależności od tego, jaki wynik system ma dostarczyć.

Krótkie punktowe podsumowanie trzech artykułów może wymagać tylko krótkiego horyzontu i niewielkiej pamięci. Przegląd literatury z macierzą porównawczą, uwagami o niepewności i wymogami dowodowymi potrzebuje mocniejszej obsługi stanu, być może planowania i na pewno jawnej logiki zatrzymania.

Właśnie dlatego „wykonaj zadanie dobrze” nie jest użytecznym celem. Nie daje systemowi żadnego kryterium rozstrzygającego, czy trzeba jeszcze szukać, podsumować już teraz czy poprosić użytkownika o zawężenie zakresu.

Sformułowania otwarte a ograniczone

Porównajmy dwa poniższe sformułowania zadania.

Sformułowanie	Co mówi systemowi	Konsekwencja architektoniczna
„Przejrzyj najnowsze prace o edukacyjnych asystentach QA.”	Szeroki temat, brak ograniczeń, brak progu sukcesu, brak warunku zatrzymania	Zachęca do otwartego wyszukiwania i sprawia, że zatrzymanie staje się arbitralne
„Przygotuj przegląd na 1 200 słów o asystentach QA dla studentów, których odpowiedzi są oparte na cytowaniach, z lat 2021-2024, używając 8-12 artykułów, jeśli to możliwe, z macierzą porównawczą i jawnymi uwagami o niepewności.”	Zakres, horyzont, forma wyniku, cel dowodowy i sygnały ukończenia	Wspiera ograniczone wyszukiwanie, kontrolowane sporządzanie notatek i uzasadnione decyzje o zatrzymaniu

Sformułowanie ograniczone nie zawsze jest lepsze dla użytkownika. Niektórzy naprawdę chcą otwartej eksploracji. Architektura musi jednak wiedzieć, z którym przypadkiem ma do czynienia. Zadania otwarte zwykle wymagają częstszego ponownego wejścia użytkownika, mocniejszego doprecyzowania i bardziej jawnych granic autonomii.

Podcele nie są celem końcowym

Podcele pomagają systemowi robić postęp, ale muszą pozostać podporządkowane zadaniu głównemu. Dla przykładu przewodniego kandydackie podcele są następujące:

1. wygeneruj początkowe hasła wyszukiwawcze,
2. pobierz kandydackie źródła,
3. odsiej źródła pod kątem istotności,
4. sporządź notatki,
5. zsyntetyzuj wzorce architektoniczne,
6. zweryfikuj twierdzenia i cytowania,
7. zatrzymaj się albo przekaz sprawę dalej.

Żaden z tych kroków nie jest celem końcowym. System, który traktuje „pobranie artykułów” jako sukces, myli podcel z samym zadaniem. To pomieszanie jest częste w projektowaniu agentów, bo kroki pośrednie są widoczne i dają poczucie postępu. Wykonane wyszukiwanie nie jest jednak ukończonym przeglądem literatury.

Doprecyzowanie jako pełnoprawny element architektury

Doprecyzowanie staje się właściwym ruchem wtedy, gdy niejednoznaczność zmienia jedną z następujących rzeczy:

- zakres źródeł, które należy przeszukać,
- ryzyko albo odwracalność działania,
- kształt końcowego rezultatu,
- próg sukcesu,

- logikę zatrzymania albo przekazania.

Jeśli „dla studentów” może oznaczać albo systemy tutoringowe, albo asystentów wyszukiwania literatury, zestaw zapytań, reguły odsiewania i wymiary oceny będą inne. W takim przypadku doprecyzowanie jest uzasadnione. Jeśli niejednoznaczność jest drobna i nie zmienia sterowania, system może działać dalej, odnotowując przyjęte założenie.

Dobre pytanie doprecyzowujące jest wąskie, istotne dla decyzji i łatwe do udzielenia odpowiedzi. Na przykład:

Czy chodzi o asystentów używanych przez studentów do nauki i tutoring, czy o systemy pomagające studentom wyszukiwać i czytać literaturę? Ten wybór zmienia to, które artykuły uznaje się za istotne.

To pytanie nie jest uprzejmą konwersacyjną wstawką. To akt architektoniczny, który wyostża G.

Przykład krok po kroku: od mglistej prośby do ograniczonej specyfikacji zadania

Założmy, że surowa prośba brzmi:

„Proszę przejrzeć najnowsze prace o asystentach odpowiadania na pytania dla studentów, których odpowiedzi są oparte na cytowaniach.”

Ograniczona kompilacja mogłaby wyglądać tak:

```
G_review = <objective = produce a 1,200-word literature review; constraints = 2021–2024, 8–12 papers if available, architecture-first framing, comparison matrix required; success criteria = each substantive claim grounded in at least one read paper, at least three architecture patterns compared, uncertainty noted where evidence is thin; stop/handoff = ask if scope remains ambiguous, stop when criteria are met, hand off if fewer than five solid sources are accessible or if evidence conflicts on high-stakes recommendations>.
```

Warto zauważyć dwie rzeczy.

Po pierwsze, część elementów została **ujawniona**, a nie wymyślona. Dobra architektura często musi zamienić ukryte oczekiwania w elementy operacyjne.

Po drugie, ta skompilowana specyfikacja już implikuje późniejsze wybory architektoniczne:

- Wymóg dowodowy implikuje jawny stan, a być może także pamięć.
- Macierz porównawcza implikuje ustrukturyzowane sporządzanie notatek.
- Reguła niejednoznaczności implikuje dostępne działanie **ask-user**.
- Reguła zatrzymania implikuje, że pętla musi oceniać wystarczalność, a nie tylko ukończenie kolejnych kroków.

Przykład w pseudokodzie

```
procedure COMPILE_TASK_SPEC(request):
  objective <- identify_primary_deliverable(request)
  constraints <- extract_or_assume_bounds(request)
  success <- define_what_counts_as_adequate(objective, constraints)
  stop_handoff <- define_when_to_stop_ask_or_escalate(objective, constraints)

  if consequential_ambiguity(objective, constraints, success):
    return ASK_USER_FOR_CLARIFICATION
  else:
    return <objective, constraints, success, stop_handoff>
```

Ten pseudokod jest celowo skromny. Podkreśla kluczową rzecz: specyfikacja zadania jest artefaktem sterowania, który sam może uruchomić doprecyzowanie.

Aktualizacja wspólnego artefaktu

Asystent do przeglądu literatury zyskuje teraz pierwszy formalny artefakt:

Artefakt	Bieżąca zawartość
Specyfikacja zadania G_review	Cel, ograniczenia, kryteria sukcesu oraz warunki zatrzymania i przekazania dla ograniczonego zadania z przeglądem literatury

Ten artefakt pozostanie widoczny w całej dalszej części książki. Kolejne rozdziały pokażą, jak G_review wpływa na rozmieszczenie stanu, politykę pamięci, dobór narzędzi, planowanie i zatrzymanie.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Jeśli brakuje specyfikacji zadania, system ma tendencję do jednego z czterech rodzajów problemów:

1. **Dryf zakresu.** Temat stale się rozszerza, bo nic w stanie sterowania nie mówi, gdzie leży granica.
2. **Przedwczesne ukończenie.** System zatrzymuje się po wytworzeniu wiarygodnie brzmiącego tekstu, zamiast po spełnieniu jawnych kryteriów sukcesu.
3. **Przechwycenie przez podcel.** System bierze udane wyszukiwanie albo podsumowanie za sukces całego zadania.
4. **Autonomia bez kierunku.** System działa dalej bez jasności, co uzasadniałoby zatrzymanie albo zapytanie użytkownika.

Jeśli specyfikacja zadania jest używana źle, pojawia się inny problem: **nadmierne skrępowanie systemu**. Krucha specyfikacja może zakodować na sztywno niepotrzebną sekwencję zamiast opisywać rzeczywisty cel i ograniczenia. Architektura potrzebuje wskazówek, a nie skryptu udającego cel.

Tryby awarii, ograniczenia i błędne przekonania

- **„Wystarczy wykonać zadanie dobrze.”** Nie wystarczy. System potrzebuje kryteriów sukcesu, które potrafi sprawdzić.
- **Zlewanie podcelów z celem końcowym.** Pobrane artykuły, notatki i podsumowania są środkami, a nie końcem.
- **Ignorowanie kryteriów zatrzymania.** Bez jawnej logiki zatrzymania pętle mają tendencję do działania aż do zewnętrznego limitu czasu albo arbitralnej decyzji.
- **Traktowanie doprecyzowania jako porażki.** Doprecyzowanie jest często właściwym następnym ruchem, gdy niejednoznaczność zmienia sterowanie (Amershi et al., 2019).
- **Założenie, że więcej szczegółów zawsze jest lepsze.** Specyfikacja zadania powinna być wystarczająco operacyjna, a nie niepotrzebnie proceduralna.

Podsumowanie rozdziału

Cele mają znaczenie tylko wtedy, gdy są przedstawione dość jasno, by zmieniać sterowanie. Prośba użytkownika jest wejściem konwersacyjnym. Specyfikacja zadania jest operacyjnym celem architektury. Wyrażając przykład przewodni jako

`G = <objective, constraints, success criteria, stop/handoff conditions>`

sprawiamy, że późniejsze pytania projektowe stają się konkretne: jaki stan trzeba przesyłać, jaka pamięć jest potrzebna, kiedy system powinien pytać i co liczy się jako wystarczający materiał dowodowy, by się zatrzymać.

Następny rozdział pyta, gdzie te informacje istnieją w czasie. Gdy zadanie staje się jawne, można prześledzić, jak przebiega wykonanie i jak rozmieszczenie stanu wpływa na problemy architektury.

Źródła i dalsza lektura

- (Amershi et al., 2019) w kontekście doprecyzowania, zarządzania oczekiwaniami i ograniczeń widocznych dla użytkownika.
- (Parasuraman et al., 2000) jako klasyczne ujęcie wyboru poziomów automatyzacji zamiast założenia, że „więcej autonomii” zawsze jest lepsze.
- (Xi et al., 2023) jako przykłady współczesnych systemów agentowych, które jawnie albo niejawnie kompilują zadania do struktur sterowania.

Co ten rozdział dodaje do architektury

- **Dodano:** jawną specyfikację zadania `G_review`.
- **Zaadresowany problem:** mgliste zapętlenie wokół zbyt słabo zdefiniowanego celu.
- **Nowe ryzyko:** tak sztywne przesterowanie specyfikacji, że „cel” staje się ukrytym skryptem.

Rozdział 3 — Pętle, stan, kontekst i ślady wykonania

AA-S03 — Pętle sterowania, stan i kontekst

Jakie wymagania ujawnia przykład przewodni

Asystent do przeglądu literatury ma już `G_review`, ale podczas wykonania pojawia się nowy rodzaj problemu.

W kroku 1 pobiera mieszany zestaw artykułów: część rzeczywiście dotyczy odpowiadania z groundingiem w cytowaniach, inne opisują ogólne agenty tutoringowe. W kroku 2 odsiewa wyniki i decyduje, że wykluczy systemy, które jedynie pobierają fragmenty, ale nie cytują ich w odpowiedzi końcowej. W kroku 3 otwiera źródło i sporządza notatki. W kroku 4 przygotowuje syntezę.

A potem reguła wykluczania znika.

Dlaczego? Bo istniała tylko wewnątrz kontekstu jednego wcześniejszego wywołania modelu. Nic w architekturze nie umieściło jej w trwałej strukturze zewnętrznej. System zachowywał się tak, jakby „kontekst” był tym samym co „stan”. Nie jest.

Plan rozdziału

Ten rozdział czyni wykonanie widocznym w czasie. Odróżnia bieżący kontekst od przenoszonego stanu zewnętrznego, stanu świata i obserwacji, a jako główne narzędzie wyjaśniające wykorzystuje tabele śladu wykonania. Wraca też do podstawowego ograniczenia modelu z rozdziału 1: generacja ograniczona kontekstem nie jest trwałym sterowaniem.

Po tym rozdziale będzie można przeanalizować wieloetapowe wykonanie i wskazać, gdzie znajduje się informacja, gdzie powinna była się znaleźć i jak problem powstał wskutek pominięcia albo złego umiejscowienia.

Kluczowe pojęcia i definicje

Kontekst (C_t) to to, co jest obecne w bieżącym wywołaniu modelu w kroku t .

Stan zewnętrzny (S_t) to jawna informacja, którą system przenosi między krokami poza bieżącym oknem kontekstu: zmienne, pola zadania, checklisty, liczniki, listy pobranych elementów, częściowe produkty i znaczniki sterujące.

Stan świata to istotny stan środowiska zewnętrznego: jakie dokumenty istnieją, czy da się otworzyć artykuł, czy narzędzie przekracza limit czasu, czy użytkownik odpowiedział i tak dalej.

Obserwacja (O_t) to to, czego system dowiaduje się o świecie lub z wyniku narzędzia w kroku t .

Ślad wykonania to to, co rzeczywiście wydarzyło się w czasie.

Plan to zamierzona struktura przyszłości. **Ślad** to zrealizowane zachowanie z przeszłości.

Najłatwiej pomylić: kontekst ze stanem zewnętrznym i pamięcią

Kontekst to to, co model widzi teraz. **Stan zewnętrzny** to to, co pętla sterowania jawnie przenosi między krokami. **Pamięć**, wprowadzona w następnym rozdziale, jest zaprojektowaną trwałością z politykami zapisu i odczytu. Nie należy zlewać tych rzeczy w jedno mgliste pojęcie „tego, co system wie”.

Najpierw intuicja

Wieloetapowe wykonanie agenta nie jest jedną wielką myślą. To sekwencja transformacji stanu.

To ma znaczenie, bo model generatywny dostaje tylko kontekst złożony dla bieżącego kroku. Nawet jeśli system potrafi włożyć do wywołania modelu duże ilości tekstu, nadal nie rozwiązuje to architektonicznego pytania **co zasługuje na trwałość, gdzie ma istnieć i jak późniejsze kroki mają to odzyskiwać**. Dopychanie większej ilości treści do okna kontekstu jest często obejściem tymczasowym, a nie trwałym projektem (Brown et al., 2020; Packer et al., 2023).

Asystent do przeglądu literatury dobrze to ujawnia. Część informacji należy do bieżącego kontekstu, bo jest bezpośrednio istotna dla następnego wyboru. Inne informacje należą do jawnego stanu, bo późniejsze kroki od nich zależą, nawet jeśli nie warto powtarzać ich słowo w słowo w każdej turze. Jeszcze inne fakty należą do świata zewnętrznego i trzeba je ponownie zaobserwować, zamiast po prostu je zakładać.

Analiza techniczna**Cykl wykonania**

Ograniczone wykonanie zwykle podąża za takim abstrakcyjnym rytmem:

1. sprawdź bieżący stan celu i istotny stan świata,
2. złoż następny kontekst modelu C_t ,
3. wybierz ruch,
4. wykonaj ruch,
5. zapisz to, co się wydarzyło, w $S_{(t+1)}$,
6. oceń, czy kontynuować.

Jakość wykonania w dużym stopniu zależy od **umiejscowienia stanu**. Jeśli jakiś element ma znaczenie dla późniejszego sterowania, ale nigdy nie zostaje zapisany do jawnego stanu, architektura polega na szczęściu.

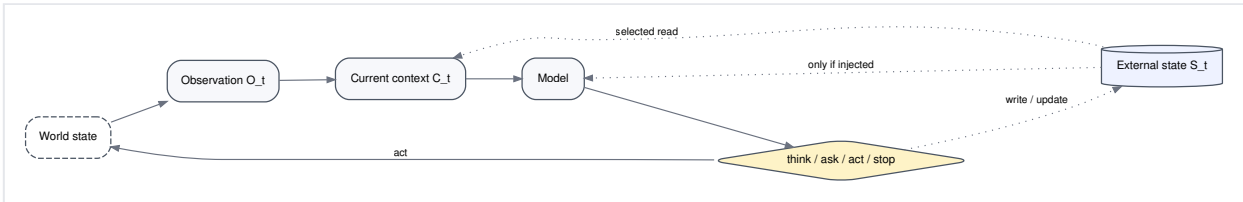
Cztery miejsca, w których może znajdować się informacja

Dla przykładu przewodniego pomocne jest klasyfikowanie informacji według miejsca, w którym się znajduje.

Przykład informacji	Najlepsze miejsce	Dlaczego
Zakres dat 2021–2024	Stan zewnętrzny S_t , a często także kopia w C_t	Powinien utrzymywać się przez całe wykonanie

Przykład informacji	Najlepsze miejsce	Dlaczego
Abstrakt aktualnie otwartego artykułu	Obserwacja O_t , a potem ewentualnie stan, jeśli zostaje zachowany	Pochodzi z narzędzia i może, ale nie musi, zasługiwać na trwałość
Lista już odsianych artykułów	Stan zewnętrzny S_t	Późniejsze kroki muszą unikać duplikacji
To, czy adres URL źródła jest obecnie osiągalny	Stan świata / obserwacja	Może się zmieniać i trzeba to sprawdzać, a nie zakładać
Tymczasowe rozumowanie, czy teraz czytać artykuł 7	Bieżący kontekst C_t	Może mieć znaczenie tylko dla bieżącego wyboru

Nie chodzi o zapamiętywanie kategorii. Chodzi o zadawanie dla każdego elementu informacji pytania: **czy późniejsze sterowanie będzie od tego zależeć, a jeśli tak, to gdzie ta informacja powinna istnieć?**



Rysunek 3.1. Miejsca informacji w ograniczonej pętli. Kontekst, stan zewnętrzny, stan świata i obserwacje zajmują różne miejsca i nie powinny być wrzucane do jednego worka z etykietą „pamięć”.

Tabele śladu wykonania

Standardowy format tabeli śladu wykonania używany w tej książce wygląda tak:

t	widok celu	kontekst	stan zewnętrzny	obserwacja	działanie	wynik	decyzja o zatrzymaniu
---	------------	----------	-----------------	------------	-----------	-------	-----------------------

To nie jest papierologia. To narzędzie diagnostyczne.

Tabela 3.1 pokazuje uproszczony ślad wykonania dla asystenta do przeglądu literatury.

t	widok celu	kontekst	stan zewnętrzny	obserwacja	działanie	wynik	decyzja o zatrzymaniu
0	Potrzebny ograniczony przegląd z macierzą	G _{review} , brak źródeł	lista zapytań pusta; zbiór odsianych pusty	brak	think	szkic początkowych zapytań i reguł odsiewania	continue
1	Potrzebne źródła-kandydaci	początkowe zapytania + reguły odsiewania	zapytania zapisane	wyszukiwanie zwraca 18 mieszanych wyników	act	zapisz kandydatów; oznacz zbyt szeroką mieszankę wyników	continue
2	Potrzebne istotne systemy oparte na	najlepsi kandydaci + reguły	zbiór odsianych pusty	abstrakty pokazują wiele ogólnych systemów	think	doprecyzuj regułę wykluczania: wymagaj cytowania	continue

t	widok celu	kontekst	stan zewnętrzny	obserwacja	działanie	wynik	decyzja o zatrzymaniu
	cytowaniach			tutoringowych		na poziomie odpowiedzi albo wsparcia cytatem	
3	Potrzebne reprezentatywne artykuły	doprecyzowana reguła + tytuły kandydatów	zbiór odsianych zaktualizowany	7 artykułów spełnia regułę, 4 są niepewne	act	otwórz pierwszy istotny artykuł i zapisz notatkę	continue
4	Potrzebna synteza wzorców	dotychczasowe notatki + wynik otwartego artykułu	magazyn notatek ma 1 notatkę; śledzony zbiór niepewnych	pierwszy artykuł kładzie nacisk na zweryfikowane cytaty	think	dodaj do macierzy wymiar „prezentacja materiału dowodowego”	continue

Porównajmy to teraz z wadliwym śladem.

t	widok celu	kontekst	stan zewnętrzny	obserwacja	działanie	wynik	decyzja o zatrzymaniu
2	Potrzebne istotne systemy oparte na cytowaniach	najlepsi kandydaci + reguły	zbiór odsianych pusty	abstrakty pokazują wiele ogólnych systemów tutoringowych	think	doprecyzuj regułę wykluczania	continue
3	Potrzebne reprezentatywne artykuły	tylko tytuły kandydatów	reguła nigdy nie została zapisana do S_t	otwórz ogólny artykuł o RAG	act	notatka sporządzona na podstawie nieistotnego źródła	continue
4	Potrzebna synteza wzorców	tylko bieżąca notatka	nieistotna notatka jest teraz traktowana jako poprawny materiał dowodowy	brak	think	zsyntetyzuj zbyt szerokie twierdzenie o „asystentach opartych na cytowaniach”	continue

Wada jest strukturalna, a nie tajemnicza: kryterium potrzebne do przyszłego sterowania istniało tylko w nietrwałym kontekście.

Dlaczego umiejscowienie stanu ma znaczenie

Umiejscowienie stanu wpływa przynajmniej na pięć rodzajów zawiedzeń:

1. **Pominięty stan.** Ważna informacja nigdy nie zostaje zapisana.
2. **Źle umieszczony stan.** Informacja jest zapisana tam, skąd późniejsze sterowanie nie potrafi jej niezawodnie odczytać.
3. **Przeładowany kontekst.** W każdym wywołaniu utrzymuje się zbyt dużo materiału o niskiej wartości, wypychając to, co ważne teraz.
4. **Niezaobserwowane zmiany świata.** System zakłada, że po działaniu świat pozostał taki sam.
5. **Pomieszczenie planu ze śladem.** Architektura traktuje zamierzone przyszłe kroki tak, jakby już się wydarzyły.

Przykład przewodni jest dość bogaty, by wygenerować wszystkie pięć przypadków. Notatka „artykuł 4 wyglądał na istotny” nie jest tym samym co zapisanie faktu, że artykuł 4 naprawdę został przeczytany. Zaplanowana macierz porównawcza nie jest tym samym co macierz istniejąca już w stanie. Pobrany adres URL nie jest tym samym co dostępny dokument.

Okna kontekstu raz jeszcze

Rozdział 1 wprowadził pojęcie generacji ograniczonej kontekstem. Tutaj jego konsekwencja staje się operacyjna.

Nawet przy długich oknach kontekstu kontekst pozostaje **skonstruowanym widokiem** dla bieżącego wywołania. Nie jest tym samym co trwały stan. Trwały stan wymaga od architektury podjęcia decyzji:

- co przenosić dalej,
- w jakiej reprezentacji,
- według jakich reguł aktualizacji,
- i jak odróżnić bieżący widok od trwałych faktów.

To rozróżnienie stanie się jeszcze ostrzejsze, gdy do gry wejdzie pamięć. Pamięć rozszerzy trwałość. Nie zlikwiduje różnicy między `C_t` a `S_t`.

Plan a ślad wykonania

Dla początkujących jedno z najważniejszych rozróżnień architektonicznych jest następujące:

- **plan** mówi, co system zamierza zrobić,
- **ślad** mówi, co system rzeczywiście zrobił.

Asystent do przeglądu literatury może mieć plan w rodzaju:

1. szukaj szeroko,
2. doprecyzuj zakres,
3. przeczytaj osiem artykułów,
4. zsyntetyzuj wzorce,

5. zweryfikuj cytowania,
6. zatrzymaj się.

Ale ślad może pokazywać coś innego:

1. szerokie wyszukiwanie,
2. ponowne szerokie wyszukiwanie,
3. prośba do użytkownika o doprecyzowanie,
4. lektura czterech artykułów,
5. odkrycie skąpego materiału dowodowego,
6. przekazanie sprawy dalej.

Dobrze zaprojektowana architektura traktuje oba obiekty oddzielnie. W przeciwnym razie system może błędnie założyć ukończenie zadania tylko dlatego, że plan zawierał krok, którego ślad nigdy nie wykonał.

Przykład krok po kroku: śledzenie ograniczonego wykonania

Prześledźmy nieco dłuższy fragment przykładu przewodniego.

- **Krok 0:** system ładuje `G_review` i inicjalizuje stan pustymi polami kandydatów, odsianych elementów, notatek i macierzy.
- **Krok 1:** uruchamia zapytanie wyszukiwawcze. Narzędzie `search` zwraca mieszane wyniki, w tym artykuły o ogólnym retrieval-augmented generation oraz artykuły o jawnym cytowaniu w odpowiedziach.
- **Krok 2:** system zaostrza kryterium odsiewania. Kryterium zostaje zapisane do stanu zewnętrznego.
- **Krok 3:** odsiewa abstrakty i tworzy dwa zbiory: zaakceptowane i niepewne.
- **Krok 4:** otwiera zaakceptowany artykuł i zapisuje ustrukturyzowaną notatkę: wzorzec architektoniczny, metoda grounding, zależność od narzędzi, typ materiału dowodowego, ograniczenie.
- **Krok 5:** sprawdza szkic macierzy porównawczej i zauważa, że wyłonił się wymiar „prezentacja materiału dowodowego dla użytkownika”. Schemat macierzy zostaje zaktualizowany w stanie.
- **Krok 6:** tymczasowo przestaje czytać nowe artykuły i przygotowuje prowizoryczną syntezę wzorców.

Wszystko powyżej zależy od jawnego stanu. Bez zapisanych zbiorów kandydatów, reguł odsiewania, obiektów notatek i schematu macierzy następny krok byłby zmuszony rekonstruować swój świat z częściowego kontekstu.

Przykład w pseudokodzie

```

procedure EXECUTE_STEP(G, S_t):
  O_t <- observe_world_or_tool_result(S_t)
  C_t <- assemble_context(G, S_t, O_t)

  A_t <- choose_next_move(C_t)

  if A_t == act:
    result <- perform_action(S_t)
    S_(t+1) <- write_result_to_state(S_t, result)
  else if A_t == think:
    update <- derive_control_update(C_t)
    S_(t+1) <- merge(S_t, update)
  else if A_t == ask:
    return clarification_request(C_t)
  else:
    return stop_or_handoff(S_t)

  return S_(t+1)

```

Główne pytanie projektowe ukryte w tym pseudokodzie nie dotyczy wywołania modelu. Dotyczy ścieżki zapisu `write_result_to_state`.

Aktualizacja wspólnego artefaktu

Asystent do przeglądania literatury zyskuje teraz drugi widoczny artefakt: jawny **schemat śladu wykonania**.

Artefakt	Bieżąca zawartość		---		---		Schemat śladu		t		widok celu		kontekst
stan zewnętrzny	obserwacja						działanie		wynik		decyzja o zatrzymaniu		
Inwentarz stanu zapytania, lista kandydatów, zbiór odsianych, zbiór niepewnych, notatki, schemat macierzy, znaczniki ukończenia													

Ten inwentarz stanu stanie się podstawą wyborów pamięciowych w rozdziale 4.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Jeśli brakuje jawnego stanu, architektura zachowuje się tak, jakby bieżący kontekst wystarczał. W przykładzie przewodnim prowadzi to do zapomnianych reguł odsiewania, ponownego czytania tych samych artykułów i syntezy bez wystarczającego oparcia.

Jeśli stan jest używany źle, pojawiają się dwa przeciwstawne błędy:

- **zbyt mało stanu:** system gubi kryteria i wyniki;
- **zbyt dużo stanu:** każdy detal jest przenoszony dalej, zasypując to, co ważne dla następnego wyboru.

Projektowanie stanu jest więc selektywne. Dobre architektury zachowują to, czego potrzebuje późniejsze sterowanie, a nie wszystko, co system kiedykolwiek zobaczył.

Tryby awarii, ograniczenia i błędne przekonania

- **„Okno kontekstu równa się pamięci.”** Nie. Kontekst jest bieżącym widokiem. Może zawierać materiał pobrany albo zapisany, ale sam nie jest polityką trwałości.
- **Utożsamianie kontekstu z całym stanem.** Część informacji powinna pozostać zewnętrzna i ustrukturyzowana, nawet jeśli tylko podzbiór trafia do następnego wywołania.
- **Zapominanie o stanie świata.** Wyniki narzędzi mogą zawodzić, dokumenty mogą znikać, a użytkownicy mogą zmieniać ograniczenia. Rzeczywistość zewnętrzna nie jest zamrożona.
- **Mylenie planu ze śladem.** Checklista nie jest dowodem, że kroki rzeczywiście się wydarzyły.

Podsumowanie rozdziału

Ten rozdział uczynił czas widocznym. Działający system agentowy jest sekwencją transformacji stanu, a nie jedną długą myślą. Kontekst, stan zewnętrzny, stan świata i obserwacje odgrywają różne role. Tabele śladu wykonania pokazują, czy role te są respektowane. Przygotowują też grunt pod następny krok: jeśli stan musi przetrwać poza bieżącym wykonaniem albo między sesjami, potrzebna jest architektura pamięci, a nie improwizowane pola przenoszone dalej.

Źródła i dalsza lektura

- (Brown et al., 2020) jako tło dla generatywnego podłoża ograniczonego kontekstem.
- (Packer et al., 2023) jako systemowe omówienie ograniczeń kontekstu i warstw pamięci.
- (Park et al., 2023) jako zapadający w pamięć przykład, w którym obserwacja, planowanie i refleksja zależą od jawnych struktur stanu.

Co ten rozdział dodaje do architektury

- **Dodano:** jawne rozróżnienie między `C_t`, `S_t`, stanem świata i `O_t`, a także standardową tabelę śladu wykonania.
- **Zaadresowany problem:** utrata ważnych kryteriów albo wyników, ponieważ istniały tylko w nietrwałym kontekście.
- **Nowe ryzyko:** przenoszenie zbyt dużej ilości stanu i zaciemnianie tego, co ma znaczenie teraz.

Rozdział 4 — Typy pamięci, trwałość i polityki

AA-S04 — Architektury pamięci i polityki trwałości

Jakie wymagania ujawnia przykład przewodni

Po rozdziale 3 asystent do przeglądu literatury potrafi już przenosić jawny stan w obrębie jednego wykonania. Pojawiają się jednak dwa nowe problemy.

W jednej wersji asystent jest niemal bez pamięci. Ma dobre narzędzia, więc za każdym razem, gdy pojawia się nowe pytanie o syntezę, ponownie wyszukuje, znowu otwiera artykuły i odtwarza notatki. Rezultat jest marnotrawny i kruchy.

W wersji przeciwnej asystent zapisuje wszystko. Szkice zapytań, surowe notatki, stare podsumowania z wcześniejszych tematów, preferencje użytkownika z niezwiązanych sesji i procedury wielokrotnego użytku trafiają do jednego nierozróżnionego magazynu. Skutkiem jest skażenie: nieaktualne notatki przenikają do bieżącego przeglądu, a system nie potrafi odróżnić świeżego materiału dowodowego od starej wygody.

Ten rozdział odpowiada na tę presję projektową, traktując pamięć jako **reprezentację plus politykę**, a nie po prostu jako „więcej miejsca do przechowywania”.

Plan rozdziału

Pamięć rozszerza zarządzanie stanem na dłuższe horyzonty. Kluczowe pytania brzmią:

- co powinno trwać,
- w jakiej postaci,
- kiedy należy to zapisywać,
- kiedy należy to odczytywać,
- jak radzić sobie z aktualnością i skażeniem,
- i kiedy system powinien polegać na narzędziach zamiast na pamięci?

Rozdział odróżnia role pamięci roboczej, epizodycznej, semantycznej oraz proceduralnej lub zewnętrznej i stosuje je do ograniczonego asystenta do przeglądu literatury.

Kluczowe pojęcia i definicje

Pamięć robocza to materiał zadaniowy o krótkim horyzoncie używany podczas bieżącego wykonania: aktywne notatki, listy kandydatów, częściowe wpisy do macierzy i nierozstrzygnięte pytania.

Pamięć epizodyczna przechowuje ślady konkretnych wcześniejszych wykonań lub kroków: jakie zapytania próbowano uruchomić, jakie źródła odsiano, jakiego doprecyzowania udzielił użytkownik i jaki problem wystąpił.

Pamięć semantyczna przechowuje abstrahowalne struktury wiedzy wielokrotnego użytku: stabilne notatki pojęciowe, glosariusze albo zinterpretowane relacje, które mogą pomóc w przyszłych zadaniach.

Pamięć proceduralna lub zewnętrzna przechowuje procedury wielokrotnego użytku, szablony i ustrukturyzowane artefakty poza wywołaniem modelu: szablony przeglądów, schematy notatek, checklisty sprawdzania cytowań i inne wzorce operacyjne.

Polityka pamięci określa wyzwalacze zapisu, wyzwalacze odczytu, reguły aktualności i mechanizmy kontroli zawieżeń.

Najłatwiej pomylić: przechowywanie z pamięcią

Nie każda baza danych czy magazyn plików jest architekturą pamięci. Staje się pamięcią w sensie architektonicznym dopiero wtedy, gdy system ma jawne wybory reprezentacji oraz polityki określające, kiedy zapisywać, odczytywać, odświeżać, ignorować albo zapominać.

Podobnie pamięć nie jest tym samym co stan. Stan może być całkowicie lokalny dla danego wykonania. Pamięć jest zaprojektowaną trwałością wykraczającą poza pojedynczy krok, a często także poza jedno wykonanie.

Najpierw intuicja

Jeśli rozdział 3 pytał: „gdzie ta informacja istnieje teraz?”, rozdział 4 pyta: „co zasługuje na trwałość i według jakich reguł?”.

Przykład przewodniego asystenta do przeglądu literatury pokazuje, dlaczego to drugie pytanie jest ważne. Część informacji warto zachować tylko w obrębie danego wykonania: bieżących kandydatów na artykuły, tymczasowe fragmenty syntezy i dzisiejsze znaczniki niepewności. Inne mogą przydać się później: notatka pojęciowa wyjaśniająca różnicę między pobranym materiałem dowodowym a wsparciem cytowaniem na poziomie odpowiedzi albo szablon procedury przeglądu, który może ustrukturyzować przyszłe zadania. Jeszcze inne rzeczy prawdopodobnie **nie powinny** utrzymywać się w formie wielokrotnego użytku: wczesna błędna interpretacja tematu albo surowa notatka, której twierdzenia nigdy nie zostały zweryfikowane.

Pamięć pomaga wtedy, gdy ogranicza powtarzalną pracę bez psucia przyszłego sterowania. Pamięć szkodzi wtedy, gdy ponownie wprowadza nieaktualny albo słabo ugruntowany materiał tylko dlatego, że jest pod ręką.

Analiza techniczna

Cztery role pamięci w przykładzie przewodnim

Dla asystenta do przeglądu literatury kandydackie magazyny pamięci są następujące:

1. **Tymczasowe notatki robocze** dla bieżącego przeglądu.
2. **Historia zapytań i odsiewania** z bieżących albo niedawnych wykonań.
3. **Notatki pojęciowe wielokrotnego użytku** dotyczące rozróżnień architektonicznych.
4. **Szablony procedur** opisujące, jak przeprowadzać ograniczony przegląd.

Nie wszystkie zasługują na takie samo traktowanie.

Kandydat na magazyn	Rola pamięci	Po co istnieje	Czy powinien trwać poza bieżącym wykonaniem?
Notatki robocze o aktualnie czytanych artykułach	Pamięć robocza	Wspierają bieżącą syntezę	Zwykle nie, chyba że zostaną skondensowane
Historia zapytań i lista odrzuconych elementów	Pamięć epizodyczna	Pozwala uniknąć powtarzania nieudanych wyszukiwań i duplikacji lektury	Czasem, z kontrolą aktualności
Stabilna notatka pojęciowa o „groundingu a formatowaniu cytowań”	Pamięć semantyczna	Pozwala ponownie użyć doprecyzowanego rozróżnienia w przyszłych przeglądach	Tak, jeśli została starannie opracowana
Szablon checklisty przeglądu literatury	Pamięć proceduralna	Pozwala ponownie użyć sprawdzonej procedury	Tak, jeśli jest wersjonowany

Ważna idea architektoniczna jest taka, że **fakty, notatki, plany i procedury nie są tym samym rodzajem rzeczy**. Wrzucenie ich do jednego magazynu z jedną regułą odczytu to proszenie się o problemy.

Reprezentacja ma znaczenie

Założmy, że asystent zapisuje notatkę o artykule jako swobodny tekst:

„Interesujący artykuł. Używa retrieval i podaje referencje. Może pasować.”

To jest słaba reprezentacja. Późniejsze sterowanie nie potrafi łatwo ustalić:

- jakie twierdzenie zostało wsparte,
- czy odwołania były na poziomie odpowiedzi, czy stanowiły jedynie tło,
- czy notatka została zweryfikowana,
- kiedy ją zapisano,
- ani czy artykuł ostatecznie został włączony.

Silniejszą reprezentacją jest notatka ustrukturyzowana:

- identyfikator źródła,
- dopasowanie do tematu,
- metoda groundingu,
- typ materiału dowodowego,
- ograniczenie,
- status weryfikacji,
- data zapisania.

Ta książka celowo omija wnętrze retrieval, embeddingi i algorytmy indeksowania. Punkt architektoniczny jest prostszy: **reprezentacja określa, co późniejsza polityka może zrobić**.

Pamięć to polityka, a nie tylko przechowywanie

Standardowa tabela polityki pamięci używana w tej książce ma postać:

magazyn | zawartość | wyzwalacz zapisu | wyzwalacz odczytu | reguła aktualności | tryb awarii

Tabela 4.1 proponuje minimalną politykę dla przykładu przewodniego.

magazyn	zawartość	wyzwalacz zapisu	wyzwalacz odczytu	reguła aktualności	tryb awarii
Notatnik roboczy	bieżące notatki, szkice wierszy macierzy, nierozstrzygnięte pytania	po każdym kroku czytania albo syntezy	następny krok w tym samym wykonaniu	wyczyść po zakończeniu zadania, chyba że materiał został skondensowany	zaszumiony przeniesiony materiał, bałagan
Historia zapytań	wykonane zapytania, jakość wyników, wyniki odsiewania	po kroku wyszukiwania albo odsiewania	gdy wyszukiwanie wydaje się zablokowane albo powtarzalne	ważne tylko w obrębie tego tematu i zakresu czasu	zakotwiczenie na słabych początkowych zapytaniach
Notatki pojęciowe	starannie opracowane rozróżnienia i definicje architektoniczne	tylko po weryfikacji albo świadomym opracowaniu	gdy zadanie uruchamia to samo pojęcie	przegląd okresowy; nie traktuj jako bieżącego materiału dowodowego	nieaktualne abstrakcje podszywające się pod materiał dowodowy
Szablony procedur	checklista przeglądu, schemat notatek, checklista zatrzymania	gdy stabilna procedura okazuje się przydatna	przy inicjalizacji zadania	aktualizuj, gdy zmienia się polityka	sztynne ponowne użycie dla zadań o innym profilu

Warto zauważyć, czego ta tabela **nie** mówi. Nie mówi: „zapisuj wszystko, bo może się przydać później”. Dobre projektowanie pamięci jest selektywne.

Aktualność i skażenie nieaktualnymi danymi

Dwie z najważniejszych polityk pamięci to:

- **aktualność:** kiedy zapisany element jest jeszcze godny zaufania dla bieżącego zadania?
- **kontrola skażenia:** jak zapobiec temu, by stary albo nieistotny materiał kształtował bieżące wykonanie?

Dla asystenta do przeglądu literatury semantyczna notatka o pojęciu groundingu może pozostać użyteczna przez długi czas. Notatka typu „artykuł X jest reprezentatywnym systemem z 2023 roku” może się szybko zestarzeć, zwłaszcza jeśli bieżący przegląd ma węższą domenę albo nowszy zakres. Historia zapytań z wcześniejszego przeglądu z obszaru ochrony zdrowia nie jest tylko nieaktualna; prawdopodobnie wprowadza też w błąd.

Przydatną regułą jest oddzielanie **pojęć wielokrotnego użytku** od **materiału dowodowego zadania**. Pojęcia wielokrotnego użytku mogą trwać dłużej i być odświeżane wolniej. Materiał dowodowy zadania musi być ściśle ograniczony zakresem i opatrzony znacznikiem czasu.

Kiedy pomaga pamięć, a kiedy lepsze są narzędzia

Pamięć nie zawsze jest właściwą odpowiedzią. Architektura powinna preferować świeżą obserwację z narzędzi wtedy, gdy:

- świat mógł się zmienić,
- materiał dowodowy musi być aktualny,
- koszt ponownego sprawdzenia jest niski,
- a nieaktualna pamięć byłaby ryzykowna.

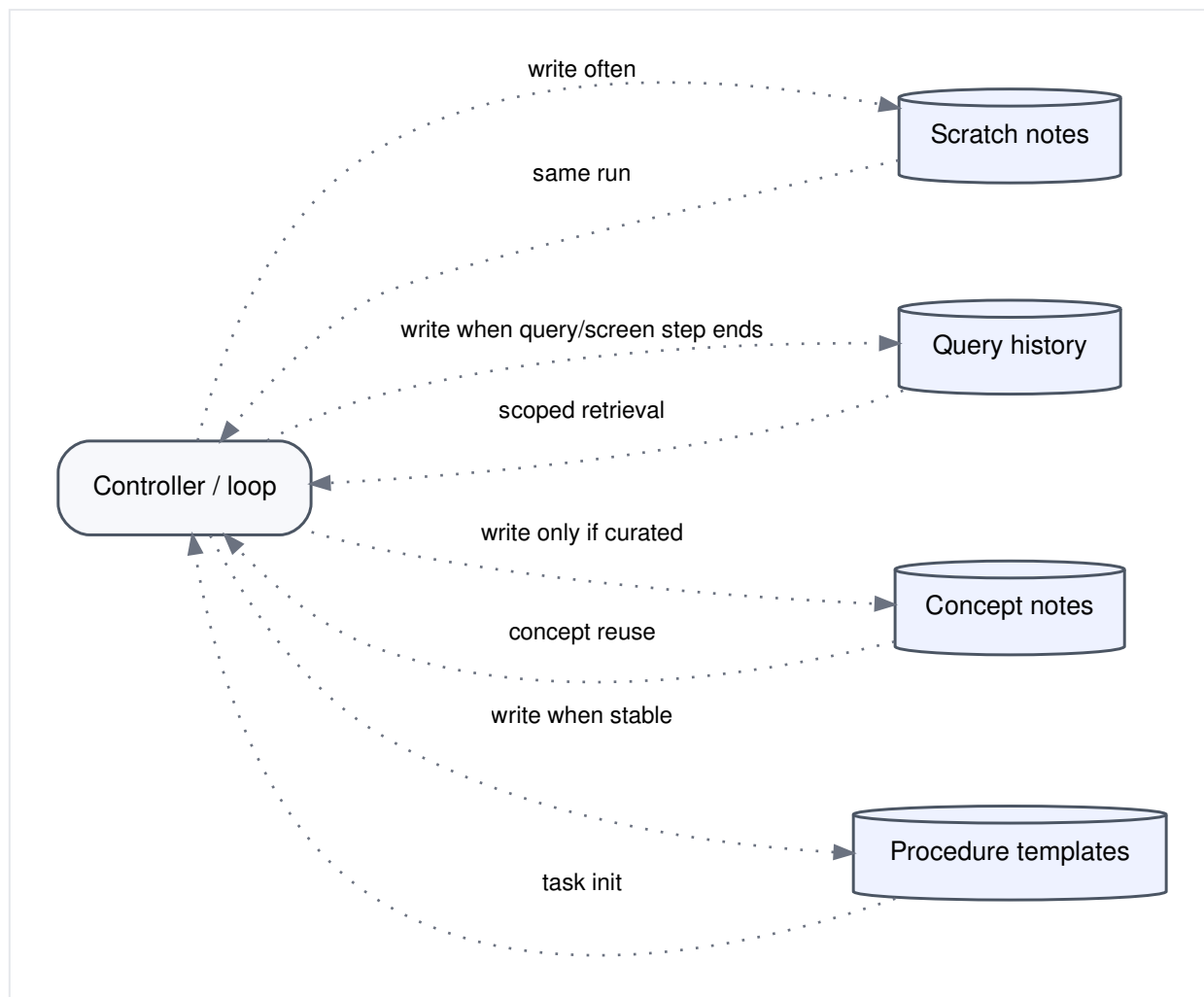
W przykładzie przewodnim często lepiej jest ponownie otworzyć artykuł albo uruchomić zapytanie jeszcze raz, niż zaufać starej, mglistej notatce. Z kolei stabilny szablon procedury, taki jak schemat notatek, jest dokładnie tym rodzajem rzeczy, który pamięć powinna zachowywać.

Ten kompromis prowadzi do porównania, które będzie powracać w całej książce.

Warianty oparte głównie na pamięci i głównie na narzędziach

Ograniczonego asystenta do przeglądu literatury można zbudować jako wariant **bogaty w pamięć i ubogi w narzędzia** albo **ubogi w pamięć i bogaty w narzędzia**. Żaden z tych wariantów nie jest uniwersalnie najlepszy.

Wariant	Siła	Słabość	Typowe użycie
Ubogi w pamięć, bogaty w narzędzia	Świeży materiał dowodowy, mniejsze skażenie, prosta polityka	Powtarza pracę, może być wolny, słaba ciągłość	Krótkie zadania albo bardzo dynamiczny materiał dowodowy
Bogaty w pamięć, ubogi w narzędzia	Ponownie używa struktury i wcześniejszej pracy, mocniejsza ciągłość	Ryzyko nieaktualnych notatek, złożoność polityk, ukryte przecieki	Powtarzalne rodziny zadań ze starannie przygotowanymi magazynami



Rysunek 4.1. Odrębne magazyny z różnymi politykami zapisu i odczytu. Przerywane strzałki pokazują selektywne odczyty i zapisy zamiast bezładnego wrzucania wszystkiego do jednego magazynu.

W praktyce najmocniejszym rozwiązaniem dla przykładu przewodniego jest **minimalna architektura pamięci**: starannie przygotowane szablony procedur, ograniczona zakresem epizodyczna historia zapytań oraz lokalne dla wykonania notatki robocze, przy czym notatki semantyczne są dopuszczane tylko po świadomym opracowaniu.

Przykład krok po kroku: klasyfikowanie kandydatów na magazyny

Klasyfikujemy po kolei kandydackie magazyny z przykładu przewodniego.

Tymczasowe notatki robocze. Należą do pamięci roboczej. Pomagają bieżącej syntezie i zwykle powinny zostać wyczyszczone po zakończeniu zadania, chyba że zostaną skondensowane do czegoś wielokrotnego użytku.

Historia wcześniejszych wyszukiwań albo zapytań. To pamięć epizodyczna. Może zapobiegać powtórzeniom w obrębie wykonania, a czasem także między blisko spokrewnionymi wykonaniami. Wymaga jednak ograniczenia zakresem tematycznym i kontroli aktualności.

Notatki pojęciowe wielokrotnego użytku. To pamięć semantyczna. Powinny być rzadkie i starannie opracowane. Skondensowana notatka o różnicy między „retrieval” a

„prezentacją materiału dowodowego na poziomie odpowiedzi” może pomóc w przyszłych przeglądach. Półzweryfikowane twierdzenie o jednym artykule nie powinno trafiać do pamięci semantycznej.

Szablony procedur przeglądu. To pamięć proceduralna. Niezawodna checklista odsiewania, sporządzania notatek i weryfikacji cytowań jest dobrym kandydatem do trwałości.

Przykład w pseudokodzie

```
procedure MEMORY_WRITE_DECISION(item, store_type):
  if store_type == scratch:
    write(item)
  else if store_type == episodic:
    write_if(item.is_scoped and item.is_likely_reused_soon)
  else if store_type == semantic:
    write_if(item.is_curated and item.is_verified)
  else if store_type == procedural:
    write_if(item.is_generalizable and item.is_stable)

procedure MEMORY_RETRIEVE(query, current_task):
  candidates <- fetch_relevant_items(query)
  return filter(candidates,
    scope_matches(current_task)
    and freshness_ok(current_task)
    and contamination_risk_low)
```

Najważniejsze operacje nie są matematycznie wyrafinowane. To kontrole polityk.

Aktualizacja wspólnego artefaktu

Asystent do przeglądu literatury zyskuje teraz minimalną politykę pamięci.

Artefakt	Bieżąca zawartość
Polityka pamięci	odrębne magazyny dla notatek roboczych, epizodycznej historii zapytań, starannie opracowanych notatek pojęciowych i szablonów procedur
Jawna reguła	nie mieszaj materiału dowodowego zadania, pojęć wielokrotnego użytku i procedur w jednym nierozróżnionym magazynie

Ten rozdział wprowadza też drugie ważne porównanie w książce: wariant **ubogi w pamięć, bogaty w narzędzia** kontra **bogaty w pamięć, ubogi w narzędzia**.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Jeśli pamięci brakuje, system powtarza pracę, zapomina użyteczną strukturę i nie potrafi budować ciągłości w dłuższych zadaniach.

Jeśli pamięć jest używana źle, problemy bywają jeszcze poważniejsze:

1. **nieaktualność**: stare notatki są traktowane jak bieżący materiał dowodowy,

2. **skażenie:** niezwiązane wcześniejsze wykonania przeciekają do bieżącego zadania,
3. **przeciek preferencji:** wcześniejsze założenia zależne od użytkownika albo zadania kształtują nowy przegląd,
4. **zawalenie magazynu:** fakty, plany i procedury stają się nierozróżnialne.

Dla przykładu przewodniego najgroźniejszym problemem nie jest zapominanie. Jest nim pamiętanie niewłaściwej rzeczy w niewłaściwej formie.

Tryby awarii, ograniczenia i błędne przekonania

- **„Każda baza danych jest pamięcią.”** Nie. Baza danych staje się pamięcią dopiero wtedy, gdy reprezentacja i polityka są jawne.
- **„Więcej pamięci zawsze jest lepsze.”** Nie. Więcej źle zarządzanej pamięci często pogarsza sterowanie.
- **Traktowanie faktów, planów i notatek roboczych jak jednego magazynu.** Niszczy to precyzję odczytu i jasność polityk.
- **Ignorowanie aktualności.** Stare notatki mogą być całkowicie spójne, a mimo to nie nadawać się do bieżącego zadania.
- **Używanie pamięci po to, by unikać obserwacji.** Gdy liczy się aktualny materiał dowodowy, narzędzia powinny często wygrywać z zapisanymi przypuszczeniami (Lewis et al., 2020; Packer et al., 2023).

Podsumowanie rozdziału

Pamięć nie jest „dodatkowym kontekstem”. Jest zaprojektowaną trwałością zarządzaną przez jawną politykę. Różne magazyny pełnią różne role, a asystent do przeglądu literatury korzysta najbardziej z małej liczby starannie zarządzanych magazynów, a nie z bezładnej akumulacji. Następny rozdział przechodzi od trwałości do działania: skoro system wie, co próbuje zrobić i co pamięta, to jak wybiera między myśleniem, pytaniem, działaniem i zatrzymaniem?

Źródła i dalsza lektura

- (Lewis et al., 2020) w kontekście jawnej nieparametrycznej pamięci w generacji.
- (Park et al., 2023) jako przykład zapadającej w pamięć architektury, która odróżnia obserwację, pamięć, refleksję i planowanie.
- (Packer et al., 2023) jako warstwowe ujęcie pamięci motywowane ograniczeniami kontekstu.

Co ten rozdział dodaje do architektury

- **Dodano:** odrębne role pamięci i jawną tabelę polityki pamięci.
- **Zaadresowany problem:** powtarzana praca albo improwizowana trwałość, która gubi strukturę.
- **Nowe ryzyko:** nieaktualna, skażona albo zbyt uogólniona pamięć wpływająca na bieżące sterowanie.

Część II — Deliberacja, działanie i regulacja

Pokazuje, jak ograniczeni agenci wybierają działania, korzystają z narzędzi, planują, odbierają informację zwrotną i decydują, czy kontynuować.

Rozdział 5 — Wybór działań, korzystanie z narzędzi i sprzężenie ze środowiskiem

AA-S05 — Wybór działań, narzędzia i interfejsy ze środowiskiem

Jakie wymagania ujawnia przykład przewodni

Asystent do przeglądu literatury ma już specyfikację zadania, jawny stan i minimalną politykę pamięci. W każdym kroku nadal jednak stoi przed kluczową decyzją:

- Czy powinien jeszcze chwilę pomyśleć?
- Czy powinien poprosić użytkownika o doprecyzowanie zakresu?
- Czy powinien wywołać narzędzie search, read albo citation-check?
- Czy powinien się zatrzymać?

To pierwszy moment, w którym architektura naprawdę spotyka świat. Przykład przewodni z tej książki staje się operacyjny dopiero wtedy, gdy system potrafi wybierać między **rozumowaniem**, **pytaniem**, **działaniem** i **zatrzymaniem** przy jawnych założeniach dotyczących interfejsu.

Plan rozdziału

Ten rozdział wprowadza zbiór wyborów `think / ask / act / stop`, kontrakty narzędzi, interfejsy środowiskowe oraz krótkie wprowadzenie do groundingu. Główna lekcja brzmi: narzędzia nie tylko dodają możliwości. Zmieniają to, co system może wiedzieć, jakie skutki uboczne może wywołać i co liczy się jako materiał dowodowy.

Po tym rozdziale będzie można opisać narzędzie nie jako „coś, czego agent może użyć”, lecz jako kontrakt z wejściami, warunkami wstępnymi, skutkami ubocznymi, zwracanymi obserwacjami i ograniczeniami.

Kluczowe pojęcia i definicje

Krok rozumowania aktualizuje interpretację albo sterowanie bez bezpośredniej zmiany środowiska zewnętrznego.

Krok działania zmienia środowisko zewnętrzne albo pobiera z niego informacje przez interfejs narzędziowy.

Krok meta-sterowania zmienia sam tryb sterowania, na przykład przez doprecyzowanie, ponowne planowanie, zatrzymanie albo przekazanie.

Zwięzły zbiór wyborów używany w tej książce to:

`A_t ∈ {think, ask, act, stop}`

Kontrakt narzędzia określa:

- inputs,
- preconditions,
- side effects,
- observation returned,

- limits.

Wynik ma **grounding**, gdy jest odpowiednio powiązany z obserwacjami albo materiałem dowodowym z narzędzi lub ze środowiska. Artefakt tekstowy nie zyskuje groundingu tylko dlatego, że wyszedł z narzędzia.

Najłatwiej pomylić: udane wywołanie narzędzia z udanym zadaniem

Wywołanie narzędzia może zakończyć się sukcesem technicznym, a zadanie nadal może zawieść koncepcyjnie. Narzędzie search może zwrócić wyniki; te wyniki nadal mogą być nieistotne. Narzędzie do zapisu notatek może zapisać notatkę; ta notatka wciąż może nie mieć wystarczającego oparcia. Sukces narzędzia i sukces zadania to różne warstwy oceny.

Najpierw intuicja

Gdy początkujący po raz pierwszy spotykają systemy agentowe, narzędzia często wydają się najbardziej ekscytujące. Search! Przeglądarki! Kalkulatory! Systemy plików! Architektura zaczyna się jednak krok wcześniej. Pytanie nie brzmi „jakie narzędzia są dostępne?”. Brzmi: „w jakich warunkach wywołanie narzędzia jest właściwym następnym ruchem?”.

Asystent do przeglądu literatury pokazuje, dlaczego ma to znaczenie. Jeśli obecna niejednoznaczność dotyczy tego, czy „dla studentów” oznacza systemy tutoringowe czy asystentów wyszukiwania literatury, natychmiastowe wywołanie search może być gorsze niż pytanie do użytkownika. Jeśli bieżące twierdzenie w syntezie potrzebuje wsparcia, narzędzie citation-check może być lepsze niż dodatkowe swobodne rozumowanie. Jeśli materiału dowodowego jest już dość, dalsze wyszukiwanie może być gorsze niż zatrzymanie.

Dobrze zaprojektowany agent nie wybiera działania domyślnie. Uzasadnia działanie.

Analiza techniczna

Schemat wyboru działania

W dowolnym kroku system wybiera spośród czterech szerokich rodzajów ruchu:

- **think:** zrewiduj interpretację, zdecyduj, co ma znaczenie dalej, albo zaktualizuj lokalny stan sterowania;
- **ask:** uzyskaj od użytkownika brakującą informację o zadaniu;
- **act:** zapytaj środowisko albo wpłynąć na nie za pomocą narzędzi;
- **stop:** zakończ wykonanie albo przygotuj przekazanie.

To ujęcie ma znaczenie, bo zapobiega częstemu błędowi projektowemu: traktowaniu każdej niepewności jako sygnału „pomyśl mocniej”. Czasem niepewność powinna uruchamiać obserwację, a nie dalszą generację. Czasem powinna uruchamiać doprecyzowanie albo zatrzymanie.

Możliwości i ograniczenia narzędzi

Narzędzie zmienia jednocześnie dwie rzeczy:

1. co system może **zrobić**,
2. co system może **wiedzieć**.

Dla przykładu przewodniego narzędzia koncepcyjne to:

- **search:** wyszukiwanie kandydackich źródeł,
- **document read:** czytanie treści artykułu,
- **note write:** ustrukturyzowane zapisywanie notatek,
- **citation check:** sprawdzanie, czy twierdzenia są powiązane z rzeczywistymi źródłami.

Każde narzędzie ma możliwości i ograniczenia. Search potrafi pobrać kandydatów, ale nie potwierdza istotności ani faktycznego wsparcia. Czytanie może odsłonić zawartość źródła, lecz tylko dla dostępnych dokumentów. Zapis notatki utrwala interpretację, ale jakość notatki zależy od tego, co przeczytano i jak to przedstawiono. Citation-check może potwierdzić, że cytowane źródło istnieje w zbiorze notatek, ale niekoniecznie, że wspiera całe twierdzenie.

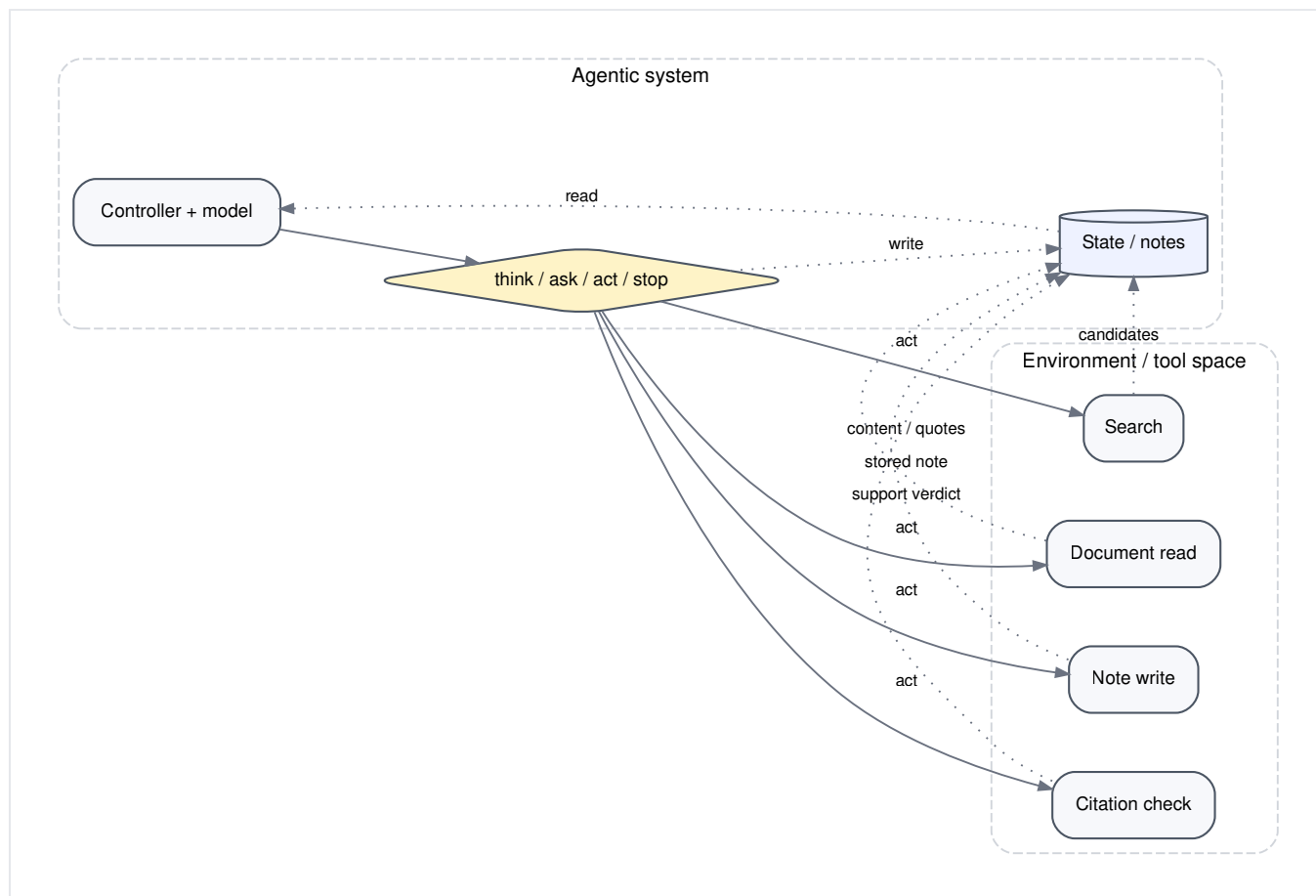
Kontrakty narzędzi w przykładzie przewodnim

Standardowy format tabeli kontraktów narzędzi ma postać:

```
tool | inputs | preconditions | side effects | observation returned | limits
```

Tabela 5.1 szkicuje koncepcyjne kontrakty narzędzi asystenta do przeglądu literatury.

narzędzie	wejścia	warunki wstępne	skutki uboczne	zwracana obserwacja	ograniczenia
Search	zapytanie, zakres czasu, zakres źródeł	zapytanie mieści się w zakresie zadania	brak poza logami retrieval	uporządkowana lista kandydatów z metadanymi	jakość wyników zależy od zapytania i pokrycia indeksu
Document read	identyfikator źródła albo URL	źródło jest osiągalne i dozwolone	może zużyć budżet czasu	abstrakt, sekcje, referencje, cytaty	niedostępne albo zaszumione dokumenty mogą blokować postęp
Note write	identyfikator źródła, pola ustrukturyzowane	źródło zostało wystarczająco przeczytane albo zaobserwowane	zapisuje do magazynu notatek	potwierdzenie zapisania notatki	zła struktura notatki może skamienieć słabą interpretację
Citation check	szkic twierdzenia, kandydackie źródła wspierające	źródła wspierające zostały przeczytane i zindeksowane w stanie	może oznaczyć twierdzenie jako niewspierane	werdykt wsparcia, niedopasowanie albo niejednoznaczność	działa tylko tak dobrze, jak pokrycie źródeł i reguła mapowania



Rysunek 5.1. Sprzężenie ze środowiskiem w przykładzie przewodnim. Granica środowiska ma znaczenie, bo agent może wiedzieć tylko poprzez swoje kanały obserwacji i może wpływać jedynie przez dostępne działania.

Krótkie wprowadzenie do grounding

Grounding nie jest synonimem brzmienia w sposób konkretny. To powiązanie twierdzeń albo decyzji z obserwacjami lub materiałem dowodowym.

Dla asystenta do przeglądu literatury następujące rzeczy są różne:

- „Wiem, że ten artykuł wspiera prezentację cytowań na poziomie odpowiedzi, bo przeczytałem odpowiedni fragment.” — potencjalnie ma grounding.
- „To prawdopodobnie wspiera prezentację cytowań na poziomie odpowiedzi, bo artykuły z tego obszaru zwykle tak robią.” — bez groundingu.
- „Narzędzie search zwróciło ten artykuł.” — obserwacja z groundingiem dotyczącym retrieval, ale nie treści artykułu.
- „Narzędzie citation-check znalazło powiązanie ze źródłem.” — obserwacja z groundingiem dotyczącym mapowania, a nie automatyczny dowód, że twierdzenie jest wspierane.

To rozróżnienie pokazuje, dlaczego wyniki narzędzi trzeba interpretować, a nie tylko powtarzać (Nakano et al., 2021; Menick et al., 2022).

Czasem lepiej zapytać niż działać

Działanie **ask** jest często niedoceniane, bo wydaje się mniej autonomiczne. W systemach ograniczonych to odwrócona intuicja. Pytanie bywa czasem najbardziej inteligentnym ruchem sterującym, jaki jest dostępny.

W przykładzie przewodnim wyobraźmy sobie, że po wstępnej interpretacji fraza „dla studentów” nadal pozostaje niejednoznaczna. Jeśli ta niejednoznaczność zmienia to, które artykuły liczą się jako istotne, pytanie doprecyzowujące jest lepsze niż uruchomienie szerokiego wyszukiwania. Szerokie wyszukiwanie stworzyłoby zaszumionych kandydatów, słabe notatki i zmarnowany wysiłek syntezy. Jedno małe doprecyzowanie może oszczędzić wiele późniejszych działań naprawczych.

Skutki uboczne mają znaczenie

Korzystanie z narzędzi nie jest epistemicznie neutralne. Nawet w systemie zorientowanym na informację narzędzia mogą mieć skutki uboczne:

- zapisywanie notatek zmienia przyszły retrieval i przyszłe sterowanie,
- logowanie nieudanych zapytań może zakotwiczać przyszłe zachowanie wyszukiwawcze,
- edycja szkicu zmienia to, na czym operują późniejsze sprawdzenia cytowań,
- doprecyzowanie od użytkownika staje się częścią operacyjnej specyfikacji zadania.

Dlatego silna architektura traktuje wyniki narzędzi jako obserwacje, a skutki uboczne narzędzi jako zmiany stanu.

Przykład krok po kroku: właściwy następny ruch

Założmy, że asystent do przeglądu literatury wykonał już jedno zapytanie i otrzymał głównie artykuły o ogólnych agentach tutoringowych. Teraz stoi przed trzema kandydackimi następnymi ruchami.

1. **Think**: doprecyzuj zapytanie w świetle niedopasowania.
2. **Ask**: doprecyzuj, czy asystenci tutoringowi wchodzą do zakresu.
3. **Act**: od razu uruchom kolejne zapytanie wyszukiwawcze.

Który z nich jest najlepszy?

- Jeśli niejednoznaczność między asystentami tutoringowymi a asystentami do literatury pozostaje nierozwiązana i silnie zmieniałaby istotność, najlepsze jest **ask**.
- Jeśli intencja użytkownika jest już jasna, ale pierwsze zapytanie było po prostu zbyt szerokie, najlepsze jest **think**, a potem **act**.
- Jeśli materiału dowodowego jest już dość i brakuje tylko wsparcia twierdzeń, najlepsze może być **act** z citation-check.

Ważne jest to, że „użyj narzędzia” nie jest opcją domyślną. To jeden wybór spośród kilku.

Przykład w pseudokodzie

```

procedure CHOOSE_ACTION(G, S_t, O_t):
  if consequential_scope_ambiguity(G, S_t):
    return ask
  if success_criteria_met(G, S_t):
    return stop
  if evidence_gap_detected(S_t):
    return act_with(citation_check)
  if observation_needed_for_progress(S_t):
    return act_with(search_or_read)
  return think

```

Ten pseudokod jest celowo bardziej zbliżony do polityki niż do modelocentrycznego opisu. Architektura może konsultować model podczas podejmowania wyboru, ale *sam wybór* jest tu obiektem projektowym.

Aktualizacja wspólnego artefaktu

Asystent do przeglądu literatury zyskuje teraz kartę kontraktów narzędzi i jawne słownictwo wyboru działań.

Artefakt	Bieżąca zawartość
Kontrakty narzędzi	search, document read, note write, citation check
Zbiór działań	think, ask, act, stop
Krótkie wprowadzenie do groundingu	twierdzenia muszą być powiązane z obserwacjami, a nie tylko z wiarygodnie brzmiącym tekstem

Te artefakty tworzą konkretną przestrzeń działań, na której w następnym rozdziale będzie operować planowanie.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Jeśli architektura nie ma jawnej logiki wyboru działań, pojawia się jeden z dwóch zdegenerowanych wzorców:

- **odruch narzędziowy:** niepewność domyślnie uruchamia użycie narzędzia, nawet wtedy, gdy lepsze byłoby doprecyzowanie albo zatrzymanie;
- **odruch rozumowania:** niepewność uruchamia dalszą generację wewnętrzną, nawet gdy materiał dowodowy musi pochodzić ze środowiska.

Jeśli kontrakty narzędzi są niedookreślone, pojawiają się kolejne problemy:

1. system błędnie odczytuje, co wynik narzędzia rzeczywiście gwarantuje,
2. skutki uboczne są ignorowane,
3. granica środowiska staje się pojęciowo niewidoczna,
4. twierdzenia bez groundingu dziedziczą fałszywy autorytet tylko dlatego, że sąsiadowały z wynikiem narzędzia.

Tryby awarii, ograniczenia i błędne przekonania

- **Użycie narzędzia jako domyślne.** Narzędzia są opcjami, a nie automatycznymi rozwiązaniami.
- **„Udane wywołanie narzędzia oznacza, że zadanie się powiodło.”** Nie. Sukces narzędzia jest tylko jednym składnikiem sukcesu zadania.
- **Ignorowanie skutków ubocznych.** Zapis, logowanie albo edycja zmieniają przyszłe sterowanie.
- **Traktowanie każdego tekstowego wyniku narzędzia jako materiału dowodowego z groundingiem.** Narzędzie może zwrócić tekst, który nadal wymaga interpretacji i weryfikacji.
- **Zapominanie o granicy środowiska.** System nie może wiedzieć ani wpływać na to, czego jego narzędzia nie ujawniają (Schick et al., 2023; Nakano et al., 2021).

Podsumowanie rozdziału

Ten rozdział połączył architekturę ze środowiskiem. Kluczowa lekcja brzmi: wybór działania nie polega na „posiadaniu narzędzi”, lecz na wybieraniu między myśleniem, pytaniem, działaniem i zatrzymaniem przy jawnych kontraktach. Użycie narzędzia zmienia zarówno możliwości, jak i pozycję epistemiczną systemu. To tutaj zaczyna się grounding: twierdzenia zyskują status tylko wtedy, gdy są odpowiednio powiązane z obserwacjami.

Następny rozdział pyta, jak system powinien strukturyzować pracę rozciągniętą na wiele wyborów działań. Gdy przestrzeń działań już istnieje, planowanie staje się możliwe.

Źródła i dalsza lektura

- (Schick et al., 2023) w kontekście modeli uczących się, kiedy używać narzędzi.
- (Nakano et al., 2021) w kontekście zbierania materiału dowodowego za pośrednictwem przeglądarki.
- (Menick et al., 2022) w kontekście wspierania odpowiedzi jawnym materiałem dowodowym.
- (Yao et al., 2022) jako przykład śladów rozumowania i działania, które uwidaczniają wybór działania.

Co ten rozdział dodaje do architektury

- **Dodano:** jawne wybory sterowania `think / ask / act / stop` oraz kontrakty narzędzi.
- **Zaadresowany problem:** mgliste użycie narzędzi, które myli sukces działania z postępem zadania.
- **Nowe ryzyko:** nadaktywne używanie narzędzi albo błędne zaufanie do tekstu zwracanego przez narzędzia.

Rozdział 6 — Planowanie, dekompozycja i ponowne planowanie

AA-S06 — Planowanie, dekompozycja i intuicja przeszukiwania

Jakie wymagania ujawnia przykład przewodni

Asystent do przeglądu literatury wie już, jak wybierać działania i korzystać z narzędzi. Kolejna presja ma charakter strukturalny.

Jedna wersja systemu odpowiada za jednym zamachem. Mało szuka, mało czyta i ma tendencję do spłaszczania tematu do ogólnego podsumowania. Inna wersja dekomponuje zbyt agresywnie do kruchych mikro-kroków: „zapytanie 1, potem zapytanie 2, potem otwórz dokładnie osiem artykułów, potem napisz sekcję A, potem sekcję B”. Załamuje się w chwili, gdy materiał dowodowy nie pasuje do skryptu.

Pytanie architektoniczne nie brzmi, czy planowanie jest dobre w sensie abstrakcyjnym. Brzmi: **kiedy ustrukturyzowane prowadzenie przyszłych kroków poprawia sterowanie na tyle, by uzasadnić jego koszt.**

Plan rozdziału

Ten rozdział przedstawia planowanie jako selektywną strategię sterowania. Wprowadza dekompozycję hierarchiczną, reprezentacje planu, wyzwacze ponownego planowania oraz lekką intuicję przeszukiwania opartą na rozgałęzieniach, spojrzeniu naprzód, cofaniu się i wartości informacji. Asystent do przeglądu literatury służy do porównania trzech trybów:

- odpowiedź bezpośrednia,
- plan stały,
- iteracyjne ponowne planowanie.

Kluczowe pojęcia i definicje

Architektura **odpowiedzi bezpośredniej** wybiera i wykonuje następny ruch bez budowania rozbudowanej jawnej struktury przyszłości.

Plan to zamierzona struktura przyszłości. Może być reprezentowany jako lista, drzewo, checklista albo graf świadomy zależności.

Dekompozycja rozбивa zadanie na podcele albo podproblemy.

Ponowne planowanie rewiduje bieżący plan, ponieważ obserwacje pokazują, że wcześniejszy plan nie pasuje już do zadania albo stanu świata.

Wyzwalacz planowania to warunek, który sprawia, że jawne planowanie staje się warte wykonania: długi horyzont, niepewność, zależności, rzadkie zasoby albo potrzeba porównania gałęzi.

Najłatwiej pomylić: plan ze śladem wykonania

Plan mówi, co system zamierza zrobić dalej. Ślad zapisuje to, co system faktycznie zrobił. Checklista nie staje się śladem tylko dlatego, że została zapisana. To rozróżnienie ma jeszcze większe znaczenie wtedy, gdy zaczyna się ponowne planowanie, bo bieżący plan może się zmieniać, podczas gdy ślad pozostaje stały.

Najpierw intuicja

Planowanie jest wyborem sterującym, a nie synonimem inteligencji.

Zadanie zasługuje na jawne planowanie wtedy, gdy przyszłe kroki zależą od siebie nawzajem na tyle mocno, że myślenie tylko o jednym ruchu naprzód jest kosztowne. Asystent do przeglądu literatury pokazuje kilka powodów, dla których tak się dzieje:

- przegląd ma kilka elementów końcowego rezultatu,
- wybory dotyczące lektury wpływają na późniejszą jakość syntezy,
- wczesny materiał dowodowy może ujawnić, że pierwotny zakres był zbyt szeroki,
- a kroki weryfikacyjne nie powinny zostać zapomniane na końcu.

Planowanie też jednak kosztuje. Zużywa czas, może nadmiernie przywiązać system do wczesnego obrazu zadania i może tworzyć fałszywe poczucie postępu, jeśli plan zostanie pomyłony z samą pracą. Dlatego właściwe pytanie nie brzmi „czy agenci powinni planować?”. Brzmi: „jaka forma planowania, jeśli w ogóle, pasuje do tego profilu zadania?”.

Analiza techniczna

Odpowiedź bezpośrednia, plan stały i ponowne planowanie

Przykład przewodni czyni kontrast bardzo konkretnym.

Odpowiedź bezpośrednia. System wykonuje trochę retrieval i od razu pisze. To może działać dla krótkich, znanych zadań. Zawodzi wtedy, gdy materiał dowodowy trzeba zebrać i porównać w sposób ustrukturyzowany.

Stała dekompozycja. System planuje raz na początku, na przykład:

1. sformułuj zapytania wyszukiwawcze,
2. przeczytaj osiem artykułów,
3. uzupełnij macierz porównawczą,
4. napisz przegląd,
5. sprawdź cytowania,
6. zatrzymaj się.

To jest lepsze wtedy, gdy zadanie ma umiarkowaną strukturę, a materiał dowodowy raczej nie zaskoczy systemu. Jeżeli jednak pierwsze zapytania pokazują, że cytowanie na poziomie odpowiedzi jest dużo rzadsze, niż zakładano, stały plan może nadal wymuszać niepomocny limit „przeczytaj osiem artykułów”.

Iteracyjne ponowne planowanie. System zaczyna od planu, ale koryguje go po ważnych obserwacjach. To często najlepiej pasuje do ograniczonego przeglądu literatury,

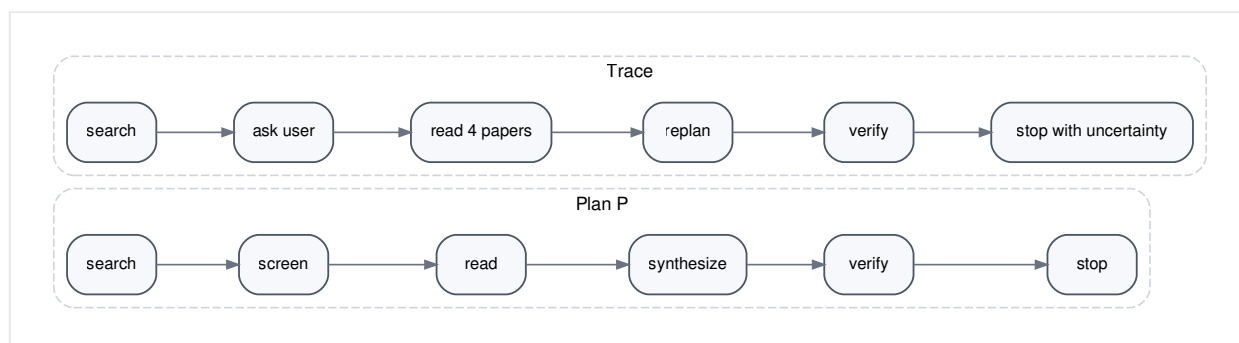
bo jakość materiału dowodowego i granice kategorii stają się wyraźniejsze w miarę wyszukiwania.

Reprezentacje planu

Plan można przedstawić w kilku lekkich formach.

Reprezentacja	Przykład w przykładzie przewodnim	Mocna strona	Słaba strona
Lista	szukaj → przesiej → czytaj → syntetyzuj → weryfikuj	prosta, czytelna	słabo ujmuje zależności
Lista kontrolna	„czy doprecyzowano zapytanie?”, „czy zaktualizowano schemat macierzy?”, „czy wykonano sprawdzenie cytowań?”	dobra do kontroli ukończenia zadania	słaba przy rozgałęzieniach
Drzewo	gałąź wyszukiwania A vs gałąź B, a następnie lektura w obrębie gałęzi	uwidacznia alternatywy	może nadmiernie komplikować małe zadania
Struktura świadoma zależności	nie syntetyzuj końcowych twierdzeń, dopóki nie ma dość zweryfikowanych notatek	dobrze ujmuje warunki wstępne	trudniejsza do zaprojektowania i utrzymania

Dla początkujących najważniejsza lekcja nie dotyczy tego, który formalizm jest najlepszy. Chodzi o to, że różne kształty planu pasują do różnych zadań.



Rysunek 6.1. Plan a ślad. Plan wyraża zamierzoną strukturę przyszłych działań, a ślad zapisuje już zrealizowane zachowanie. Ponowne planowanie zmienia to pierwsze, nie to drugie.

Intuicja przeszukiwania bez formalnych algorytmów

Planowanie często korzysta z odrobiny intuicji przeszukiwania, nawet w systemach niesymbolicznych (Russell and Norvig, 2021). Najbardziej liczą się tu cztery idee.

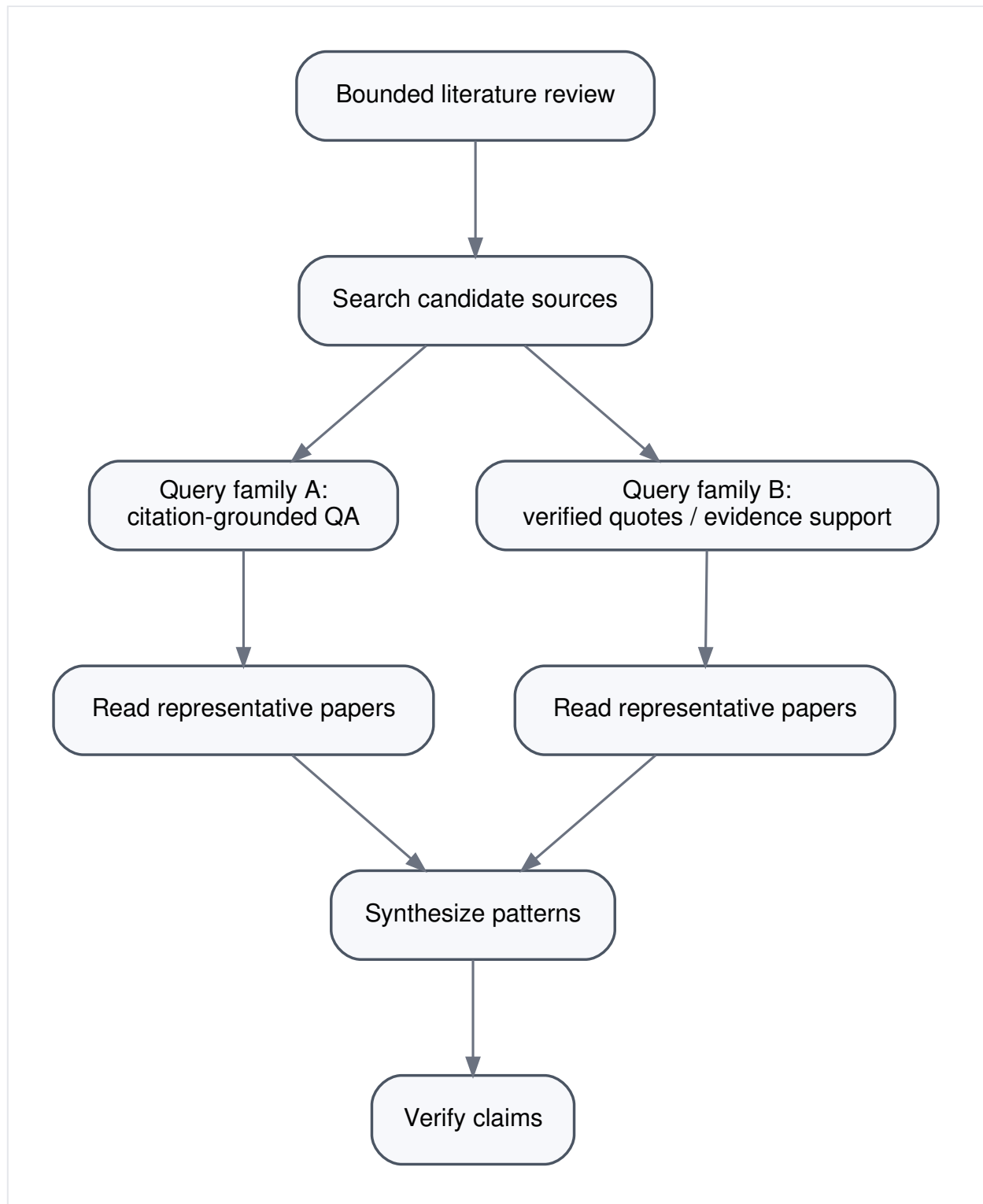
Rozgałęzianie. Może istnieć kilka wiarygodnych dalszych ścieżek. W przykładzie przewodnim jedna rodzina zapytań może koncentrować się na „citation-grounded QA”, inna na „verified quotes”, a jeszcze inna na „evidence-backed generation”.

Spojrzenie naprzód. Zanim system przywiąże się do danej gałęzi, może oszacować, która z nich z większym prawdopodobieństwem dostarczy użytecznego materiału dowodowego.

Cofanie się. Jeśli gałąź zawiedzie, system może wrócić do wcześniejszego wyboru, zamiast na siłę trzymać się nieudanej ścieżki.

Wartość informacji. Czasem najlepszym kolejnym działaniem jest takie, które zmniejsza niepewność, zamiast od razu produkować końcowy artefakt.

Nie są to idee właściwe wyłącznie symbolicznemu AI. To ogólne intuicje sterowania.

Małe przykładowe rozwidlenie przeszukiwania

Rysunek 6.2. Małe przykładowe rozwidlenie przeszukiwania. Nie chodzi tu o optymalność algorytmiczną, lecz o intuicję, że alternatywne gałęzie zapytań można porównywać, porzucać albo do nich wracać.

Wyobraźmy sobie, że asystent rozważa dwie początkowe gałęzie zapytań:

- **Branch A:** “citation-grounded question answering assistants students”
- **Branch B:** “verified quotes educational QA systems”

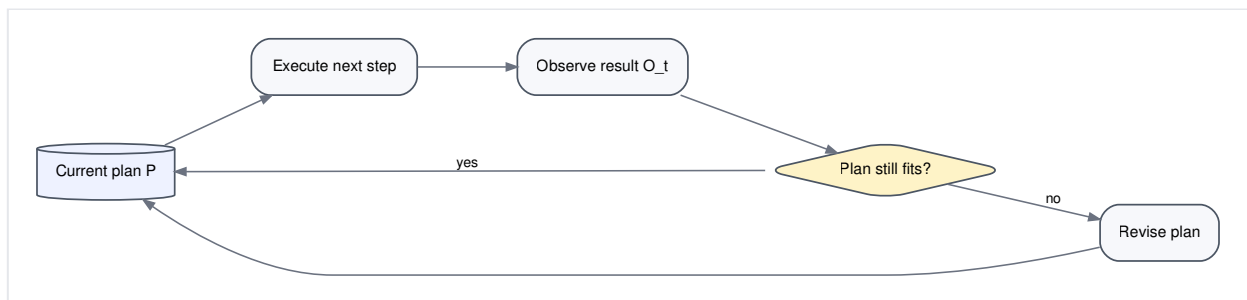
Szybkie spojrzenie naprzód ujawnia, że gałąź A zwraca wiele ogólnych agentów tutoringowych z pobranym kontekstem, ale z niewielką liczbą jawnych cytowań w odpowiedzi. Gałąź B daje mniej wyników, ale więcej bezpośredniego materiału o wsparciu odpowiedzi dowodami. System świadomy planowania może zacząć od gałęzi B, a A zachować w rezerwie dla szerszego pokrycia.

To właśnie jest planowanie w szerokim sensie używanym w tej książce: reprezentowanie alternatyw i podejmowanie kontrolowanego wyboru.

Wyzwalacze ponownego planowania

Ponowne planowanie jest uzasadnione wtedy, gdy obserwacje zmieniają założenia stojące za bieżącym planem. Typowe wyzwalacze w przykładzie przewodnim to:

- początkowe wyniki wyszukiwania są zbyt szerokie albo zbyt skąpe,
- pojawia się nowe rozróżnienie architektoniczne, które zmienia macierz porównawczą,
- ważne źródła okazują się niedostępne,
- materiał dowodowy przeczy wcześniejszej hipotezie roboczej,
- albo próg zatrzymania wyraźnie nie jest osiągalny w bieżących granicach.



Rysunek 6.3. Pętla ponownego planowania. Plan jest rewidowany dopiero wtedy, gdy obserwacje pokazują, że bieżący plan nie pasuje już do zadania albo do materiału dowodowego.

Sama obecność pętli ponownego planowania nie oznacza, że system powinien planować od nowa bez przerwy. Częste, niepotrzebne ponowne planowanie jest osobnym trybem awarii.

Nadmierna fragmentacja

Jednym z najczęstszych błędów początkujących jest zbyt daleko posunięta dekompozycja. Plan zamienia się wtedy w kruchy mikroskrypt:

1. wyszukaj dokładnie trzy razy,
2. otwórz dokładnie osiem abstraktów,
3. napisz dokładnie jedną notatkę na artykuł,
4. wypełnij dokładnie cztery kolumny macierzy,
5. przygotuj dokładnie pięć akapitów.

Wygląda to na uporządkowane, ale często niszczy zdolność adaptacji. Dobra dekompozycja zachowuje strukturę istotną dla decyzji, a jednocześnie zostawia miejsce na osąd w czasie wykonania.

Przykład krok po kroku: trzy wersje tego samego zadania

Rozważmy to samo ograniczone zadanie przeglądu literatury.

Odpowiedź jednorazowa. System pisze odpowiedź po minimalnym zebraniu materiału dowodowego. Jest szybka, ale płytka. Często pomija granice kategorii i poziom niepewności.

Stała dekompozycja. System tworzy plan początkowy i ściśle się go trzyma. Dobrze pilnuje, by pojawiły się wszystkie wymagane elementy rezultatu. Ma trudność wtedy, gdy nowy materiał dowodowy zmienia znaczenie słowa „istotny”.

Iteracyjne ponowne planowanie. System zaczyna od szkieletowego planu:

1. szukaj szeroko,
2. zidentyfikuj kandydackie rodziny architektur,
3. przeczytaj reprezentatywne artykuły,
4. doprecyzuj wymiary macierzy,
5. zweryfikuj twierdzenia,
6. zatrzymaj się albo przekaz sprawę dalej.

Po dwóch lekturach zauważa, że „sposób prezentacji materiału dowodowego” jest ważniejszy, niż wcześniej zakładano, a jedna planowana rodzina ma zbyt mało materiału. Rewiduje więc plan: mniej artykułów ze słabo udokumentowanej rodziny, bardziej ukierunkowane wyszukiwanie wokół „verified quotes” oraz dodatkowa uwaga o niepewności w końcowym rezultacie.

To jest dokładnie ten rodzaj ograniczonej adaptacji, który planowanie powinno wspierać.

Przykład w pseudokodzie

```
procedure PLAN_ACT_REPLAN(G, S_t):
  if no_current_plan(S_t):
    P <- make_initial_plan(G, S_t)

  while true:
    next_step <- choose_from_plan(P, S_t)
    O_t <- execute_or_observe(next_step)
    S_t <- update_state(S_t, O_t)

    if stop_condition_met(G, S_t):
      return finalize(S_t)

    if replan_triggered(P, S_t, O_t):
      P <- revise_plan(G, P, S_t)
```

Pytanie architektoniczne nie brzmi, jak wyrafinowane jest `make_initial_plan`. Brzmi: kiedy uzasadnione jest `revise_plan`.

Aktualizacja wspólnego artefaktu

Asystent do przeglądu literatury zyskuje teraz jawny artefakt planu `P`.

Artefakt	Bieżąca zawartość
Bieżący plan P	ograniczona dekompozycja od wyszukiwania do weryfikacji i zatrzymania
Wyzwalacze ponownego planowania	szeroki albo skąpy materiał dowodowy, zmienione kategorie, niedostępne źródła, sprzeczność, nieosiągalny próg zatrzymania

Ten artefakt będzie później współdziałał z informacją zwrotną i regułami zatrzymania.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Jeśli w zadaniu, które tego wymaga, zabraknie planowania, architektura staje się krótkowzroczna. Może wielokrotnie wybierać lokalnie wiarygodne działania, nie zachowując jednak ogólnej struktury.

Jeśli planowanie zostanie użyte źle, pojawiają się inne problemy:

1. **kult planu:** system traktuje plan jak prawdę, zamiast go rewidować,
2. **nadmierna fragmentacja:** kruche mikro-kroki zastępują osąd,
3. **mylenie planu ze śladem:** zapisany zamiar jest brany za wykonaną pracę,
4. **rozchwianie ponownego planowania:** plan zmienia się tak często, że praca nigdy się nie stabilizuje.

Przykład przewodni potrzebuje planowania, ale tylko w ograniczonej formie, którą można rewidować.

Tryby awarii, ograniczenia i błędne przekonania

- **„Planowanie zawsze jest lepsze.”** Nie. Planowanie zwiększa narzut i może nadmiernie dopasować się do wczesnych założeń.
- **Mylenie planu ze śladem wykonania.** Naszkicowana struktura nie jest dowodem postępu.
- **Nadmierne rozbijanie zadań.** Zbyt wiele mikro-kroków może zamienić adaptację w kruchość.
- **Traktowanie intuicji przeszukiwania jako czegoś właściwego tylko symbolicznemu AI.** Rozgałęzianie, spojrzenie naprzód i cofanie się to ogólne idee sterowania, a nie własność jednej tradycji historycznej (Russell and Norvig, 2021; Huang et al., 2022).
- **Ignorowanie wartości zbierania informacji.** Niektóre działania są warte wykonania dlatego, że wyostrzają późniejsze wybory, a nie dlatego, że od razu produkują rezultat końcowy.

Podsumowanie rozdziału

Planowanie jest selektywną strategią sterowania nad przestrzenią działań. Poprawne porównanie nie przebiega między „zaplanowanym” a „inteligentnym”, lecz między odpowiedzią bezpośrednią, stałą dekompozycją i iteracyjnym ponownym planowaniem. Asystent do przeglądu literatury korzysta z niewielkiego, rewidowalnego planu, ponieważ

jakość materiału dowodowego i granice kategorii ujawniają się dopiero w trakcie działania.

Następny rozdział pyta, w jaki sposób architektura decyduje, czy bieżący plan, notatki albo twierdzenia pośrednie są na tyle wiarygodne, by dalej za nimi podążać. To właśnie tu wchodzi grounding, informacja zwrotna i strukturalna analiza błędów.

Źródła i dalsza lektura

- (Wang et al., 2023) o jawnych promptach planujących w zadaniach rozumowania.
- (Yao et al., 2023) o rozgałęzieniach i świadomej eksploracji pośrednich toków myślenia.
- (Huang et al., 2022; Huang et al., 2022) o planowaniu i informacji zwrotnej w ustawieniach ucieleśnionych.
- (Russell and Norvig, 2021) jako sąsiednie tło dla klasycznej intuicji przeszukiwania i planowania.

Co ten rozdział dodaje do architektury

- **Dodano:** jawny plan P oraz wyzwalacze ponownego planowania.
- **Zaadresowany problem:** czysto lokalny wybór następnego kroku w zadaniach z zależnościami i niepewnością materiału dowodowego.
- **Nowe ryzyko:** kruche albo nadaktywne planowanie, które myli strukturę z postępem.

Rozdział 7 — Grounding, informacja zwrotna, refleksja i analiza błędów strukturalnych

AA-S07 — Grounding, informacja zwrotna, refleksja i analiza błędów strukturalnych

Jakie wymagania ujawnia przykład przewodni

Asystent do przeglądu literatury ma już cele, stan, pamięć, narzędzia i plan, który można korygować. Nadal jednak może zawodzić w znany sposób: system wciąż się porusza, ruch wygląda na uporządkowany, ale wynik nie jest godny zaufania.

W jednym wykonaniu system pisze dopracowaną syntezę, twierdząc, że „najnowsze systemy zwykle wspierają odpowiedzi zweryfikowanymi cytatami”. Krok citation-check potwierdza, że cytowane artykuły istnieją, ale bliższa analiza pokazuje, że tylko część z nich rzeczywiście daje wsparcie na poziomie cytatu. Architektura miała ogólny krok refleksji — „przejrzyj odpowiedź pod kątem jakości” — ale nie miała weryfikacji niosącej materiał dowodowy, powiązanej z samym twierdzeniem.

W innym wykonaniu do bieżącego przeglądu przecieka nieaktualna notatka pojęciowa z wcześniejszego zadania, a system z dużą pewnością przesadnie uogólnia wzorzec, którego obecny materiał dowodowy nie wspiera.

Takich zawieżeń nie rozwiązuje polecenie modelowi, by „był ostrożniejszy”. Wymagają one lepszego sprzężenia między twierdzeniami, materiałem dowodowym, informacją zwrotną i sterowaniem.

Plan rozdziału

Ten rozdział pogłębia grounding i wprowadza informację zwrotną jako szerszą klasę sygnałów, które mogą zmieniać to, co system robi dalej. Odróżnia refleksję od weryfikacji i wykorzystuje taksonomię błędów strukturalnych do diagnozowania problemów na poziomie celów, stanu, pamięci, planowania, działania, groundingu i oceny.

Główna lekcja jest prosta: **refleksja pomaga tylko wtedy, gdy odpowiada przed materiałem dowodowym i jest powiązana z działaniem korygującym.**

Kluczowe pojęcia i definicje

Luka w groundingu istnieje wtedy, gdy twierdzenie albo decyzja są niewystarczająco powiązane z obserwacjami albo materiałem dowodowym.

Luka dowodowa istnieje wtedy, gdy architekturze brakuje obserwacji potrzebnych do uzasadnienia bieżącego twierdzenia albo następnego kroku.

Informacja zwrotna to dowolny sygnał, który może zmienić sterowanie: wyniki narzędzi, odpowiedzi środowiska, korekty od użytkownika, kontrole wewnętrzne albo jawne heurystyki oceny.

Refleksja to wewnętrzny przegląd bieżącego wyniku, planu albo stanu.

Weryfikacja to sprawdzanie niosące materiał dowodowy. Bada, czy twierdzenie, notatka albo decyzja są wspierane przez odpowiednie obserwacje.

Po działaniu system może zachować **resztkową niepewność**. Dobre architektury ujawniają tę niepewność zamiast zamieniać ją w nieuzasadnioną pewność.

Najłatwiej pomylić: refleksję z weryfikacją

Refleksja pyta: „Czy to wygląda poprawnie, kompletnie albo spójnie?”. **Weryfikacja** pyta: „Jaki materiał dowodowy wspiera to twierdzenie albo decyzję i czy rzeczywiście do nich pasuje?”. Refleksja może poprawić styl albo strukturę. Weryfikacja wiąże wyniki ze światem.

Najpierw intuicja

Ogólny prompt do samokrytyki potrafi czasem poprawić wynik (Madaan et al., 2023). W architekturze ważne pytanie nie brzmi jednak, czy samokrytyka czasem pomaga. Brzmi: czy system ma dostęp do **właściwego materiału dowodowego** i czy ta krytyka może uruchomić **właściwą naprawę**.

Asystent do przeglądu literatury czyni to bardzo konkretnym. Załóżmy, że szkic mówi: „Większość najnowszych edukacyjnych asystentów QA podaje jawne cytowania w odpowiedziach końcowych”. Ogólny krok refleksji może zapytać, czy to zdanie nie jest zbyt szerokie. System może nawet trochę je złagodzić. Ale dopóki architektura nie potrafi przejrzeć zbioru notatek, zmapować każdej klauzuli na źródła wspierające i wykryć brakujący albo sprzeczny materiał dowodowy, refleksja nadal unosi się ponad rzeczywistym zadaniem.

To grounding i informacja zwrotna sprawiają, że refleksja zaczyna za coś odpowiadać.

Analiza techniczna

Źródła informacji zwrotnej

Dla ograniczonego przeglądu literatury użyteczna informacja zwrotna może pochodzić z czterech źródeł.

1. **Narzędzia.** Jakość search, dostępność dokumentów, wyniki citation-check.
2. **Środowisko.** Brakujące dokumenty, zmieniona dostępność źródeł, sprzeczna treść.
3. **Użytkownik.** Doprecyzowania, korekty, zmienione priorytety.
4. **Kontrole wewnętrzne.** Testy pokrycia, mapowanie wsparcia, kontrole aktualności, wykrywanie sprzeczności.

Te sygnały są użyteczne tylko wtedy, gdy architektura kieruje je do decyzji. System, który loguje informację zwrotną, ale nigdy nie pozwala jej zmienić sterowania, nie jest naprawdę napędzany informacją zwrotną.

Grounding twierdzeń na podstawie obserwacji

Przydatną dyscypliną architektoniczną jest traktowanie każdego istotnego twierdzenia jako wymagającego ścieżki dowodowej.

Dla przykładu przewodniego twierdzenie w rodzaju „systemy oparte na zweryfikowanych cytatach poświęcają pokrycie na rzecz silniejszego wsparcia cytowaniami” powinno idealnie łączyć się z:

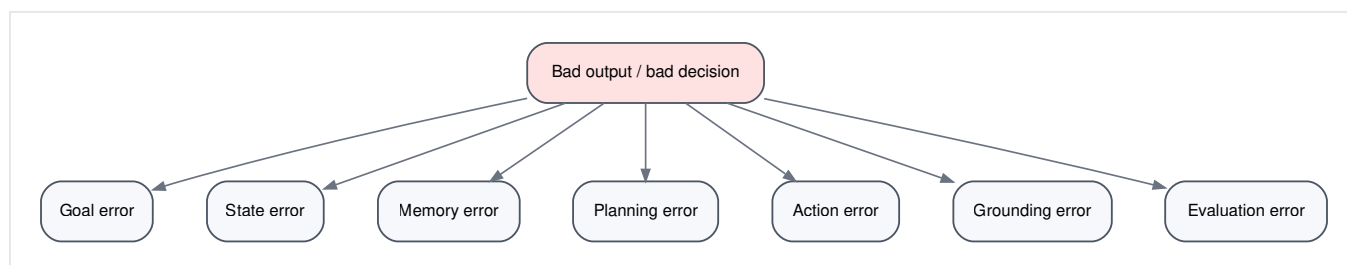
- konkretne przeczytane artykuły,
- pola notatek wychytujące istotną własność architektoniczną,
- a być może także pole sprzeczności albo ograniczenia, jeśli materiał dowodowy jest mieszany.

To nie wymaga formalnej maszyny dowodowej. Wymaga od architektury zachowania użytecznego mapowania między twierdzeniami a źródłami.

Taksonomia błędów strukturalnych

Aby diagnozować problemy bez dryfowania w stronę oceny opartej głównie na benchmarkach, książka używa taksonomii strukturalnej.

Klasa błędu	Typowy objaw w przykładzie przewodnim	Prawdopodobna przyczyna źródłowa	Typowe przeprojektowanie
Błąd celu	przegląd odpowiada na niewłaściwe pytanie	specyfikacja zadania jest zbyt mglista albo źle zrozumiana	doprecyzuj albo skompiluj G na nowo
Błąd stanu	reguła wykluczania znika w środku wykonania	kryterium żyło tylko w kontekście	zapisz regułę do S_t
Błąd pamięci	nieaktualna notatka kształtuje bieżącą syntezę	słaba polityka aktualności albo skażenia	rozdziel magazyny, dodaj kontrole aktualności
Błąd planowania	system podąża za nieaktualnym planem	słaby wyzwalacz ponownego planowania	zrewiduj warunki replanu
Błąd działania	zbyt wiele wyszukiwań, zbyt mało czytania	słaba polityka wyboru think/ask/act/stop	popraw wybór działań
Błąd groundingu	twierdzenie brzmi wiarygodnie, ale brakuje wsparcia	brak mapowania twierdzenie-materiał dowodowy	dodaj krok weryfikacji
Błąd oceny	system zbyt wcześnie ogłasza sukces	słabe kryteria sukcesu albo zatrzymania	zaostż kontrole sukcesu



Rysunek 7.1. Taksonomia błędów strukturalnych dla analizy ograniczonych agentów. Ten sam zły wynik może wynikać z różnych przyczyn źródłowych; diagnozę należy zaczynać od architektury, a nie od szukania winnego.

Ta taksonomia ma znaczenie, bo słaby wynik końcowy nie mówi jeszcze, który komponent zawiódł. Stwierdzenie „model halucynował” jest często zbyt grube, by było użyteczne. Architektura mogła po prostu całkowicie pominąć ścieżkę dowodową.

Ślad zawodzenia 1: słaba weryfikacja oparcia w źródłach

Rozważmy następujący fragment śladu.

t	obserwacja	działanie	wynik
5	cztery notatki wspominają retrieval; jedna notatka wspomina „verified quotes”	think	szkic zdania syntezy: „najnowsze systemy zwykle wspierają odpowiedzi zweryfikowanymi cytatami”
6	ogólny prompt refleksyjny zwraca „wygląda wiarygodnie”	think	zdanie zostaje zachowane
7	citation-check wykrywa, że cytowane artykuły istnieją w zbiorze notatek	act	zdanie zostaje potraktowane jako wspierane

Architektura zawiodła, bo jej cel weryfikacji był zbyt słaby. Sprawdziła, czy cytowane źródła są obecne, a nie czy **twierdzenie** rzeczywiście do nich pasuje.

Lepszy projekt wymagałby, aby zdanie syntezy wskazywało identyfikatory notatek wspierających, i odrzucałby albo osłabiał twierdzenie, gdy mapowanie byłoby niewystarczające.

Ślad zawodzenia 2: skażenie nieaktualnymi notatkami

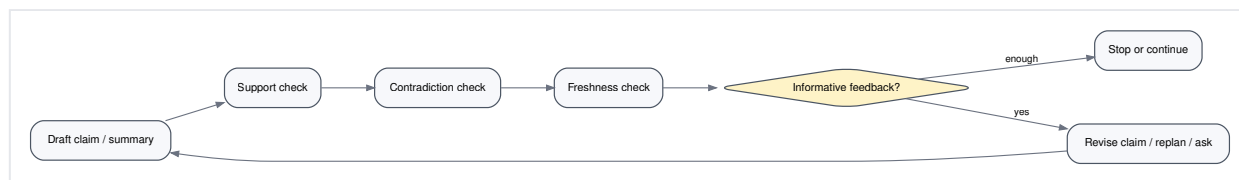
Rozważmy teraz inne wykonanie. System pobiera notatkę semantyczną ze starszego przeglądu o systemach medycznych QA, w której zapisano, że „retrieval augmentation jest zwykle przedstawiane jako grounding”. Bieżące zadanie jest węższe: chodzi o wsparcie cytowaniami na poziomie odpowiedzi dla studentów. Stara notatka obciąża syntezę i system nadmiernie uogólnia.

To nie jest przede wszystkim problem generacji. To problem polityki pamięci. Zła abstrakcja została pobrana dla niewłaściwego zakresu.

Projektowanie użytecznych pętli informacji zwrotnej

Pętla informacji zwrotnej jest użyteczna tylko wtedy, gdy mówi systemowi coś, na podstawie czego da się działać. Ogólna samokrytyka często tego testu nie przechodzi. Bardziej informacyjne pętli w przykładzie przewodnim obejmują:

- **kontrola pokrycia:** czy przeczytano dość reprezentatywnych wzorców architektonicznych?
- **sprawdzenie wsparcia:** czy każde istotne twierdzenie szkicu mapuje się na co najmniej jedno zweryfikowane źródło?
- **kontrola aktualności:** czy pobrana notatka albo szablon pasują do bieżącego zakresu?
- **kontrola sprzeczności:** czy notatki źródłowe nie zgadzają się co do wzorca albo ograniczenia?
- **kontrola ukończenia:** czy bieżące artefakty spełniają `G_review`?



Rysunek 7.2. Zrewidowana pętla informacji zwrotnej. Refleksja zostaje zachowana, ale napędzają ją jawne kontrole wsparcia, kontrole sprzeczności i kontrole ukończenia, a nie sama ogólna samokrytyka.

Warto zauważyć, co się tutaj zmienia. Informacja zwrotna nie jest już tylko komentarzem do szkicu. Staje się wyzwalaczem ponownego planowania, dodatkowej lektury, rewizji twierdzeń, doprecyzowania, zatrzymania albo przekazania.

Refleksja po sprawdzeniu źródeł, nie zamiast sprawdzania źródeł

Refleksja nadal ma swoją rolę. Może pomóc asystentowi zauważyć brakujące wymiary w macierzy porównawczej, słabe przejścia, język nadmiernie kategoriyczny albo nierozstrzygniętą niejednoznaczność. Jej wartość jest jednak największa **po tym**, jak architektura ujawni odpowiedni materiał dowodowy i niepewność.

W tym sensie refleksję najlepiej często traktować jako konsumenta ustrukturyzowanego materiału dowodowego, a nie jego zamiennik (Shinn et al., 2023; Madaan et al., 2023).

Przykład krok po kroku: przeprojektowanie pętli

Założmy, że asystent do przeglądu literatury kończy szkic i uruchamia dwie kontrole.

1. **Kontrola mapowania wsparcia.** Dwa zdania w podsumowaniu nie mają silnego wsparcia na poziomie notatek.
2. **Kontrola sprzeczności.** Trzy artykuły nie zgadzają się co do tego, czy cytowania w tekście poprawiały zaufanie do faktów w zastosowaniach studenckich.

Teraz architektura ma informacyjne opcje:

- zrewidować zdania bez wsparcia,
- przeczytać jeden dodatkowy artykuł skierowany na obszar sprzeczności,
- jawnie ujawnić rozbieżność,
- albo przekazać sprawę dalej, jeśli konflikt wykracza poza kompetencję systemu albo dostępny materiał dowodowy.

Kluczowa różnica polega na tym, że informacja zwrotna jest powiązana z konkretnymi działaniami naprawczymi. Sama ogólna samokrytyka nie powiedziała by systemowi, który ruch jest uzasadniony.

Przykład w pseudokodzie

```

procedure VERIFY_AND_REFLECT(draft, S_t):
  unsupported <- find_claims_without_support(draft, S_t.notes)
  contradictions <- detect_contradictions(S_t.notes)
  stale_items <- find_stale_retrievals(S_t.memory_reads)

  if unsupported or contradictions or stale_items:
    return feedback_packet(unsupported, contradictions, stale_items)

  reflections <- reflect_on_structure_and_clarity(draft, supported_by=S_t.notes)
  return feedback_packet(reflections)

```

Ważnym wynikiem jest **pakiet informacji zwrotnej**, a nie introspekcyjna proza. Ten pakiet musi sterować dalszym działaniem.

Aktualizacja wspólnego artefaktu

Asystent do przeglądu literatury zyskuje teraz jawne reguły informacji zwrotnej i macierz porażek strukturalnych.

Artefakt	Bieżąca zawartość
Reguły informacji zwrotnej	sprawdzenie wsparcia, kontrola sprzeczności, kontrola aktualności, kontrola ukończenia
Macierz porażek	taksonomia strukturalna obejmująca cele, stan, pamięć, planowanie, działanie, grounding i ocenę

Ten rozdział wraca też do analizy błędów jako lekkiej formy oceny: chodzi o zlokalizowanie przyczyn źródłowych i przeprojektowanie pętli, a nie o tworzenie tabel benchmarkowych.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Jeśli grounding i informacja zwrotna są słabe, architektura staje się zorientowana na wynik zamiast na materiał dowodowy. Wciąż produkuje tekst, ale bez mocnego powodu, by mu ufać albo go korygować.

Jeśli refleksja jest używana źle, często pojawiają się następujące problemy:

1. **ogólna krytyka bez ścieżki dowodowej**,
2. **inflacja pewności**, w której dopracowaną prozę bierze się za zweryfikowane wsparcie,
3. **błędna diagnoza**, w której za każdy problem obwinia się model zamiast architektury,
4. **niekontrolowany pozór postępu**, w którym udane kroki pośrednie bierze się za sukces całego zadania.

Tryby awarii, ograniczenia i błędne przekonania

- **„Po prostu dodaj refleksję.”** Refleksja bez materiału dowodowego często produkuje ładniejsze sformułowania, a nie bardziej wiarygodne wyniki.

- **Pewność nie oznacza poprawności.** Silny model może wygenerować pewną siebie, spójną, ale niewspieraną syntezę.
- **Obwinianie modelu za porażki architektoniczne.** Brakujący stan, nieaktualna pamięć albo słaba weryfikacja to problemy projektowe systemu.
- **Traktowanie każdego udanego kroku pośredniego jak ostatecznego sukcesu.** Dobra notatka, udane wyszukiwanie albo technicznie poprawne powiązanie cytowania nie kończą zadania (Menick et al., 2022; Nakano et al., 2021).

Podsumowanie rozdziału

Ten rozdział uczył system, jak poddawać się korekcie, a nie tylko jak iść dalej. Grounding wiąże twierdzenia i decyzje z obserwacjami. Informacja zwrotna jest szerszą klasą sygnałów, które zmieniają sterowanie. Refleksja jest użyteczna, ale tylko wtedy, gdy konsumuje materiał dowodowy i wskazuje działanie korygujące. Analiza błędów strukturalnych pomaga diagnozować porażki na właściwym poziomie: celu, stanu, pamięci, planowania, działania, groundingu albo oceny.

Następny rozdział domyka opowieść o sterowaniu, pytając, jak system decyduje, czy kontynuować, pytać, zatrzymać się czy przekazać sprawę dalej.

Źródła i dalsza lektura

- (Menick et al., 2022) w kontekście jawnego wsparcia materiałem dowodowym.
- (Nakano et al., 2021) w kontekście zbierania materiału dowodowego wraz z referencjami.
- (Madaan et al., 2023; Shinn et al., 2023) w kontekście iteracyjnych metod refleksji oraz ich mocnych i słabych stron.
- (Huang et al., 2022) jako przykład informacji zwrotnej kształtującej sterowanie w ustawieniach interaktywnych.

Co ten rozdział dodaje do architektury

- **Dodano:** weryfikację niosącą materiał dowodowy, reguły informacji zwrotnej i taksonomię błędów strukturalnych.
- **Zaadresowany problem:** ogólna samokrytyka, która nie potrafi diagnozować ani naprawiać niewspieranych wyników.
- **Nowe ryzyko:** nadmierne sprawdzanie albo źle zaprojektowana informacja zwrotna, która spowalnia wykonanie bez dostarczania informacji nadającej się do działania.

Rozdział 8 — Zatrzymanie, doprecyzowanie, przekazanie i ograniczona autonomia

AA-S08 — Zatrzymanie, przekazanie i ograniczona autonomia

Jakie wymagania ujawnia przykład przewodni

Częsta naiwna wersja asystenta do przeglądu literatury nie ma uzasadnionego zakończenia. Wciąż wyszukuje, bo być może istnieje jeszcze więcej materiału dowodowego, albo wciąż przepisuje, bo szkic może się poprawić. Inna wersja zatrzymuje się zbyt wcześnie po znalezieniu kilku artykułów, mimo że specyfikacja zadania wymagała macierzy porównawczej i jawnych uwag o niepewności. Trzecia wersja nigdy niczego nie pyta użytkownika i każdą niejednoznaczność traktuje jako problem do rozwiązania wewnątrz.

To nie są produkcyjne drobiazgi. To porażki projektowe na poziomie koncepcji. Silna architektura definiuje nie tylko to, jak kontynuować, ale także kiedy pytać, kiedy się zatrzymać i kiedy przekazać sprawę dalej.

Plan rozdziału

Ten rozdział domyka główną opowieść o sterowaniu. Odróżnia ukończenie od porzucenia, wprowadza progi wystarczalności i niepewności oraz traktuje doprecyzowanie i przekazanie jako pełnoprawne ruchy architektoniczne. Łączy też te idee z **ograniczoną autonomią**: zgodą na dalsze działanie w ramach jawnych ograniczeń.

Po tym rozdziale będzie można przeczytać opis agenta i zapytać: skąd on wie, kiedy już wystarczy, kiedy niejednoznaczność wymaga użytkownika i kiedy dalsze działanie przekroczyłoby jego zaprojektowaną władzę decyzyjną?

Kluczowe pojęcia i definicje

Ukończenie oznacza, że zostały spełnione kryteria sukcesu ze specyfikacji zadania.

Porzucenie oznacza, że system zatrzymał się bez wykonania zadania, często dlatego, że nie potrafi zrobić uzasadnionego postępu.

Próg wystarczalności to punkt, w którym dostępny materiał dowodowy i artefakty wystarczają do ukończenia.

Próg niepewności to punkt, w którym niepewność jest zbyt wysoka, by kontynuować bez doprecyzowania, dalszego materiału dowodowego albo przekazania.

Doprecyzowanie to kontrolowany powrót użytkownika do procesu w celu rozwiązania niejednoznaczności mającej znaczenie.

Przekazanie to przeniesienie sterowania albo odpowiedzialności, ponieważ bieżący system nie powinien dalej działać samodzielnie.

Ograniczona autonomia oznacza, że system może działać dalej w ramach jawnych ograniczeń. Nie oznacza otwartej inicjatywy bez granic.

Przydatnym rozróżnieniem pojęciowym jest podział na działania **odwracalne** i **nieodwracalne**. Zbieranie informacji i pisanie szkicu są zwykle odwracalne. Publikowanie, usuwanie, zatwierdzanie albo wysyłanie powiadomień mogą być nieodwracalne. Nawet w asystencie zorientowanym na informację klasa działania kształtuje politykę autonomii.

Najłatwiej pomylić: ukończenie z zatrzymaniem i przekazaniem

Ukończenie jest własnością stanu zadania: kryteria sukcesu zostały spełnione. **Zatrzymanie** jest decyzją sterującą: system nie będzie kontynuował. **Przekazanie** jest szczególnym rodzajem zatrzymania, w którym praca zostaje eskalowana albo zwrócona człowiekowi lub innemu komponentowi. System może się zatrzymać, bo zadanie zostało ukończony, albo dlatego, że nie powinien działać dalej.

Najpierw intuicja

Początkujący często traktują zatrzymanie jako późniejszy dodatek: skoro zbudowano już „interesujące” części wyszukiwania, pamięci i planowania, to architektura jakoś sama dokończy pracę. W praktyce to właśnie zatrzymanie zamienia niekontrolowaną aktywność w zachowanie ograniczone.

Asystent do przeglądu literatury potrzebuje logiki zatrzymania przynajmniej dla czterech typowych sytuacji:

1. **wystarczające ukończenie**: zebrano dość materiału dowodowego, a wynik spełnia kryteria sukcesu;
2. **słaby materiał dowodowy**: asystent nie może odpowiedzialnie działać dalej, bo baza źródeł jest zbyt słaba;
3. **niejednoznaczny zakres**: asystent może kontynuować dopiero po doprecyzowaniu;
4. **sprzeczne źródła**: asystent może opisać rozbieżność, ale po przekroczeniu pewnego punktu powinien przekazać sprawę dalej, zamiast wymyślać rozwiązanie.

Zatrzymanie nie dotyczy więc tylko zakończeń. Chodzi o ochronę architektury przed nieuzasadnioną kontynuacją.

Analiza techniczna

Kontynuować, zapytać użytkownika, zatrzymać się czy przekazać dalej

Na wysokim poziomie decyzja sterująca przyjmuje teraz postać:

- **continue**, jeśli następny ruch jest uzasadniony i ograniczony;
- **ask-user**, jeśli niejednoznaczność mająca znaczenie blokuje uzasadnioną kontynuację;
- **stop**, jeśli zadanie jest ukończone albo dalsze działanie nie jest uzasadnione;
- **handoff**, jeśli system nie powinien działać dalej samodzielnie.

Ten wybór jest zależny od zadania. Krótkie eksploracyjne zadanie wyszukiwawcze może łatwiej tolerować skąpy materiał dowodowy. Asystent do przeglądu literatury, który obiecuje syntezę opartą na materiale dowodowym, powinien być bardziej rygorystyczny.

Progi wystarczalności

Próg wystarczalności nie oznacza „pełnego pokrycia”. To zależy od zadania punkt adekwatności.

Dla przykładu przewodniego rozsądny próg wystarczalności może wymagać:

- reprezentatywnego zestawu 8-12 artykułów, jeśli są dostępne, albo mniejszej liczby z jawną uwagą o niepewności, jeśli pole jest małe;
- pokrycia co najmniej trzech wzorców architektonicznych, jeśli literatura wspiera aż tyle;
- ukończonej macierzy porównawczej dla wszystkich uwzględnionych artykułów;
- mapowania wsparcia dla głównych twierdzeń syntezy;
- jawnej notatki o nierozstrzygniętych rozbieżnościach albo słabych punktach materiału.

Próg powinien być na tyle wysoki, by chronić jakość zadania, i na tyle niski, by umożliwiać ograniczone ukończenie.

Progi niepewności

System nie powinien działać w nieskończoność przy nierozwiązanej niepewności. Typowe progi niepewności w przykładzie przewodnim obejmują:

- zakres pozostaje niejednoznaczny mimo dostępnego rozumowania wewnętrznego,
- dostępny materiał dowodowy jest zbyt skąpy, by wesprzeć obiecany rezultat,
- pobrane notatki nie zgadzają się ze sobą w sposób, którego asystent nie potrafi rozstrzygnąć dostępnymi narzędziami,
- albo pozostała niepewność istotnie zmieniłaby sposób ujęcia wyniku.

Gdy progi te zostają przekroczone, doprecyzowanie albo przekazanie stają się lepsze niż dalsza generacja.

Doprecyzowanie jako ponowne włączenie użytkownika do procesu

Doprecyzowanie jest najlepsze wtedy, gdy niejednoznaczność jest:

- konsekwencjalna,
- rozstrzygalna przez użytkownika,
- i prawdopodobnie zmniejszająca marnowaną pracę.

Dla asystenta do przeglądu literatury dobry przypadek doprecyzowania wygląda tak:

Czy przegląd ma uwzględniać asystentów tutoringowych, którzy pobierają materiał dowodowy na użytek wewnętrzny, czy tylko systemy, które pokazują cytowania albo cytaty bezpośrednio w końcowych odpowiedziach?

To pytanie zmienia to, co liczy się jako istotne. Dlatego lepiej zapytać, niż działać dalej pod ukrytym założeniem.

Warunki przekazania

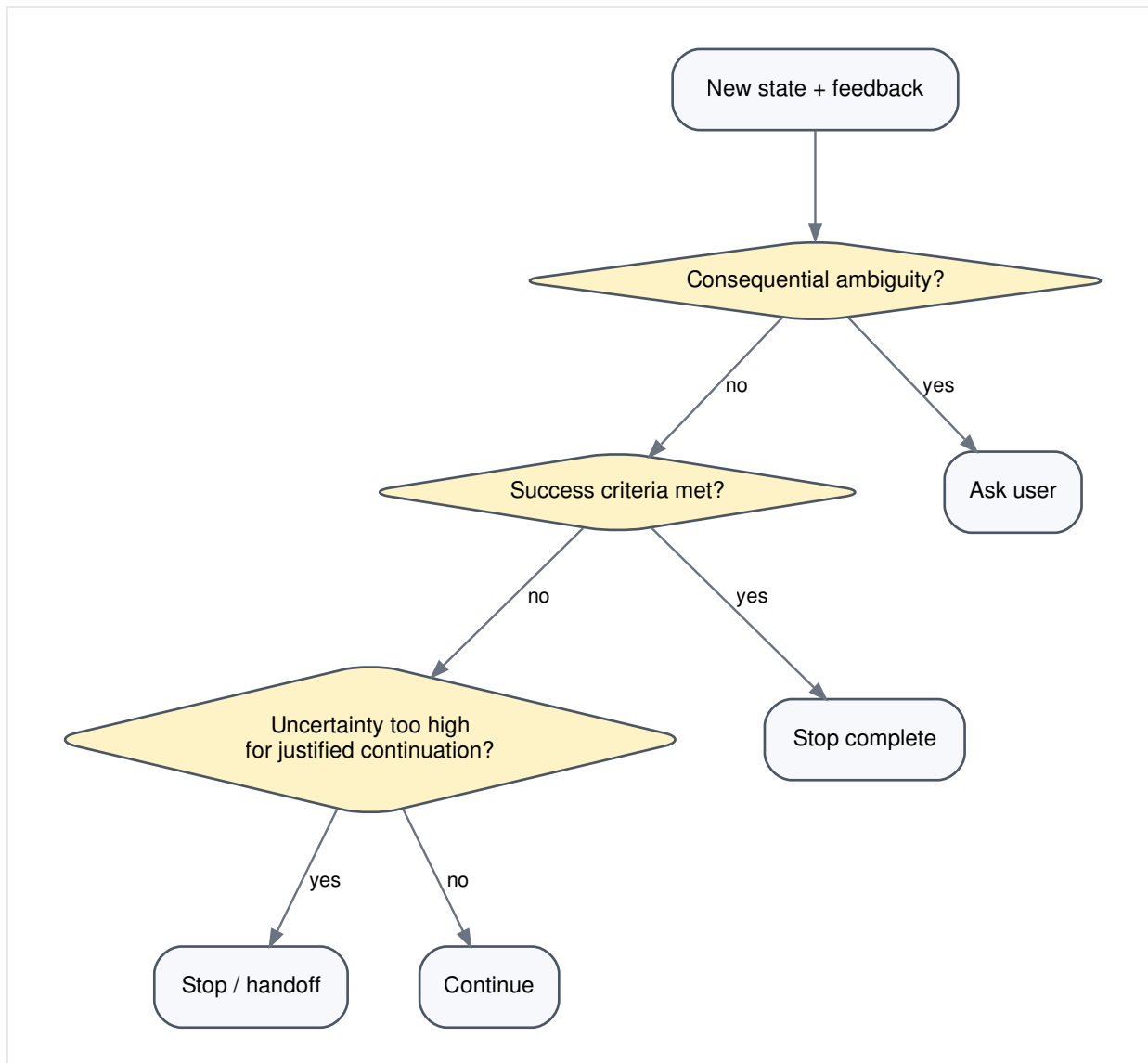
Przekazanie jest właściwe wtedy, gdy:

- system nie ma uprawnień, by kontynuować,
- materiał dowodowy jest zbyt słaby dla obiecanej siły twierdzeń,
- konflikt między źródłami wymaga interpretacji dziedzinowej wykraczającej poza zakres architektury,
- albo działanie przeniosłoby system z odwracalnej pracy informacyjnej do domeny bardziej konsekwencjalnej.

Dla przykładu przewodniego jasny przypadek przekazania wygląda tak: asystent znajduje sprzeczne twierdzenia o wpływie jawnych cytowań na zaufanie edukacyjne, a użytkownik oczekuje zaleceń politycznych albo pedagogicznych. Asystent może podsumować konflikt, ale to ekspert-człowiek powinien zinterpretować jego konsekwencje.

Tabela polityki zatrzymania dla przykładu przewodniego

Scenariusz	Zalecana decyzja	Dlaczego
Przeczytano dość reprezentatywnych artykułów, macierz jest kompletna, główne twierdzenia są wspierane	zatrzymaj się z końcowym przeglądem	próg wystarczalności został spełniony
Materiał dowodowy jest słaby po ukierunkowanym wyszukiwaniu, a zakres jest jasny	zatrzymaj się z ograniczonym przeglądem i jawną niepewnością albo przekaż sprawę dalej, jeśli obiecane wyniki nie da się spełnić	dalsze działanie mogłoby tylko sztucznie pompować pewność
„Dla studentów” pozostaje niejednoznaczne i zmienia kryteria włączania	zapytaj użytkownika	niejednoznaczność blokuje uzasadnioną kontynuację
Źródła są sprzeczne w kwestii interpretacyjnej rekomendacji wykraczającej poza władzę systemu	przekaż sprawę dalej	eskalacja jest bezpieczniejsza niż wymuszone rozstrzygnięcie
Dalsze wyszukiwanie może dać niewielką wartość marginalną, ale kryteria sukcesu są już spełnione	zatrzymaj się	ograniczona autonomia wymaga powściągliwości



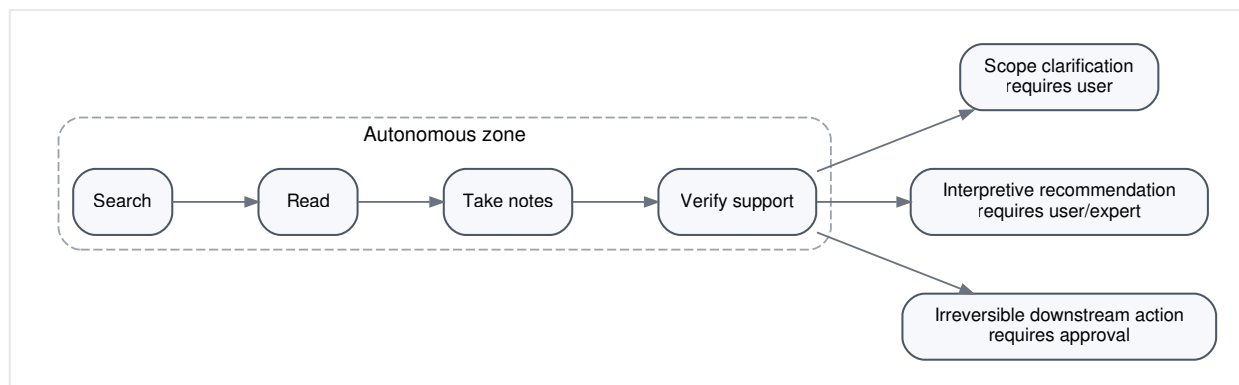
Rysunek 8.1. Proste drzewo decyzji o zatrzymaniu. Dobre architektury nie tylko wiedzą, jak kontynuować; wiedzą też, kiedy kontynuacja przestaje być uzasadniona.

Ograniczona autonomia

Ograniczona autonomia jest decyzją projektową o tym, co system może robić bez potwierdzenia użytkownika.

Dla przykładu przewodniego rozsądna granica autonomii wygląda tak:

- dozwolone bez potwierdzenia: wyszukiwanie, czytanie, sporządzanie notatek, doprecyzowanie planu, szkicowanie przeglądu, uruchamianie kontroli weryfikacyjnych;
- wymagające doprecyzowania: decyzje o zakresie, które są istotnie niejednoznaczne;
- wymagające przekazania albo zgody: twierdzenia wykraczające poza dostępny materiał dowodowy, rekomendacje wykraczające poza syntezę albo nieodwracalne działania dalszego ciągu.



Rysunek 8.2. Granica autonomii dla asystenta do przeglądu literatury prowadzącego czytelnika przez całą książkę. Celem tej granicy nie jest uczynienie systemu biernym, lecz pozwolenie mu na pewne działanie w ramach jawnych ograniczeń.

Badania HCI nad systemami człowiek-AI wielokrotnie podkreślają zarządzanie oczekiwaniami, odzyskiwalność i kontrolę użytkownika (Amershi et al., 2019). Ten rozdział przekłada tę lekcję na architekturę: ponowne wejście użytkownika jest częścią sterowania, a nie wstydliwym planem awaryjnym.

Przykład krok po kroku: zapisywanie polityki zatrzymania i przekazania

Założmy, że asystent do przeglądu literatury przeczytał sześć mocnych artykułów i dwa niepewne. Ma dość materiału, by wskazać trzy wzorce architektoniczne i wypełnić większość macierzy porównawczej. Jednak jeden z wymaganych wątków — wpływ na zaufanie studentów — jest wspierany tylko słabo i przez sprzeczne badania.

Uzasadniona polityka mogłaby mówić:

- zatrzymaj główny przegląd, jeśli twierdzenia o architekturze i groundingu są wystarczająco wspierane;
- jawnie oznacz sekcję o zaufaniu studentów jako opartą na materiale mieszanym albo skąpym;
- przekaz sprawę dalej, jeśli użytkownik chce praktycznych rekomendacji dotyczących wdrożenia w klasie;
- poproś o doprecyzowanie, jeśli użytkownik wolałby rozszerzyć zakres tak, by objąć szerszą klasę asystentów edukacyjnych.

Taka polityka szanuje ograniczoną autonomię. Asystent nie udaje, że każdą nierozwiązaną kwestię można usunąć większą ilością wewnętrznej pracy.

Przykład w pseudokodzie

```

procedure DECIDE_CONTROL_AFTER_FEEDBACK(G, S_t, feedback):
  if consequential_ambiguity(S_t):
    return ask_user
  if success_criteria_met(G, S_t):
    return stop_complete
  if evidence_too_thin_for_promised_output(G, S_t):
    if user_can_reframe_task(G):
      return ask_user
    else:
      return handoff_or_stop_with_limits
  if conflict_exceeds_authority(S_t):
    return handoff
  return continue

```

To pierwsze miejsce w książce, w którym „władza decyzyjna” wchodzi wprost do gry. Nie dlatego, że asystent jest aktorem prawnym albo organizacyjnym, ale dlatego, że projekt ograniczony zawsze zawiera pojęcie tego, co systemowi wolno rozstrzygać samodzielnie.

Aktualizacja wspólnego artefaktu

Asystent do przeglądu literatury zyskuje teraz politykę zatrzymania i przekazania.

Artefakt	Bieżąca zawartość
Polityka zatrzymania	próg wystarczalności, próg niepewności oraz decyzje zależne od scenariusza
Granica autonomii	co asystent może dalej robić samodzielnie, a co wymaga ponownego wejścia użytkownika albo eksperta

Ten artefakt zamyka główną pętlę. Następny rozdział użyje wszystkich zgromadzonych artefaktów do porównywania całych architektur według profilu zadania.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Jeśli brakuje logiki zatrzymania, system albo zapętla się bez końca, albo zatrzymuje się arbitralnie. Jeśli brakuje doprecyzowania, system ukrywa założenia zamiast je ujawniać. Jeśli brakuje przekazania, system może działać poza zakresem swojej uzasadnionej kompetencji.

Źle użyta logika zatrzymania powoduje przeciwstawne problemy:

1. **przedwczesne zatrzymanie**, w którym łatwe do spełnienia częściowe kryteria podszywają się pod ukończenie;
2. **nigdy dość**, w którym system nadal szuka, bo doskonała pewność jest niemożliwa;
3. **niechęć do doprecyzowania**, w której system traktuje każdą niejednoznaczność jako coś, co trzeba rozwiązać wewnątrz;
4. **fałszywa autonomia**, w której system wykonuje działania wymagające zgody albo eskalacji.

Tryby awarii, ograniczenia i błędne przekonania

- **Nieskończone pętle koncepcyjne.** Bez reguły zatrzymania można wyobrażać sobie poprawę bez końca.
- **Przedwczesne zatrzymanie.** Spójny akapit nie jest tym samym co ukończenie zadania.
- **Traktowanie doprecyzowania jako porażki.** Doprecyzowanie jest uzasadnionym działaniem sterującym.
- **Traktowanie zatrzymania i przekazania jako szczegółu operacyjnego.** To część integralności pojęciowej architektury (Parasuraman et al., 2000; Amershi et al., 2019).

Podsumowanie rozdziału

Silna architektura nie tylko dobrze kontynuuje pracę. Potrafi też dobrze ją zakończyć. Ukończenie, zatrzymanie, doprecyzowanie i przekazanie to różne wyniki sterowania i każdy z nich wymaga jawnych kryteriów. Ograniczona autonomia jest dyscypliną projektową, która utrzymuje te wyniki w spójności. Następny rozdział cofa się o krok i używa wszystkiego, co zostało dotąd zbudowane, by porównywać całe architektury według profilu zadania i zakresu.

Źródła i dalsza lektura

- (Amershi et al., 2019) w kontekście kontroli użytkownika, ustawiania oczekiwań i odzyskiwalności w interakcji człowiek-AI.
- (Parasuraman et al., 2000) w kontekście rozumowania o poziomach automatyzacji i o tym, kiedy nie warto automatyzować dalej.
- (Xi et al., 2023) jako szeroki przegląd wzorców projektowania agentów, w których autonomia i przekazanie są traktowane odmiennie.

Co ten rozdział dodaje do architektury

- **Dodano:** kryteria zatrzymania, reguły doprecyzowania, warunki przekazania i granicę autonomii.
- **Zaadresowany problem:** nieskończona kontynuacja albo arbitralne zatrzymanie.
- **Nowe ryzyko:** progi, które są albo zbyt surowe, by ukończyć pracę, albo zbyt luźne, by chronić przed nieuzasadnionym działaniem.

Część III — Ocena architektur i synteza

Przekształca wiedzę o komponentach w porównywanie, dopasowanie zadanie-architektura, granice zakresu i obroniony projekt końcowy.

Rozdział 9 — Porównywanie architektur według profilu zadania i zakresu

AA-S09 — Porównywanie architektur i granice zastosowania

Jakie wymagania ujawnia przykład przewodni

Do tej pory asystent do przeglądu literatury zgromadził już sporą architekturę: specyfikację zadania, jawny stan, politykę pamięci, kontrakty narzędzi, plan możliwy do rewizji, kontrole informacji zwrotnej oraz reguły zatrzymania i przekazania. To jednak nie oznacza, że każde ograniczone zadanie informacyjne powinno otrzymać tę samą architekturę.

Jeśli użytkownik potrzebuje tylko trzech najnowszych artykułów przeglądowych i po jednym zdaniu o każdym z nich, ciężki system plan-memory-act-reflect-stop może być po prostu nadmiernie rozbudowany. Jeśli użytkownik potrzebuje starannego przeglądu literatury z perspektywy architektury i z jawną obsługą niepewności, odpowiedź prompt-only jest prawdopodobnie zbyt uboga.

Wymaganie projektowe w tym rozdziale dotyczy więc osądu: **jak porównywać architektury według profilu zadania, a nie według modnych haseł?**

Plan rozdziału

Ten rozdział wprowadza soczewkę profilu zadania z sześcioma głównymi wymiarami:

- horizon,
- uncertainty,
- memory need,
- tool dependence,
- feedback richness,
- autonomy bound.

Używa tej soczewki do porównania trzech często spotykanych architektur dla tego samego przykładu przewodniego — interakcji prompt-only, skryptowego przepływu pracy opartego na retrieval oraz ograniczonej pętli plan-memory-act-reflect-stop — a następnie zestawia je z zadaniem o krótszym horyzoncie, w którym pełniejszy agent jest niepotrzebny.

Kluczowe pojęcia i definicje

Profil zadania to strukturalny profil zadania: jak długie jest to zadanie, jak duża jest niepewność, jak bardzo zależy od narzędzi, jakiej pamięci wymaga, jak bogate są sygnały informacji zwrotnej i jaką autonomię uzasadnia.

Wzorzec architektoniczny to często spotykany układ sterowania, taki jak interakcja prompt-only albo ograniczona pętla adaptacyjna.

Dopasowanie zadanie-architektura to stopień, w jakim dany wzorzec pasuje do strukturalnych wymagań zadania.

Granica zakresu wskazuje, gdzie temat tej książki ustępuje miejsca obszarom sąsiednim, takim jak reinforcement learning, wewnątrz information retrieval, planowanie symboliczne, wzorce projektowe oprogramowania, frameworki albo operations.

Najłatwiej pomylić: wzorzec architektoniczny z frameworkiem implementacyjnym

Wzorzec architektoniczny jest pojęciową organizacją celów, pętli, stanu, pamięci, działania, informacji zwrotnej i zatrzymania. Framework jest nośnikiem programistycznym, który może implementować wiele wzorców. Ten rozdział porównuje wzorce, a nie biblioteki czy produkty.

Najpierw intuicja

Złożoność architektoniczna nie jest cnotą. To koszt, który należy ponosić tylko wtedy, gdy uzasadnia go profil zadania.

Przewodni asystent do przeglądu literatury jest użyteczny właśnie dlatego, że leży pośrodku. Nie jest tak prosty jak odpowiedź jednorazowa. Nie jest też tak otwarty jak stale działający asystent z szeroką autonomią. Wymaga tyle struktury, by ujawnić kompromisy, ale nie dryfuje jeszcze w stronę operations produkcyjnych ani teorii planowania formalnego.

Dobry analityk pyta więc nie „która architektura jest najbardziej zaawansowana?”, lecz „która architektura jest adekwatna, uzasadniona i nie bardziej rozbudowana, niż to konieczne?”.

Analiza techniczna

Sześć wymiarów profilu zadania

Praktyczna soczewka porównawcza jest następująca:

1. **Horyzont.** Ile zależnych od siebie kroków trzeba wykonać?
2. **Niepewność.** Jak prawdopodobne jest to, że obserwacje zmieniają dalszą ścieżkę?
3. **Potrzeba pamięci.** Czy system musi zachować ustrukturyzowane informacje poza bezpośrednim wywołaniem?
4. **Zależność od narzędzi.** Czy dobre wykonanie wymaga świeżej interakcji ze światem?
5. **Bogactwo informacji zwrotnej.** Czy istnieją znaczące sygnały, które mogą skorygować przebieg działania?
6. **Granica autonomii.** Jak duża samodzielna kontynuacja jest uzasadniona?

Te wymiary nie produkują oceny liczbowej. Porządkują osąd.

Kanoniczne ograniczone wzorce architektoniczne

Powracające wzorce w tej książce można podsumować następująco.

Wzorzec	Krótki opis	Najlepsze dopasowanie	Typowy problem
Interaktywny asystent prompt-only	użytkownik prowadzi pętlę, a model odpowiada		wczesne pozory kompletności, słaba

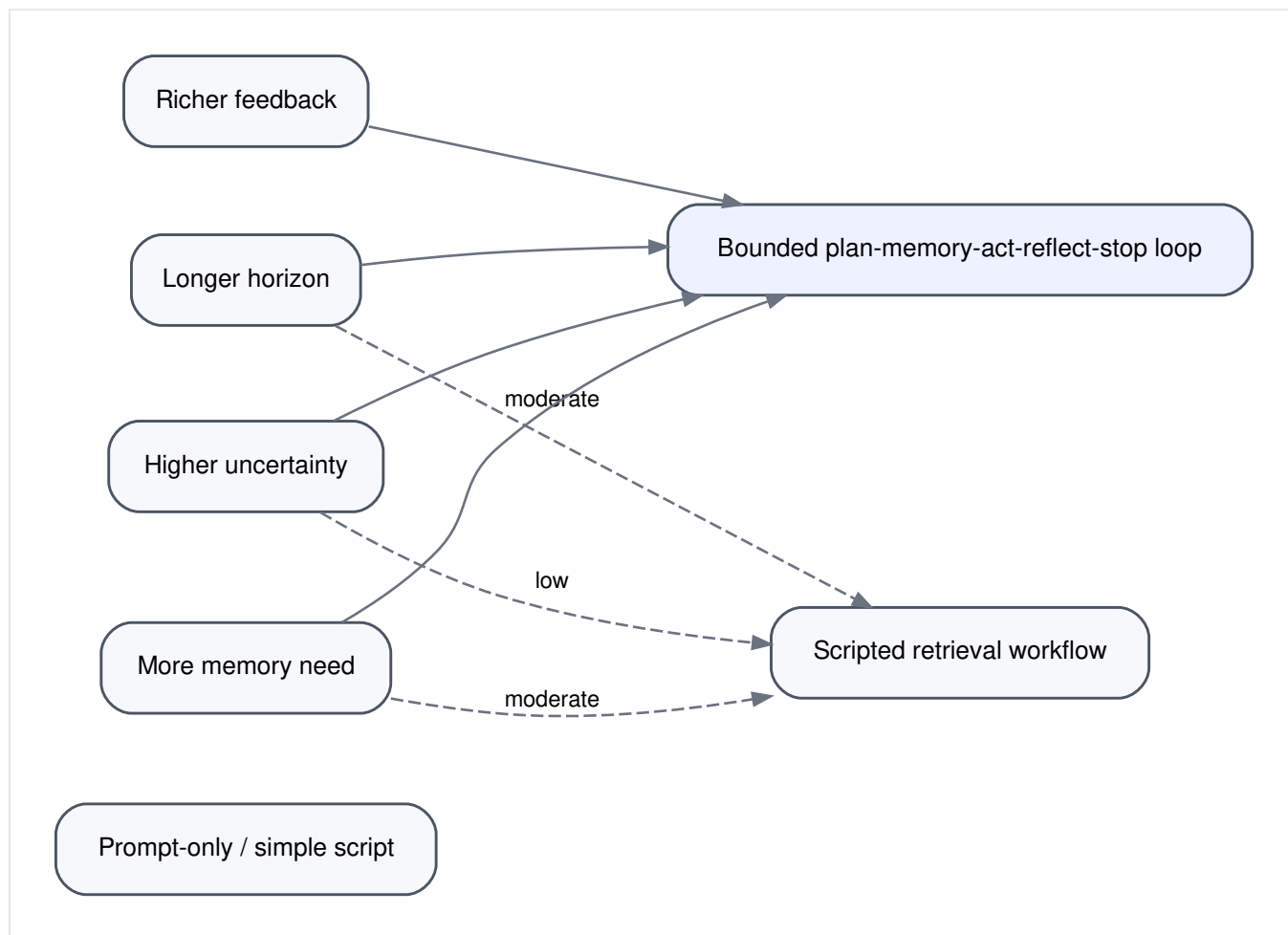
Wzorzec	Krótki opis	Najlepsze dopasowanie	Typowy problem
	na podstawie bieżącego kontekstu	krótkie, dobrze ograniczone zadania o małej zależności	obsługa materiału dowodowego
Skryptowy przepływ pracy oparty na retrieval	ustalona sekwencja typu szukaj → czytaj → streszczaj	stabilne zadania o przewidywalnej strukturze	kruchosc, gdy materiał dowodowy zaskakuje system
Adaptacyjna pętla korzystająca z narzędzi	jawny stan plus wybór działania w czasie wykonania	umiarkowana niepewność i zależność od narzędzi	słabość bez dobrych reguł zatrzymania albo informacji zwrotnej
Ograniczona pętla plan-memory-act-reflect-stop	jawna specyfikacja zadania, stan, polityka pamięci, planowanie, weryfikacja, zatrzymanie lub przekazanie	zadania o średnim horyzoncie, z wymaganiami dowodowymi i ograniczoną autonomią	przerost formy dla krótkich zadań, złożoność polityk

Porównanie na przykładzie przewodnim

Ograniczone zadanie z przeglądem literatury ma średni horyzont, umiarkowaną niepewność, zależność od narzędzi, umiarkowane potrzeby pamięciowe, bogate potencjalne sygnały informacji zwrotnej i jawnie ograniczoną autonomię. Już ten profil sugeruje, że architektura najbardziej minimalna będzie zbyt słaba, a najbardziej sztywna ustalona sekwencja okaże się zbyt krucha.

Tabela 9.1 porównuje trzy wymagane wersje przykładu przewodniego.

Architektura	Dopasowanie do horyzontu	Obsługa niepewności	Dopasowanie do pamięci	Użycie narzędzi	Informacja zwrotna i zatrzymanie	Ogólne dopasowanie do przykładu przewodniego
Interakcja prompt-only	słabe	słabe	słabe	niewielkie albo improwizowane	w dużej mierze sterowane przez użytkownika	zbyt uboga
Skryptowy przepływ pracy oparty na retrieval	umiarkowane	słabe do umiarkowanego	umiarkowane, jeśli pola są przekazywane dalej	silne dla przewidywalnych kroków	ograniczone, jeśli nie zostaną dodane jawnie	użyteczny dla węższych wariantów
Ograniczona pętla plan-memory-act-reflect-stop	silne	silne	silne, jeśli polityki pozostają minimalne	silne i selektywne	silne	najlepsze dopasowanie



Rysunek 9.1. Kształt zadania wyznacza dopasowanie architektury. Pełniejsza ograniczona pętla pasuje tam, gdzie horyzont, niepewność, potrzeba pamięci i bogactwo informacji zwrotnej są umiarkowane, a nie trywialne.

Ważne nie jest to, że trzeci wzorzec jest „bardziej agentowy”, a więc lepszy pod każdym względem. Ważne jest to, że jego dodatkowa struktura odpowiada wprost na profil zadania z przykładu przewodniego.

Zadanie badawcze o krótszym horyzoncie

Porównajmy teraz inne zadanie:

Znajdź trzy niedawne artykuły przeglądowe o retrieval-augmented generation i podaj po jednym zdaniu o tym, czego dotyczy każdy z nich.

To zadanie ma krótszy horyzont, niższą niepewność, mniejszą potrzebę pamięci i zwykle mniejsze ryzyko autonomii. Lepszym dopasowaniem może być skryptowy przepływ pracy oparty na retrieval, a nawet prompt-only z jednym krokiem retrieval. Dodawanie rozbudowanej trwałej pamięci, ponownego planowania i logiki przekazania może być tu zbędnym narzutem.

Ten przykład ma znaczenie, bo chroni przed częstym błędem początkujących: kiedy raz pozna się bogatą architekturę, kusi, by używać jej wszędzie.

Dlaczego prostsze architektury bywają czasem lepsze

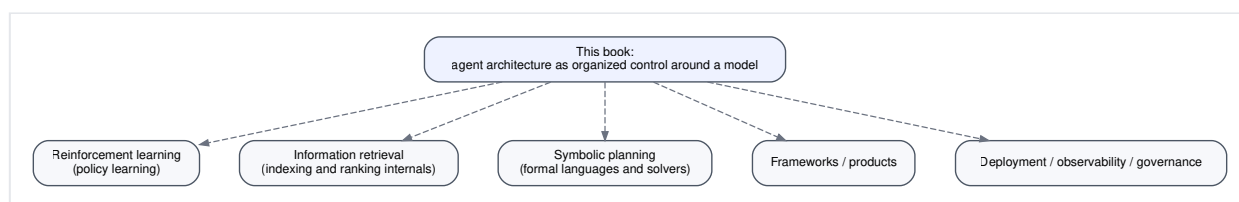
Prostsze architektury bywają lepsze, ponieważ są:

- łatwiejsze do zrozumienia,
- mniej podatne na interakcje między politykami,
- tańsze w uruchamianiu,
- łatwiejsze do ograniczenia,
- i często wystarczające dla małych zadań.

Nie jest to sceptycyzm antyagentowy. To dyscyplina projektowa zależna od rodzaju zadania.

Granice zakresu

Ta książka celowo zatrzymuje się przed kilkoma obszarami sąsiednimi.



Rysunek 9.2. Granice zakresu. Ta książka koncentruje się na organizacji architektury wokół modelu generatywnego, a nie na uczeniu polityk, wewnątrz retrieval, formalnych planerach, frameworkach czy operations produkcyjnych.

- **Reinforcement learning** pyta o to, jak polityki mogą być uczone z nagrody w czasie (Sutton and Barto, 2018). Ta książka pyta zamiast tego, jak projektować ograniczone sterowanie wtedy, gdy polityka jest w dużej mierze określona, zapromptowana albo ręcznie zaprojektowana.
- **Information retrieval** pyta o to, jak indeksować, rangować i oceniać systemy retrieval (Manning et al., 2008). Ta książka traktuje wyszukiwanie jako pojęciowy interfejs narzędziowy i nie wchodzi do środka algorytmów rangowania.
- **Planowanie symboliczne** bada formalne reprezentacje, operatory i procedury przeszukiwania (Russell and Norvig, 2021). Ta książka zapożycza intuicję planowania, ale nie uczy formalnych języków planowania ani solverów.
- **Wzorce programistyczne, frameworki i produkty orkiestracyjne** dotyczą nośników implementacyjnych. Ta książka pozostaje na poziomie wzorców architektonicznych.
- **Operations i inżynieria produkcyjna** dotyczą wdrożenia, obserwowalności, ładu i analizy zawieżeń w działających systemach. To ważne obszary, ale wykraczają poza obecny zakres pojęciowy.

Wyznaczenie tych granic jest dydaktycznie użyteczne. Czytelnik nie musi opanować wszystkich pól sąsiednich, zanim zrozumie, jak składa się architektury agentowe.

Przykład krok po kroku: wybór wzorca według profilu zadania

Zastosujmy teraz te sześć wymiarów wprost.

Powracające zadanie przeglądu literatury

- Horyzont: średni
- Niepewność: umiarkowana; jakość materiału dowodowego zmienia dalszą ścieżkę
- Potrzeba pamięci: umiarkowana; liczą się notatki i stan macierzy
- Zależność od narzędzi: wysoka; retrieval i lektura są konieczne
- Bogactwo informacji zwrotnej: wysokie; znaczenie mają sprawdzenia wsparcia i sprzeczności
- Granica autonomii: ograniczona, ale nietrywialna

Rekomendowany wzorzec: ograniczona pętla plan-memory-act-reflect-stop.

Krótkie zadanie wyszukiwania artykułów przeglądowych

- Horyzont: krótki
- Niepewność: niska do umiarkowanej
- Potrzeba pamięci: niska
- Zależność od narzędzi: umiarkowana
- Bogactwo informacji zwrotnej: ograniczone
- Granica autonomii: niska

Rekomendowany wzorzec: prompt-only z retrieval albo skryptowy przepływ pracy oparty na retrieval.

Właśnie taki sposób wyjaśniania książka próbuje uczynić drugą naturą.

Przykład w pseudokodzie

```
procedure SELECT_ARCHITECTURE(task_shape):
  if task_shape.horizon is short
    and task_shape.uncertainty is low
    and task_shape.memory_need is low:
    return prompt_or_scripted

  if task_shape.tool_dependence is high
    and task_shape.feedback_richness is moderate_or_high
    and task_shape.autonomy_bound is bounded:
    return bounded_adaptive_loop

  if task_shape.requires_structured_evidence_accumulation:
    return bounded_plan_memory_act_reflect_stop

  return simplest_architecture_that_meets_constraints
```

Ostatnia linia jest najważniejsza. Prostota nie jest planem awaryjnym. Często jest poprawną odpowiedzią.

Aktualizacja wspólnego artefaktu

Asystent do przeglądu literatury zyskuje teraz swoją macierz porównania architektur.

Artefakt	Bieżąca zawartość
Macierz porównawcza	prompt-only vs skryptowy przepływ pracy vs ograniczona pętla plan-memory-act-reflect-stop
Soczewka wyboru	horyzont, niepewność, potrzeba pamięci, zależność od narzędzi, bogactwo informacji zwrotnej, granica autonomii

Ten rozdział zamienia znajomość komponentów w osąd projektowy.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

Bez dyscypliny porównawczej wybór architektury dryfuje w stronę mody. Projektanci rozbudowują system ponad potrzebę, bo bogatsze diagramy wyglądają imponująco, albo budują zbyt mało, bo silny model wydaje się wystarczający.

Niewłaściwe użycie przybiera kilka form:

1. **wyбір według modnego hasła:** wybieranie architektury z najlepszą nazwą, a nie z najlepszym dopasowaniem,
2. **maksymalizm komponentów:** dokładanie każdego możliwego modułu „na wszelki wypadek”,
3. **mylenie pól sąsiednich:** traktowanie RL, planowania formalnego albo wnętrza IR jako warunków wstępnych, zamiast specjalizacji sąsiednich,
4. **dryf w stronę frameworków:** mówienie o produktach programistycznych zamiast o strukturze sterowania.

Tryby awarii, ograniczenia i błędne przekonania

- **Projekt typu one-size-fits-all.** Nie istnieje jedna architektura dominująca we wszystkich kształtach zadań.
- **Wyбір według modnego hasła.** Terminy takie jak „agent”, „planner” albo „memory” nie uzasadniają się same.
- **Traktowanie pól sąsiednich jak warunków wstępnych.** To sąsiedzi, a nie bramki wejściowe (Sutton and Barto, 2018; Manning et al., 2008; Russell and Norvig, 2021).
- **Dryf w stronę frameworków albo ops.** Szczegóły implementacyjne stają się ważne później, ale nie są tutaj głównym przedmiotem porównania architektonicznego.

Podsumowanie rozdziału

Osąd architektoniczny bierze się z dopasowania struktury sterowania do kształtu zadania. Najbogatsza ograniczona pętla jest odpowiednia dla asystenta do przeglądu literatury prowadzącego nas przez całą książkę, ponieważ zadanie ma umiarkowany horyzont, zależność od narzędzi, potrzebę akumulacji materiału dowodowego i ograniczoną autonomię. Prostsze architektury pozostają lepszym dopasowaniem dla krótszych i stabilniejszych zadań. Rozdział wyznaczył też granice tematu wobec RL, IR, planowania symbolicznego, frameworków i operations.

Ostatni rozdział wykorzystuje teraz zgromadzone artefakty, aby od zera zsyntetyzować i obronić projekt asystenta do przeglądu literatury.

Źródła i dalsza lektura

- (Xi et al., 2023) jako szeroki przegląd wzorców agentowych opartych na LLM.
- (Russell and Norvig, 2021) jako sąsiednie tło dla przeszukiwania i planowania.
- (Sutton and Barto, 2018) o uczeniu polityk jako innej rodzinie problemów.
- (Manning et al., 2008) o wnętrzu retrieval kryjącym się pod narzędziami wyszukiwania.

Co ten rozdział dodaje do architektury

- **Dodano:** soczewkę kształtu zadania do porównywania całych architektur.
- **Zaadresowany problem:** wybieranie architektur według mody albo pojedynczych komponentów.
- **Nowe ryzyko:** fałszywa precyzja, jeśli macierz porównawcza zostanie potraktowana jak formuła, a nie pomoc do osądu.

Rozdział 10 — Projektowanie asystenta do przeglądu literatury

AA-S10 — Synteza końcowa

Jakie wymagania ujawnia przykład przewodni

Powracający w książce asystent do przeglądu literatury był rozbudowywany rozdział po rozdziale. Ostateczna presja dotyczy syntezy: czy potrafimy teraz złożyć spójną ograniczoną architekturę i obronić każde włączenie, każde pominięcie oraz każdy kompromis?

Słaby projekt końcowy po prostu skleiłby wszystkie komponenty wspomniane w książce. Mocniejszy projekt końcowy zadaje trudniejsze pytania:

- Które komponenty są niezbędne dla tego konkretnego zadania?
- Które są opcjonalne, ale użyteczne?
- Które byłyby przerostem formy?
- Czy potrafimy prześledzić jedno wykonanie od prośby użytkownika do zatrzymania albo przekazania i pokazać, że architektura rzeczywiście trzyma się razem?

Ten rozdział odpowiada na te pytania, tworząc dwa projekty: **minimalnie wystarczającą architekturę** i **bogatszą, obronioną architekturę**.

Plan rozdziału

Synteza końcowa ma pięć zadań:

1. ponownie sformułować ograniczoną specyfikację zadania,
2. zaproponować minimalnie wystarczającą architekturę,
3. zaproponować bogatszą architekturę i uzasadnić dodane elementy,
4. przejść przez pełny ślad wykonania,
5. przewidzieć potencjalny problem i zrewidować projekt.

Rezultatem nie jest tutorial do frameworka. To notatka projektowa dla ograniczonej architektury agentowej.

Najłatwiej pomylić: minimalnie wystarczającą architekturę z architekturą nadmiernie rozbudowaną

Minimalnie wystarczająca architektura zawiera najmniejszy zestaw komponentów wymaganych przez specyfikację zadania. Architektura nadmiernie rozbudowana zawiera dodatkowe mechanizmy, których koszt sterowania, ciężar utrzymania albo powierzchnia awarii przewyższają ich wartość dla ograniczonego zadania.

Notatka projektowa: specyfikacja zadania

Zaczynamy od złożonej specyfikacji zadania, wypracowanej we wcześniejszych rozdziałach.

`G_review = <objective = produce a 1,200-word literature review on citation-grounded question-answering assistants for students; constraints = 2021–2024, 8–12 papers if available, architecture-first framing, comparison matrix required, no vendor-specific tutorials; success criteria = each substantive claim supported by at least one read source, at least three architecture patterns compared if evidence allows, uncertainty explicitly marked where evidence is mixed or thin; stop/handoff conditions = ask if scope is materially ambiguous, stop when criteria are met with adequate evidence, hand off if evidence is too thin or interpretive conflict exceeds system authority>.`

Zadanie jest ograniczone, zorientowane na informację i umiarkowanie niepewne. Nie jest otwartym autonomicznym badaniem. Nie jest wdrożeniem produkcyjnym. Potrzebuje więc architektury selektywnej, a nie maksymalnie rozbudowanego układu.

Minimalnie wystarczająca architektura

Minimalnie wystarczająca wersja obejmuje najmniejszy zestaw komponentów, który może wiarygodnie spełnić `G_review`.

Uwzględnione komponenty

1. **Jawna specyfikacja zadania `G_review`.**
2. **Stan zewnętrzny lokalny dla wykonania `S_t`** z kandydackimi źródłami, zbiorem po preselekcji, notatkami, wierszami macierzy i znacznikami ukończenia.
3. **Pojęciowe interfejsy narzędziowe** do wyszukiwania, czytania dokumentów, zapisu notatek i sprawdzania cytowań.
4. **Prosty plan** jako lista kontrolna: szukaj → przesiej → czytaj → syntetyzuj → weryfikuj → zatrzymaj się.
5. **Sprawdzenia weryfikacyjne** dla wsparcia twierdzeń i ukończenia zadania.
6. **Logika zatrzymania i doprecyzowania** dla niejednoznaczności, skąpego materiału dowodowego i wystarczającego ukończenia.

Pominięte komponenty

1. **Trwała pamięć semantyczna** wykraczająca poza starannie opracowane szablony procedur.
2. **Bogata pamięć epizodyczna między sesjami.**
3. **Rozbudowane przeszukiwanie rozgałęzione.**
4. **Dekompozycja wieloagentowa.**
5. **Otwarte autonomiczne generowanie celów.**

Te pominięcia są celowe. Zadanie nie wymaga ich, aby dało się je dobrze zrozumieć albo wykonać na ograniczonym poziomie pojęciowym.

Dlaczego ta wersja jest minimalnie wystarczająca

Zadanie wymaga akumulacji materiału dowodowego i ograniczonej syntezy, dlatego interakcja prompt-only jest niewystarczająca. Wymaga też pewnej adaptacji, więc sztywny skrypt byłby ryzykowny. Jednocześnie zadanie nie wymaga długoterminowej personalizacji, autonomicznego odkrywania narzędzi ani optymalizacji polityki

wyuczonej. Dlatego minimalnie wystarczający projekt to **ograniczona pętla adaptacyjna z lekkim planowaniem i silną weryfikacją**.

Bogatsza, obroniona architektura

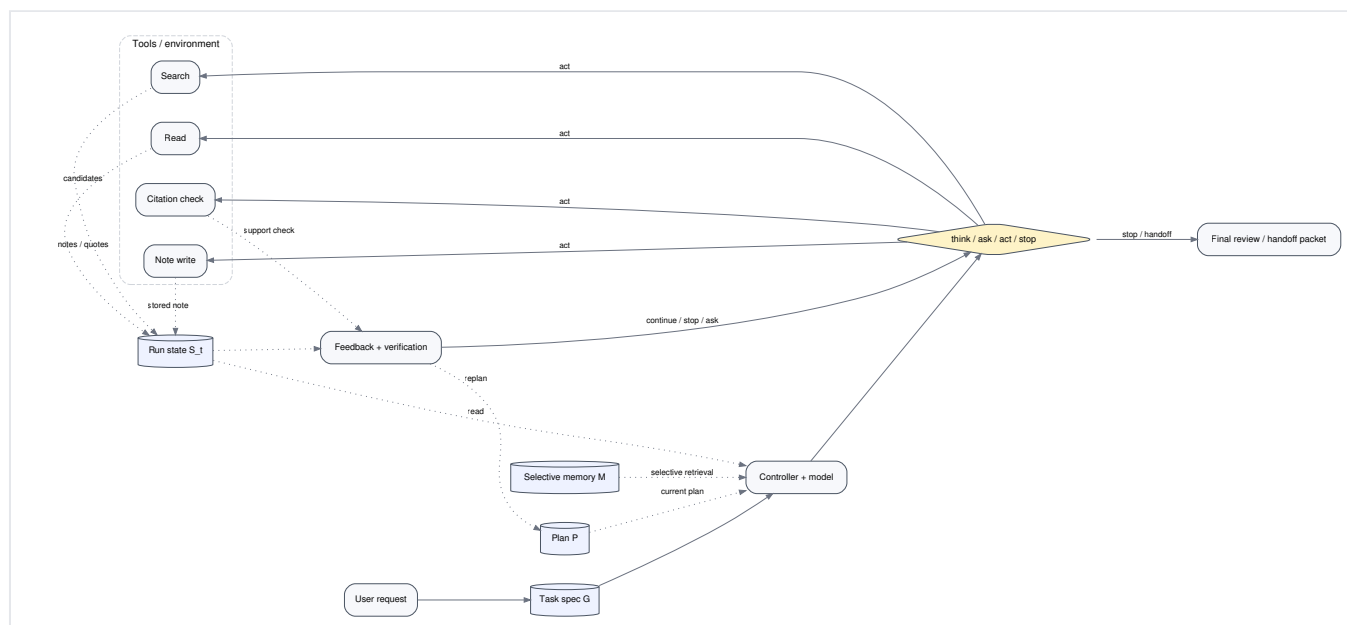
Bogatsza wersja dodaje tylko te elementy, które istotnie poprawiają odporność w przykładzie przewodnim.

Dodane komponenty

1. **Kuratorowana pamięć procedur.** Wielokrotnie używalny schemat notatek, lista kontrolna preselekcji i lista kontrolna zatrzymania.
2. **Ograniczona historia epizodyczna zapytań.** Niedawne próby zapytań i obserwowana jakość ich wyników w obrębie bieżącej rodziny tematów.
3. **Pętla ponownego planowania.** Uruchamiana wtedy, gdy materiał dowodowy okazuje się szerszy, skromniejszy albo bardziej sprzeczny, niż oczekiwano.
4. **Rejestr sprzeczności.** Wydzielona struktura do śledzenia miejsc, w których notatki ze źródeł nie zgadzają się ze sobą.
5. **Jawny pakiet przekazania.** Jeśli system zatrzyma się bez pełnego rozstrzygnięcia, przekazuje użytkownikowi pakiet zawierający założenia co do zakresu, zebrany materiał dowodowy, nierozwiązane rozbieżności i rekomendowane dalsze kroki.

Te dodatki są obronione dlatego, że odpowiadają bezpośrednio na problemy z wcześniejszych rozdziałów: powtarzane błędy w zapytaniach, nieaktualną albo brakującą strukturę procedur oraz słabą obsługę niezgodności.

Pełny diagram architektury

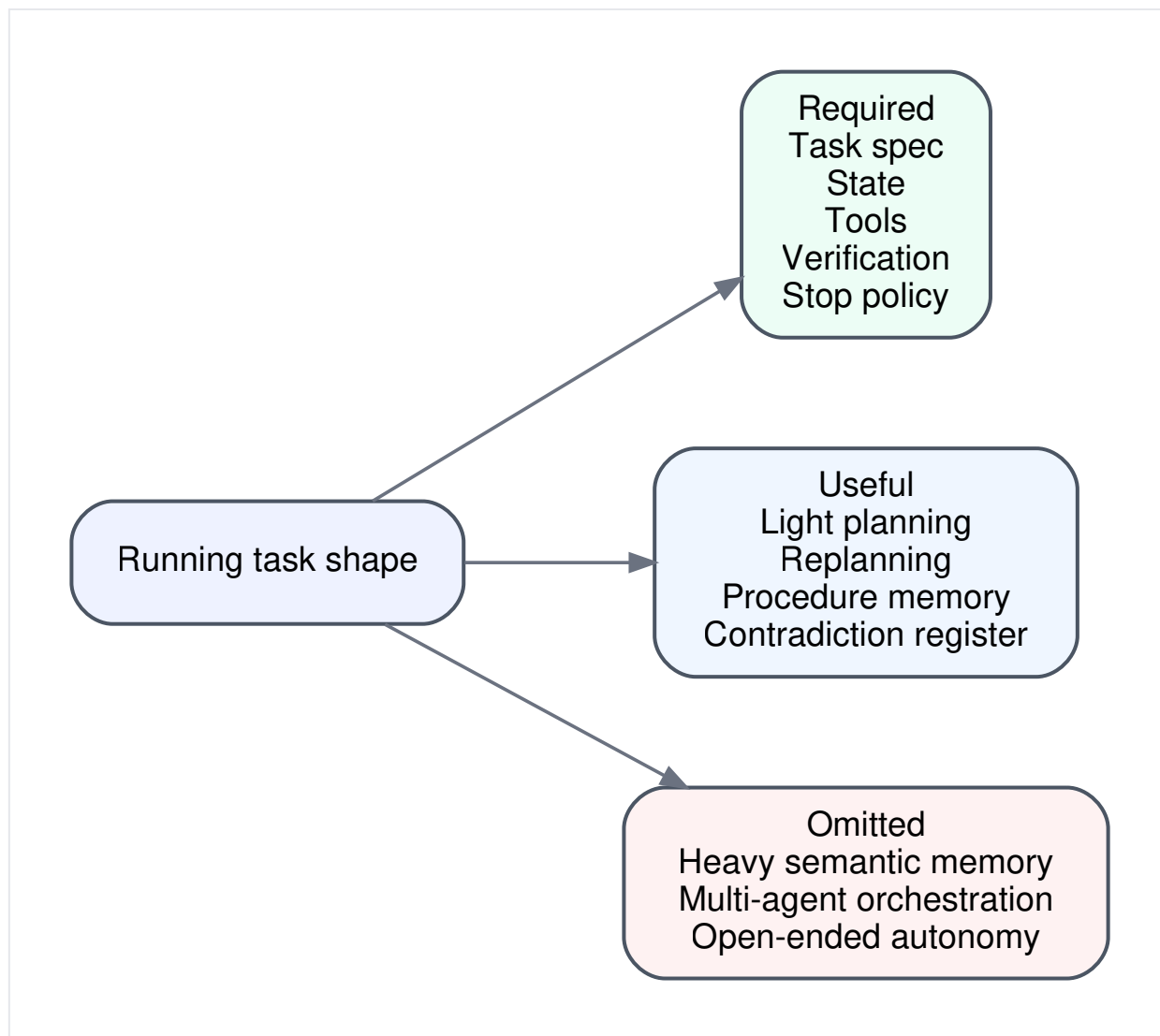


Rysunek 10.1. Pełna ograniczona architektura dla asystenta do przeglądu literatury. Model znajduje się wewnątrz pętli warunkowanej celem, z jawnym stanem, selektywną pamięcią, interfejsami narzędziowymi, planowaniem, weryfikacją oraz logiką zatrzymania lub przekazania.

Ten diagram można czytać od lewej do prawej i z powrotem:

- `G_review` ogranicza całe wykonanie.
- Kontroler utrzymuje `S_t`.
- Model pomaga wybierać między `think`, `ask`, `act` i `stop`.
- Narzędzia sprzęgają system ze środowiskiem.
- Weryfikacja i informacja zwrotna wracają do kontrolera.
- Logika zatrzymania albo przekazania kończy wykonanie w ustalonych granicach.

Uzasadnienie włączenia komponentów



Rysunek 10.2. Uzasadnienie włączenia. Komponenty są obecne dlatego, że wymaga ich kształt zadania, a nie dlatego, że nakazuje to jakiś ogólny „agent stack”.

Uzasadnienie można podsumować w Tabeli 10.1.

Komponent	Uwzględniony?	Dlaczego
Jawna specyfikacja zadania	tak	bez niej system nie potrafi ocenić sukcesu ani zatrzymania
Stan lokalny dla wykonania	tak	

Komponent	Uwzględniony?	Dlaczego
		przegląd wymaga śledzenia kandydatów, notatek, wierszy macierzy i znaczników ukończenia
Trwała pamięć semantyczna	ograniczona	ponowne użycie pojęć pomaga, ale agresywna trwałość grozi skażeniem
Interfejsy narzędziowe	tak	potrzebny jest świeży materiał dowodowy
Planowanie	tak, ale lekkie	zadanie ma zależności i umiarkowaną niepewność
Ponowne planowanie	tak	obserwacje często zmieniają to, które artykuły albo kategorie są ważne
Refleksja ogólna	ograniczona	jest użyteczna dopiero po sprawdzeniach niosących materiał dowodowy
Weryfikacja	tak	mapowanie wsparcia jest centralne dla tego zadania
Polityka zatrzymania lub przekazania	tak	ograniczona autonomia wymaga zasadnych zakończeń
Orkiestracja wieloagentowa	nie	jest zbędna dla tego ograniczonego kształtu zadania

Przykład krok po kroku: ślad wykonania

Poniższy ślad pokazuje jedno wiarygodne wykonanie bogatszego projektu.

t	widok celu	kontekst	stan zewnętrzny	obserwacja	działanie	wynik	decyzja o zatrzymaniu
0	złożenie ograniczonego zadania przeglądu	surowa prośba + założenia domyślne	puste magazyny kandydatów, notatek i macierzy	brak	think	zainicjalizowano <code>G_review</code> ; załadowano checkliście	kontynuuj
1	ustalenie, czy zakres jest jasny	<code>G_review</code> + testy niejednoznaczności	brak doprecyzowania	fraza „dla studentów” może oznaczać dwie poddziedzin	ask	użytkownik doprecyzowuje: uwzględnij asystentów do nauki i tutoringu tylko wtedy, gdy pokazują cytowania albo cytaty w końcowych odpowiedziach	kontynuuj
2	zebranie kandydackich źródeł	doprecyzowane <code>G_review</code>	pusta historia zapytań	wyszukiwanie zwraca 18 kandydatów	act	zapisano listę kandydatów;	kontynuuj

t	widok celu	kontekst	stan zewnętrzny	obserwacja	działanie	wynik	decyzja o zatrzymaniu
		+ plan początkowy		, wiele ogólnych systemów RAG		podniesiono flagę zbyt szerokich wyników	
3	dopracowanie wyszukiwania i preselekcji	flaga zbyt szerokich wyników + bieżący zbiór kandydatów	istnieje szkic reguły preselekcji	abstrakty sugerują, że cytowania na poziomie odpowiedzi są rzadsze niż proste retrieval	think	zaostrożono regułę wykluczenia ; plan zrewidowano w stronę weryfikacji cytatów i wzorców cytowań w tekście	kontynuuj
4	przeczytanie reprezentatywnych źródeł	zaakceptowani kandydaci + schemat notatek	zaktualizowano zbiór po preselekcji	pierwsze trzy artykuły ujawniają dwa różne wzorce grounding i jeden system słabo związany z tematem	act	zapisano ustrukturyzowane notatki; schemat macierzy zyskał kolumnę „sposób prezentacji materiału dowodowego”	kontynuuj
5	weryfikacja wsparcia dla tymczasowej syntezy	notatki + szkic tymczasowego podsumowania	rejestr sprzeczności i jest pusty	sprawdzenie cytowań wykrywa dwa słabo wsparte twierdzenia w szkicu	act	oznaczono niewspierane twierdzenia ; jedno zdanie złagodzone, jedno usunięto	kontynuuj
6	rozstrzygnięcie pozostałej niepewności	notatki + testy sprzeczności	cztery mocne notatki, dwie niepewne	jedno źródło nie zgadza się co do korzyści dla zaufania edukacyjnego	think	utworzono rejestr sprzeczności; do planu dodano sekcję o niepewności	kontynuuj
7	decyzja o ukończeniu	szkic podsumowania + macierz + mapa wsparcia	znaczniki sukcesu są prawie kompletne	kryteria sukcesu dla przeglądu architektur są spełnione; sekcja o wpływie na zaufanie pozostaje mieszana,	stop	wygenerowano końcowy przegląd z uwagą o niepewności i bez nadmiernych twierdzeń	zatrzymanie pełne

t	widok celu	kontekst	stan zewnętrzny	obserwacja	działanie	wynik	decyzja o zatrzymaniu
				ale jawnie oznaczona			

Ten ślad jest celowo ograniczony. Nie próbuje przeczytać wszystkiego. Pokazuje spójne wykonanie, którego działania można uzasadnić na podstawie `G_review`, `S_t`, obserwacji narzędziowych i polityki zatrzymania.

Przykład w pseudokodzie: obroniona bogatsza pętla

```

procedure RUN_LITERATURE_REVIEW(request):
  G <- compile_task_spec(request)
  S <- init_state(G)
  P <- init_light_plan(G)
  M <- load_procedure_templates()

  while true:
    O <- observe_latest(S)
    C <- build_context(G, S, P, O, M)
    A <- choose_next_move(C)          # think | ask | act | stop

    if A == ask:
      user_reply <- request_clarification(C)
      S <- update_with_user_reply(S, user_reply)
      P <- maybe_replan(P, S)

    else if A == act:
      result <- invoke_tool_or_check(C)
      S <- update_with_result(S, result)
      if replan_triggered(P, S, result):
        P <- revise_plan(P, S)

    else if A == think:
      S <- update_control_state(S, C)
      if replan_triggered(P, S, O):
        P <- revise_plan(P, S)

    else:
      return finalize_or_handoff(G, S, P)

  feedback <- run_support_contradiction_completion_checks(S, G)
  S <- apply_feedback(S, feedback)

  if should_stop(G, S, feedback):
    return finalize_or_handoff(G, S, P)

```

Ta pętla nadal pozostaje pojęciowo niewielka. Dodatkowa złożoność nie leży w komplikacji algorytmicznej, lecz w liczbie rozróżnień architektonicznych, które zachowuje osobno.

Minimalnie wystarczająca wersja a bogatsza wersja

Różnicę między dwoma projektami końcowymi warto wypowiedzieć wprost.

Wymiar	Minimalnie wystarczająca	Bogatsza, obroniona
Pamięć	stan lokalny dla wykonania plus szablon procedury	dodaje ograniczoną epizodyczną historię zapytań i starannie opracowane procedury wielokrotnego użytku
Planowanie	jedna początkowa lista kontrolna	jawne wyzwalacze ponownego planowania
Informacja zwrotna	sprawdzenie wsparcia i sprawdzenie ukończenia	dodaje rejestr sprzeczności i bogatszy pakiet informacji zwrotnej
Przekazanie	proste zatrzymanie z zastrzeżeniem	jawny pakiet przekazania z nierozwiązanymi kwestiami i dalszymi krokami
Ryzyko	może powtarzać słabe zapytania albo za słabo obsługiwać niezgodności	większa odporność, ale też większa złożoność zarządzania

Bogatsza wersja jest uzasadniona w przykładzie przewodnim, ponieważ zadanie często korzysta z ponownego użycia zapytań, śledzenia sprzeczności i ustrukturyzowanej obsługi nierozstrzygniętych wyników. Wersja minimalna pozostaje jednak poprawnym projektem dla węższego zadania przeglądownego.

Przewidywany tryb awarii i konkretna rewizja

Mocna notatka projektowa o architekturze powinna przewidywać, gdzie problem nadal jest prawdopodobny.

Przewidywany problem

Bogatsza wersja może nadal zbyt słabo obsługiwać **prawie zduplikowany materiał dowodowy**. Załóżmy, że kilka źródeł cytuje ten sam bazowy benchmark albo to samo twierdzenie. System może potraktować je jako niezależne wsparcie i przecenić poziom konsensusu.

To jest problem strukturalny, a nie tylko problem formatu cytowania. Architektura śledzi źródła i sprzeczności, ale nie zależność materiału dowodowego.

Rewizja

Dodaj do każdej notatki lekkie **pole rodowodu materiału dowodowego**:

- czy twierdzenie jest bezpośrednio obserwowane w artykule,
- czy zostało przejęte z innego cytowanego źródła,
- czy też powtarza się z powszechnie raportowanego benchmarku.

Następnie zrewiduj sprawdzenie wsparcia tak, aby wiele notatek opartych na tym samym bazowym źródle twierdzenia nie liczyło się automatycznie jako niezależne wsparcie.

To dobra rewizja końcowa, ponieważ zachowuje ograniczony charakter systemu, a zarazem wzmacnia dyscyplinę analizy błędów.

Audyt trybów awarii w projekcie końcowym

Lekka ocena strukturalna z książki wraca w tym końcowym audycie.

Obszar strukturalny	Bieżąca obrona	Resztkowa podatność
Reprezentacja celu	jawne <code>G_review</code>	ukryte oczekiwania użytkownika nadal mogą pozostać
Stan	jawne pola kandydatów, preselekcji, notatek, macierzy i ukończenia	słabe przycinanie może zagrazić stan
Pamięć	selektywna i ograniczona	nawet starannie opracowane kiedyś pojęcia mogą z czasem przeciekać do niewłaściwych zadań
Planowanie	lekkie plus ponowne planowanie	nadal możliwe jest nadaktywne ponowne planowanie
Działanie i użycie narzędzi	jawne kontrakty	wyniki narzędzi nadal mogą być zaszumione albo niepełne
Grounding	mapowanie wsparcia i sprawdzenia sprzeczności	zależność materiału dowodowego może być niedoszacowana
Zatrzymanie lub przekazanie	jawne progi i pakiet przekazania	progi pozostają zależne od zadania i od osądu

Najważniejsze nie jest to, że projekt stał się odporny na każdy problem. Najważniejsze jest to, że problemy można teraz przewidywać w kategoriach architektonicznych.

Ścieżki specjalizacji wykraczające poza tę książkę

Ta synteza końcowa zamyka główny łuk pojęciowy książki, ale jednocześnie wskazuje kierunki dalszej specjalizacji.

Czytelnik, który chce pójść głębiej, może specjalizować się w:

- **systemach silnie opartych na pamięci**, w tym w projektowaniu retrieval i zarządzaniu długim kontekstem,
- **planowaniu**, w tym w bardziej formalnych metodach przeszukiwania albo dekompozycji,
- **ocenie**, w tym w projektowaniu benchmarków i metodologii ilościowej,
- **interakcji człowiek-AI**, w tym w projektowaniu doprecyzowania i przekazania,
- **reinforcement learning**, jeśli chodzi o uczenie poprawy polityki,
- **information retrieval**, jeśli chodzi o wnętrze wyszukiwania i rangowania,
- **planowaniu symbolicznym**, jeśli chodzi o formalne operatory i planery,
- **inżynierii produkcyjnej**, jeśli chodzi o wdrożenie, obserwowalność, ład i operations.

To realne kolejne kroki. Nie są one warunkami wstępnymi zrozumienia kręgosłupa architektonicznego zbudowanego w tej książce.

Aktualizacja wspólnego artefaktu

Skumulowane artefakty tworzą teraz jedną spójną ograniczoną architekturę:

- specyfikację zadania `G_review`,
- schemat śladu i inwentarz stanu,
- politykę pamięci,
- kontrakty narzędziowe,

- plan i reguły ponownego planowania,
- reguły informacji zwrotnej i weryfikacji,
- politykę zatrzymania lub przekazania,
- macierz porównania architektur,
- końcowe projekty minimalny i bogatszy.

Co zawodzi, gdy tego komponentu brakuje albo używa się go niewłaściwie?

W projekcie końcowym najczęstszym nadużyciem jest **kopiowanie szablonu**. Projektant wkleja pamięć, planowanie, refleksję i narzędzia bez powodu wynikającego z zadania.

Dla przykładu przewodniego trzy pominięcia byłyby szczególnie szkodliwe:

1. **Brak polityki zatrzymania lub przekazania.** System nadal by szukał albo nadmiernie twierdził przy skąpym materiale dowodowym.
2. **Brak polityki pamięci.** Notatki i wiedza proceduralna zlałyby się ze sobą.
3. **Brak ścieżki weryfikacji.** Architektura mogłaby wytwarzać dopracowaną, ale słabo wspartą syntezę.

Dlatego dyscyplina projektu końcowego obejmuje jednocześnie włączanie i pomijanie. Dobry projekt mówi, czego nie ma, i dlaczego.

Tryby awarii, ograniczenia i błędne przekonania

- **Kopiowanie diagramu stosu bez dopasowania do zadania.** Architektura musi odpowiadać przed kształtem zadania.
- **Uwzględnianie każdego możliwego komponentu.** Więcej pudełek nie oznacza lepszego sterowania.
- **Zapominanie o zatrzymaniu albo przekazaniu w projekcie końcowym.** Wiele skądinąd dobrych diagramów właśnie tutaj zawodzi.
- **Pokazywanie diagramu bez śladu wykonania.** Obrona architektury powinna umieć wyjaśnić jedno ograniczone wykonanie od prośby do zatrzymania.

Podsumowanie rozdziału

Ten rozdział zamienił rozróżnienia z całej książki w obroniony projekt ograniczonego asystenta do przeglądu literatury. Końcowa architektura nie jest zdefiniowana przez narzędzia dostawców, repozytoria ani decyzje wdrożeniowe. Definiują ją reprezentacja celu, pętla sterowania, umiejscowienie stanu, polityka pamięci, kontrakty narzędziowe, planowanie, grounding, informacja zwrotna oraz logika zatrzymania albo przekazania. Synteza końcowa pokazała też, że dobra notatka projektowa przewiduje potencjalne problemy i odpowiednio rewiduje architekturę.

Na centralne pytanie kręgosłupa tej książki można teraz odpowiedzieć jasno: **model generatywny staje się ukierunkowanym na cel systemem agentowym wtedy, gdy zostaje osadzony w architekturze, która jawnie reprezentuje zadanie, niesie właściwy stan, adaptacyjnie wybiera działania na podstawie obserwacji, opiera**

swoje twierdzenia na materiale dowodowym i wie, kiedy się zatrzymać albo przekazać sprawę dalej.

Źródła i dalsza lektura

- (Yao et al., 2022; Schick et al., 2023; Nakano et al., 2021) o konkretnych systemach łączących generowanie z działaniem i zbieraniem materiału dowodowego.
- (Park et al., 2023; Packer et al., 2023) o ideach architektonicznych zorientowanych na pamięć.
- (Madaan et al., 2023; Shinn et al., 2023) o metodach zorientowanych na refleksję i ich ograniczeniach.
- (Xi et al., 2023) jako szeroka mapa badań nad agentami opartymi na LLM, wykraczająca poza rozwinięty tu projekt ograniczony.

Co ten rozdział dodaje do architektury

- **Dodano:** pełną obronioną architekturę i ślad, który czyni ją operacyjną.
- **Zaadresowany problem:** izolowana wiedza o komponentach bez syntezy całego systemu.
- **Nowe ryzyko:** nadmierna pewność, że jeden projekt końcowy da się przenieść bez zmian na każde zadanie.

Bibliografia

1. **Saleema Amershi, Daniel S. Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, et al.** (2019). "Guidelines for Human-AI Interaction." Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. [URL](#)
2. **Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al.** (2020). "Language Models are Few-Shot Learners." Advances in Neural Information Processing Systems 33. [URL](#)
3. **Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, et al.** (2022). "Inner Monologue: Embodied Reasoning through Planning with Language Models." arXiv preprint arXiv:2207.05608. [URL](#)
4. **Wenlong Huang, Pieter Abbeel, Deepak Pathak, Igor Mordatch, et al.** (2022). "Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents." arXiv preprint arXiv:2201.07207. [URL](#)
5. **Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al.** (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems 33. [URL](#)
6. **Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, et al.** (2023). "Self-Refine: Iterative Refinement with Self-Feedback." arXiv preprint arXiv:2303.17651. [URL](#)
7. **Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schutze** (2008). *Introduction to Information Retrieval*. Cambridge University Press. DOI: [10.1017/CBO9780511809071](#) [URL](#)
8. **Jacob Menick, Maja Trebacz, Vladimir Mikulik, John Aslanides, Francis Song, Martin Chadwick, et al.** (2022). "Teaching Language Models to Support Answers with Verified Quotes." arXiv preprint arXiv:2203.11147. [URL](#)
9. **Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, et al.** (2021). "WebGPT: Browser-assisted Question-Answering with Human Feedback." arXiv preprint arXiv:2112.09332. [URL](#)
10. **Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, et al.** (2023). "MemGPT: Towards LLMs as Operating Systems." arXiv preprint arXiv:2310.08560. [URL](#)
11. **Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens** (2000). "A Model for Types and Levels of Human Interaction with Automation." IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans. Vol. 30(3). pp. 286-297. DOI: [10.1109/3468.844354](#) [URL](#)
12. **Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, Michael S. Bernstein, et al.** (2023). "Generative Agents: Interactive Simulacra of Human Behavior." Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. [URL](#)
13. **Stuart Russell and Peter Norvig** (2021). *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson. [URL](#)

14. **Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, et al.** (2023). “Toolformer: Language Models Can Teach Themselves to Use Tools.” *Advances in Neural Information Processing Systems* 36. [URL](#)
15. **Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao, et al.** (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning.” arXiv preprint arXiv:2303.11366. [URL](#)
16. **Richard S. Sutton and Andrew G. Barto** (2018). *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press. [URL](#)
17. **Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, et al.** (2023). “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models.” arXiv preprint arXiv:2305.04091. [URL](#)
18. **Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, et al.** (2023). “The Rise and Potential of Large Language Model Based Agents: A Survey.” arXiv preprint arXiv:2309.07864. [URL](#)
19. **Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, et al.** (2022). “ReAct: Synergizing Reasoning and Acting in Language Models.” arXiv preprint arXiv:2210.03629. [URL](#)
20. **Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, et al.** (2023). “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” arXiv preprint arXiv:2305.10601. [URL](#)